

JOI 春合宿 2012 Day1 ビルの飾り付け 2

秋葉 拓哉

Building (JOI 春合宿 '07)

Building: ビルの飾りつけ

input ファイル “building.in”

output 標準出力

ソースファイル building.c/building.cpp/Building.java

時間制限 1 秒 / データ

国際情報オリンピックが日本で開かれることとなり、世界の選手達を歓迎するため、空港から宿泊施設までの道沿いにある高層ビルを飾りつけることにした。ある著名なデザイナーにデザインを依頼したところ、飾りつけに利用するビルは、空港から宿泊施設に向けて高くなっていく必要があると言った。つまり、飾りつけに利用するビルの高さを、空港に近いものから順に $h_1, h_2, h_3, \dots$ とおくと、 $h_1 < h_2 < h_3 < \dots$ となっていなければならない。

できるだけ飾りつけを華やかにするため、飾りつけに利用するビルの数をできるだけ多くしたい。入力として全てのビルの高さが与えられたとき、利用することのできるビルの数の最大

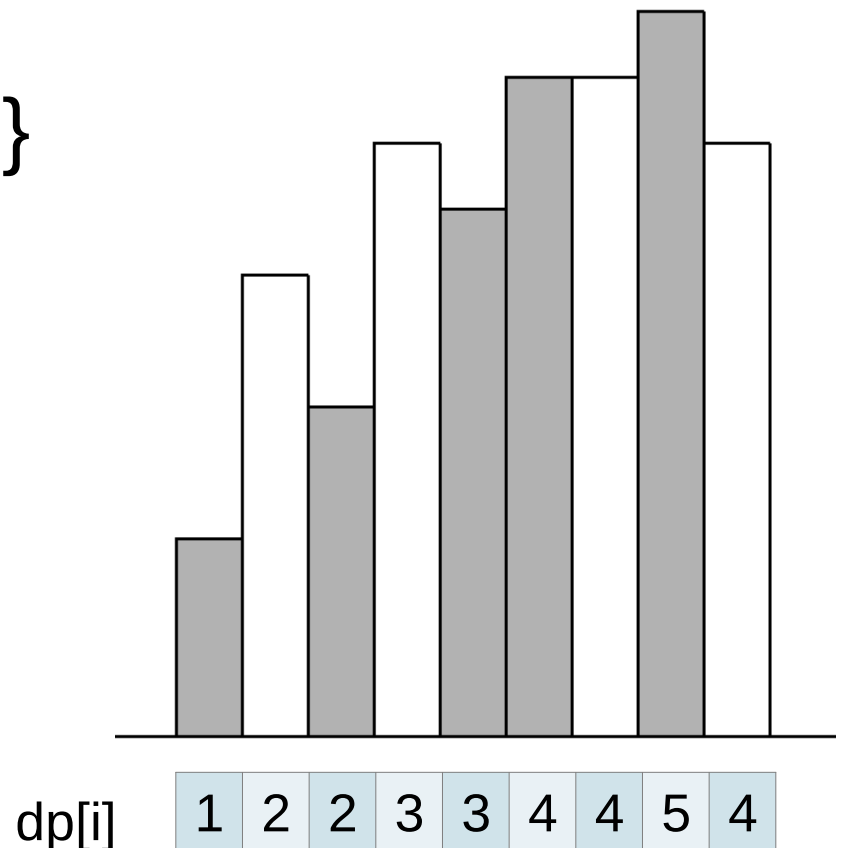
問題概要

- 最長増加部分列を求めよ
- ただしツリー上

「ツリー上のパスの部分列であって増加しているものの最長のものを求めよ」

普通の最長増加部分列

- ナイーブな DP (以下 *LIS-DP1* と呼ぶ)
- $dp[i] := \max \{dp[j] + 1\}$
– ただし $j < i, a[j] < a[i]$
- 某本 P. 64



普通の最長増加部分列

- 高速な DP (以下 *LIS-DP2* と呼ぼう)
- $dp_i[k] :=$ 場所 i までで長さ k の列を作る時の最後の数字の最小値
- ナイーブに更新すると $O(n^2)$
- 更新位置を二分探索で求めれば $O(n \log n)$
- 某本 P. 65

i	0	1	2	3	4
dp[i]	∞	∞	∞	∞	∞

↓ 4を挿入

i	0	1	2	3	4
dp[i]	4	∞	∞	∞	∞

↓ 2を挿入

i	0	1	2	3	4
dp[i]	2	∞	∞	∞	∞

↓ 3を挿入

i	0	1	2	3	4
dp[i]	2	3	∞	∞	∞

↓ 1を挿入

i	0	1	2	3	4
dp[i]	1	3	∞	∞	∞

↓ 5を挿入

i	0	1	2	3	4
dp[i]	1	3	5	∞	∞

ナイーブな解法 (10 点)

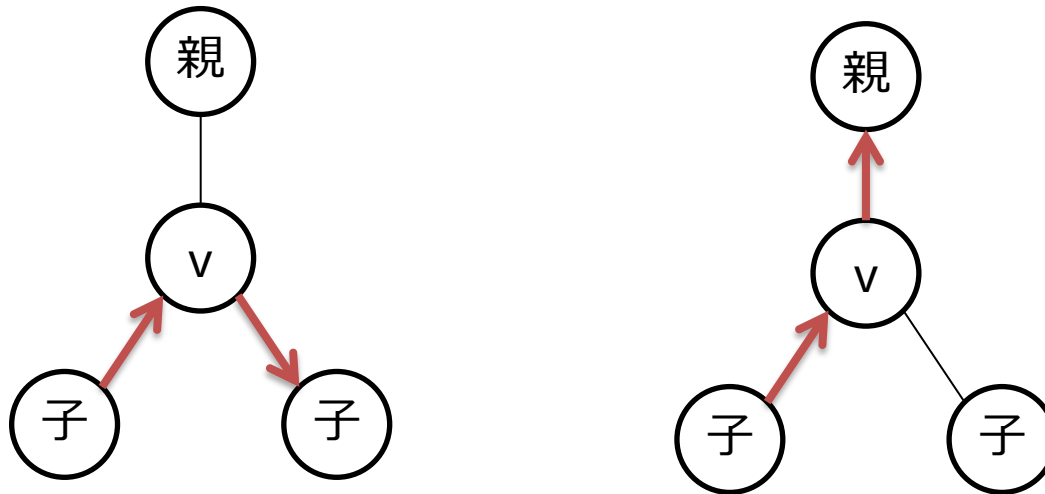
- $O(N^4) / O(N^3 \log N)$
 - 全部のパス N^2 個
 - 各パスについて $O(N^2) / O(N \log N)$ のDP
- $O(N^3)$
 - 開始位置 N 個全部試す, 再帰して木 DP
 - *LIS-DP1* と同様, 各位置を末尾にした際の最長の LIS 長を覚えておき DP

$O(n^2 \log n)$ の解法 (30 点)

- すべての開始位置を試す
- *LIS-DP2* をやりながら再帰
- 枝分かれは？
 - 再帰から戻るとき, 自分のやった変更を**巻き戻す**ようにすればよい
(そうすれば次の枝にもいける)

$O(n \log^2 n)$ の解法 (100 点)

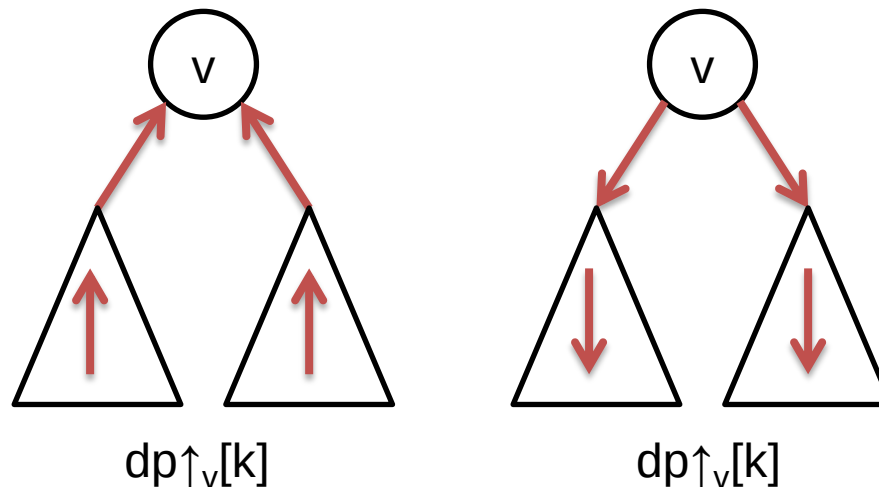
- 適当に根を決めて DFS, 木 DP
- あるノード v に関して, 答えの移動経路が v を通るとすると, 以下の 2 パターン



DP

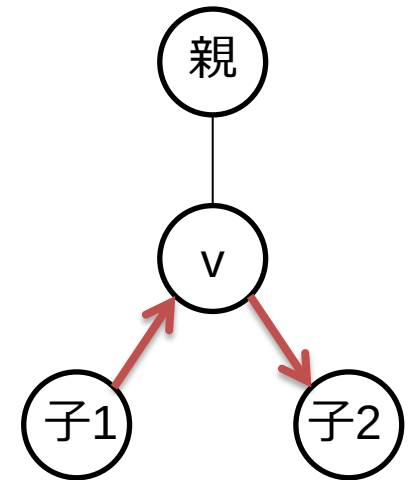
LIS-DP2 の DP 列的な物をボトムアップに計算

- $dp_{\uparrow v}[k] := v$ に向かって遡ってくる LIS で長さ k の物を作る際の最後の数の最小値
- $dp_{\downarrow v}[k] :=$ 逆 (Longest Decreasing Sequence)



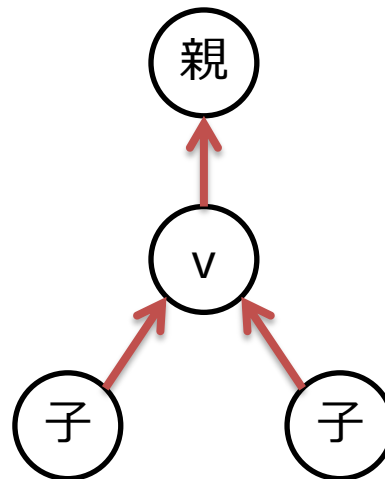
頂点 v での処理 1

- 右図みたいなパターンの分を計算
- $dp\uparrow_{子1}$ と $dp\downarrow_{子2}$ とかから求まる
- v を使う場合
 - $dp\uparrow_{子1}[i] < H[v] < dp\downarrow_{子2}[j] \rightarrow$ 長さ $i + 1 + j$ は作れる
- v を使わない場合
 - $dp\uparrow_{子1}[i] < dp\downarrow_{子2}[j] \rightarrow$ 長さ $i + j$ は作れる
 - $dp\uparrow$ か $dp\downarrow$ の片方を全部なめつつ, もう片方を二分探索すればよい



頂点 v での処理 2

- 上にいくために $dp \uparrow_v$ とかを計算
- $dp \uparrow_v[k] = \min(dp \uparrow_{子1}[k], dp \uparrow_{子2}[k])$
- $H[v]$ を挿入



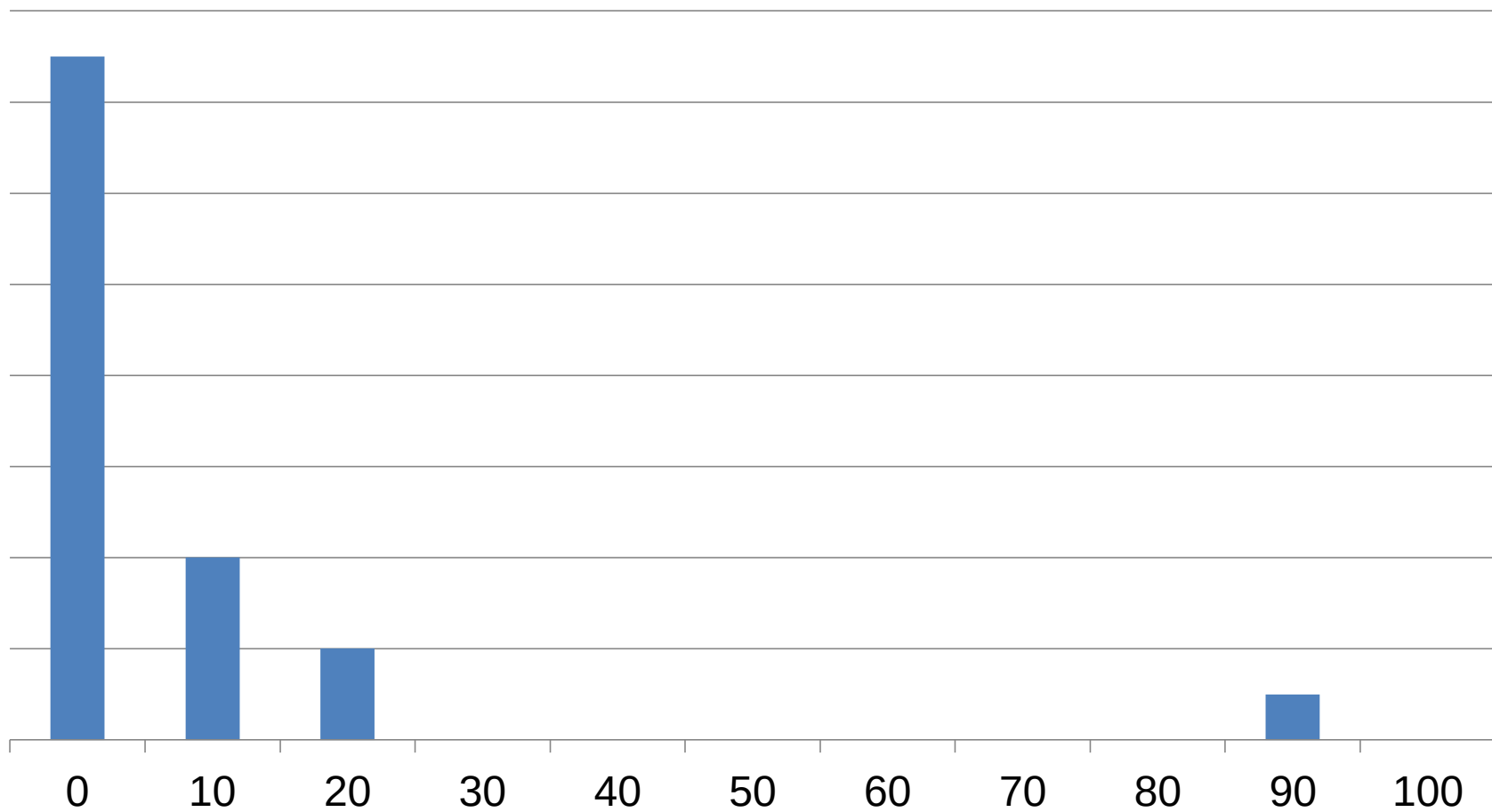
計算量

- これ毎頂点でやったらやばくない？
 - 毎回長さ $O(n)$ の DP 列舐めてたら $O(n^2)$!
- 少し工夫すれば実はやばくない
 - 短い方を舐めるようにすれば良い
 - これだけで $O(n \log n)$ になる
 - 舐められた時, 次にそいつが含まれる所のサイズは少なくとも 2 倍になる
(併合先のほうがでかい)
 - よって各要素は $O(\log n)$ 回しか舐められない.
(これはデータ構造をマージする一般的なテク)

...おわり

- 説明大変でしたが、基本的なテクの組合せ
- LIS の高速 DP
- 木上でのパスのための DP
- データ構造のマージ (的思想)

得点分布



Day1 Fish 解説

原ページが HTML 形式で公開されているため、原文のテキストと図を保持して PDF 化しています。

Source: <https://www2.ioi-jp.org/camp/2012/2012-sp-tasks/2012-sp-day1-fish-slides.html>

--問題--

N匹の魚の体長と色(赤, 青, 緑のいずれか)が与えられる.

ここから1匹以上を選んで飼うとき, 色の組み合わせは何通りあるか.

ただし, 体長が2倍以上違う魚はでかいのが小さいのを食う(cf. IOI'08)ので同時に飼えない.

例: 入力例1

体長 10 4 8 5

色 R G B B

--> 「赤1匹」「緑1匹」「青1匹」「赤と青1匹ずつ」「緑と青1匹ずつ」「青2匹」の6通り
(たとえば「赤と緑1匹ずつ」は無理: 体長10の魚は体長4の魚を食ってしまう)

--最も安直な解法--

魚の(空でない)部分集合(2^N-1 通りある)を全部試す --- 5点

--アイデア1--

入力例1だと魚が少なくて寂しいのでもうすこし魚を増やした次のような例で考えてみる.

名前 a b c d e f

体長 4 5 7 8 9 15

色 R B B G G R

- 魚a~cまではすべて同時に飼える --> 「赤1匹以下, 緑0匹以下, 青2匹以下」の組み合わせは全部作れる
- 魚b~eまではすべて同時に飼える --> 「赤0匹以下, 緑2匹以下, 青2匹以下」の組み合わせは全部作れる
- 魚c~eまでは...
- 魚d~fまでは...
- 魚e~fまでは...
- 魚f~fまでは...

これを利用すれば安直にやっても $O(N^4)$ で数えられる

もっとうまく数えたい

--アイデア2--

```
+-----+-----+
/      /      /|
+-----+-----+ |
/      /      /| +
+-----+-----+ ||
|      |      | + |
|      |      ||/ +
```

```

+-----+-----+ | / |
|         |         | + |
|         |         | / | +
+-----+-----+ | /
|         |         | +
|         |         | /
+-----+-----+

```

この図がなんだかお分かりいただけるであろうか。

これは「赤1匹以下，緑2匹以下，青1匹以下」の組み合わせすべて(12通り，ただし1匹も飼わない場合を含めた)を図示したものである。

```

      +-----+-----+
      /0,2,1/1,2,1/
+-----+-----+ |
/         /         / | +
+-----+-----+ | / |
|         |         | + |
|0,2,0|1,2,0| / | +
+-----+-----+ | / |
|         |         | + |
|0,1,0|1,1,0| / | +
+-----+-----+ | /
|         |         | +
|0,0,0|1,0,0| /
+-----+-----+

```

こういうことです(「赤がr匹，緑がg匹，青がb匹」をr,g,bと書いています)

求めたいのは，こういう直方体たちの和集合の体積（から1を引いたもの）である。

==直方体たちの和集合イメージ図==

```

      +-----+
      /         / |
      /         / |
+-----+ |
|         | +-----+
+-----+ | /         / |
/         / | /         / +-----+
/         / ;--+         / /         / |
+-----+ +-----+ +-----+ +
|         |         |         | /
|         |         |         | +-----+
|         |         |         | /
|         | +-----+
|         | /
+-----+

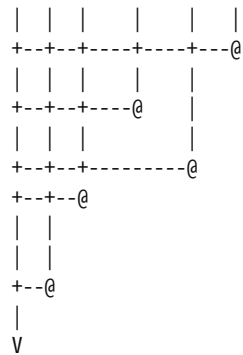
```

こういうものの体積を求めればよいので，各々の高さでの断面積がわかるといい。

```

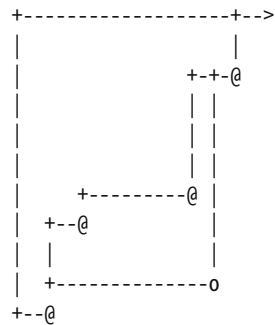
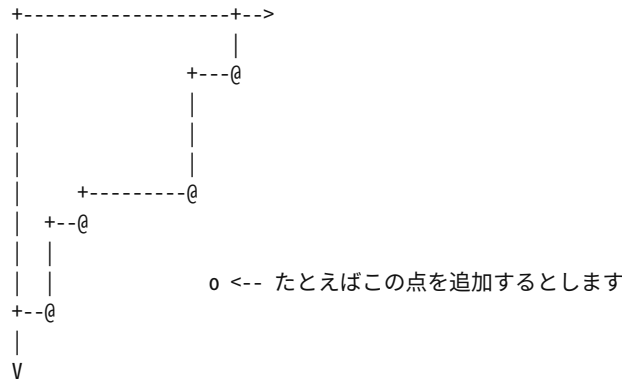
+---+---+---+---+---+---+--->

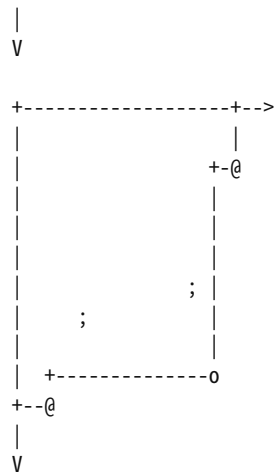
```



このような、長方形の和集合の面積が求められるとよい．たとえば $O(N \log N)$ で可能 --> $O(N^2 \log N)$

--アイデア3--
 アイデア2の方針をさらに改良することができる：
 断面を上から順に求めていくことにすると、求めたい断面はつねに直前に求めた断面に長方形を1個追加したものになる．
 --> そのような場合に対して面積の変化が爆速で計算できればよい





- 更新される区間の両端は，たとえば現在の頂点の座標たちを `std::set` などでもっておけば $O(\log N)$ で計算できる．
- 各々の長方形の頂点は1回 `set` に入れられたあと高々1回消される
--> $O(N \log N)$

点数分布

100
100
40
30
30
15
15
15
15
15
10
5
0
0
0
.....

日本情報オリンピック旗 JOI Flag



今西 健介
@japlj

問題

- 再帰的に定義された JOI 旗
 - 4つに分割して J, O, I, JOI旗
- 大きさ $2^K \times 2^K$ のマス目
- あからじめ N マスが埋まっている
 - $1 \leq K \leq 30, 1 \leq N \leq 1\,000$
- 正しいJOI旗に修正する最小コストは？

基本的な考え方

JOI旗は**再帰的に**定義されている
(レベル K の旗はレベル $K-1$ の旗を含む)

基本的な考え方

JOI旗は**再帰的に**定義されている
(レベル K の旗はレベル $K-1$ の旗を含む)



定義通りに4つに分割して
再帰的に答を計算できる

簡単な場合

- マス目を J (または O, I) のみで埋める場合
 - さすがに簡単
- たとえば J のみで埋めるなら O と I の個数を数えれば良い
 - 数え方はあとで

40%解法 その1

- 実際に $2^K \times 2^K$ ぶんのメモリをとる
 - 空間計算量 $O(4^K)$
- 文字を数えるときは実際にマス目を見ていけばよい
 - 時間計算量 $O(K 4^K)$

40%解法 その2

- さすがに無駄が多すぎ
 - 初めから埋まっているマスは 1 000 個以下
 - 文字の数え上げは $O(N)$ ができる
- それでも40点
 - 必要な再帰呼び出しの回数は $O(4^K)$ 回

100%解法 その1

- マス目の個数に比べて N は小さい！
 - $4^K \leq 4^{30} = 1\ 152\ 921\ 504\ 606\ 846\ 976$
 - $N \leq 1\ 000$
 - スカスカ
- 初めから埋まっているマスが無ければ修正に必要なコストは明らかに 0
 - 再帰呼び出しが枝刈りできる！

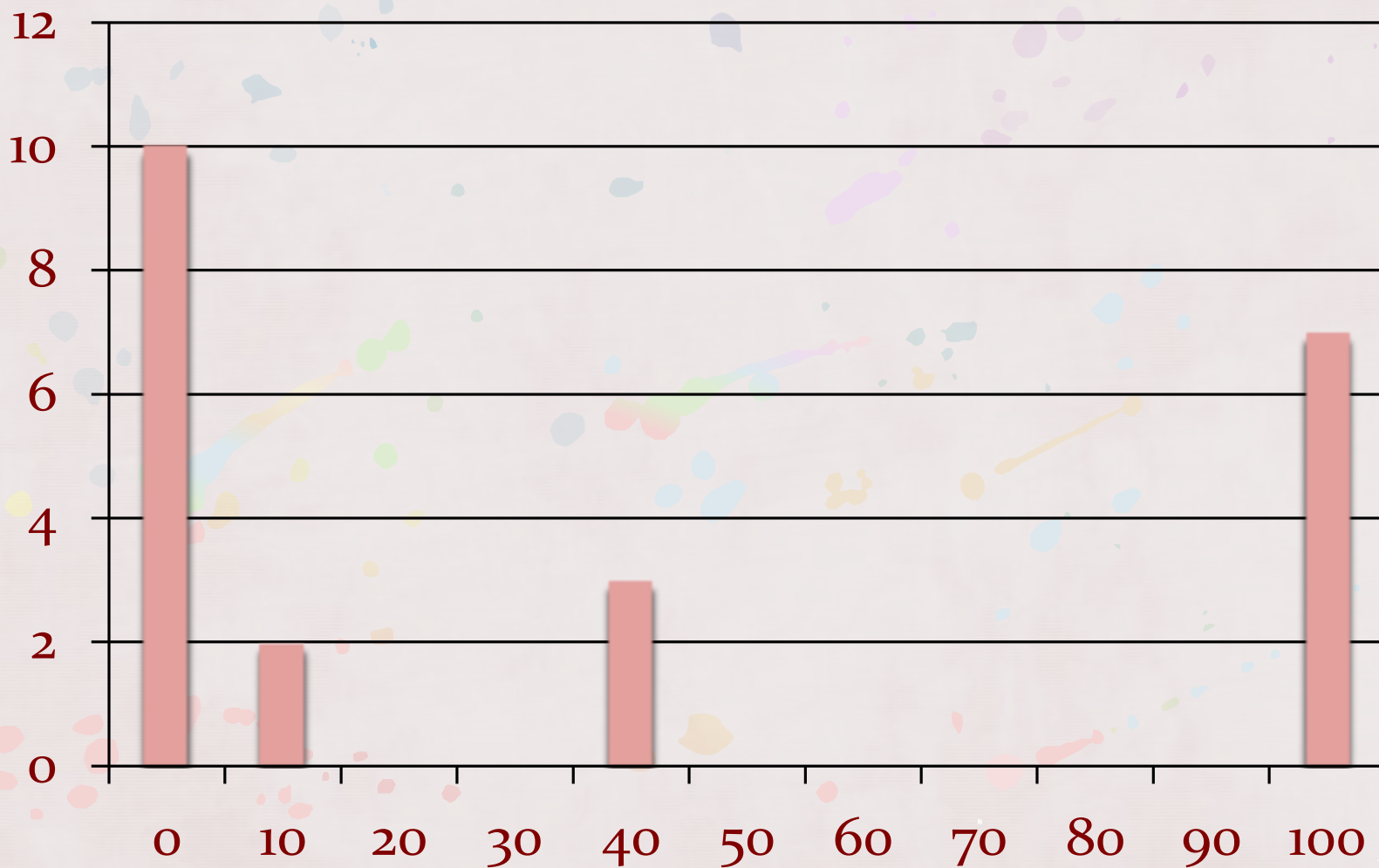
100%解法 その2

- 枝を刈ると $O(KN)$ 回の再帰呼び出し
 - 各マスが影響を与える再帰呼び出しは K 個
- 文字を $O(N)$ で数えると全体で $O(KN^2)$
 - $KN^2 \leq 3 \times 10^7$
 - 1.5秒あれば間に合う

おまけ

- 数え方によってはもっと効率良くなる
 - 全体で $O(KN + N^2)$
 - 全体で $O(KN)$

得点分布



とこるで

競技1日目を終えて

- 競技初日が終わりました
 - 結果はどうあれまだ3日あります
 - 点数にしてまだ 900 点もあります
 - 諦めてはいけない(戒め)
- 残り3日で実力を最大限発揮するために競技上の注意があります

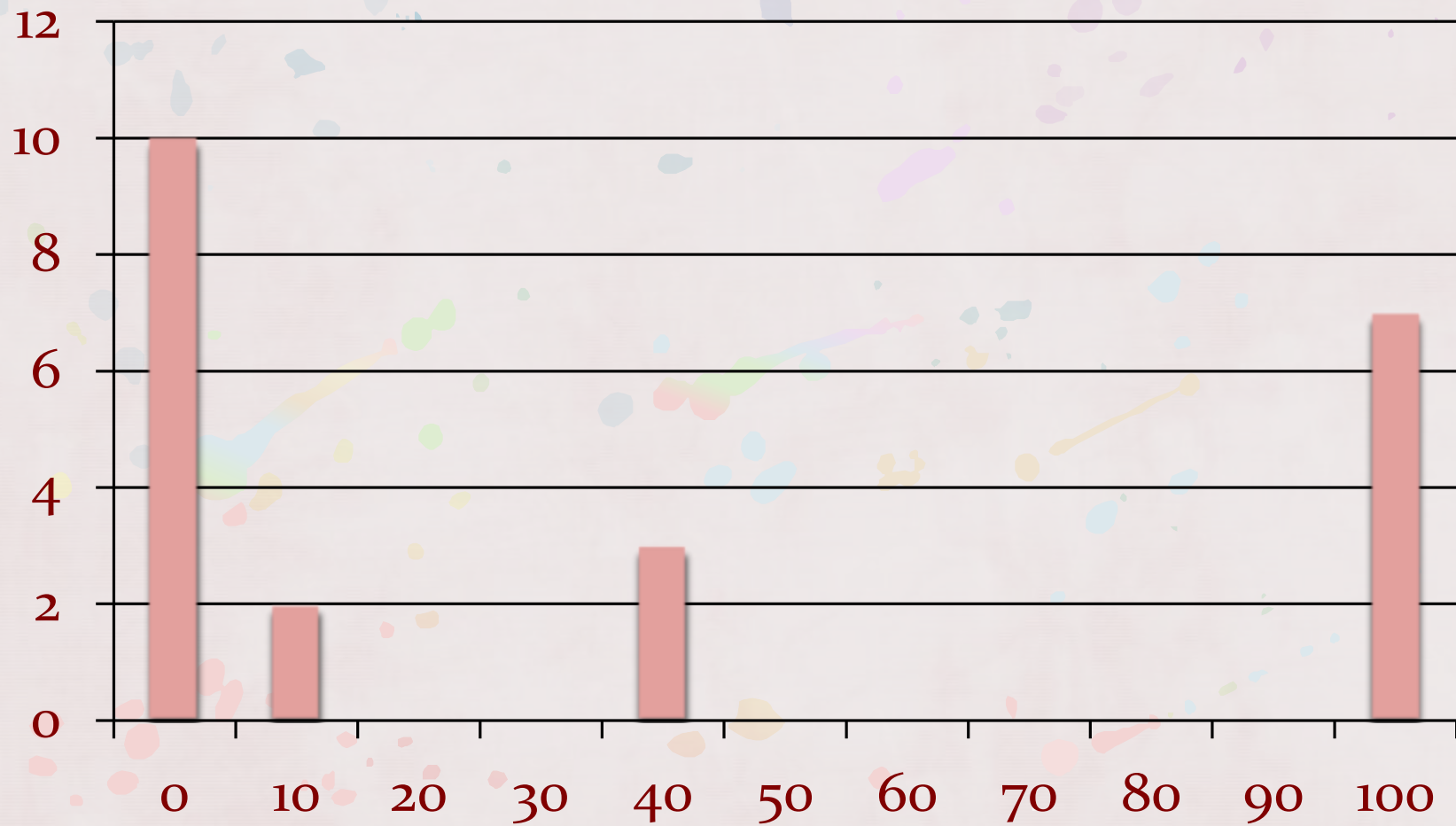
問題を読む

- 問題は**全部**読みましょう
 - 難易度順に並んでいるとは限らない
 - 「簡単な問題を読んでいなかった」は痛い
- **Overview Sheet** もちゃんと見ましょう
 - 時間制限やメモリ制限から問題の雰囲気を探ることができるかもしれない？

アルゴリズム

- 明記された**部分点**に頼る
 - 得点になる
 - 部分点のアルゴリズムがヒントになる
 - 入力のどのパラメータが重要か
- 明記された**部分点**に頼りすぎない
 - 実装時間との兼ね合い
 - 小さいサイズだと可能な操作が多い
 - 自然な部分点解法がわからないまま満点解法がわかってしまう, なんてことも

得点分布(再掲)



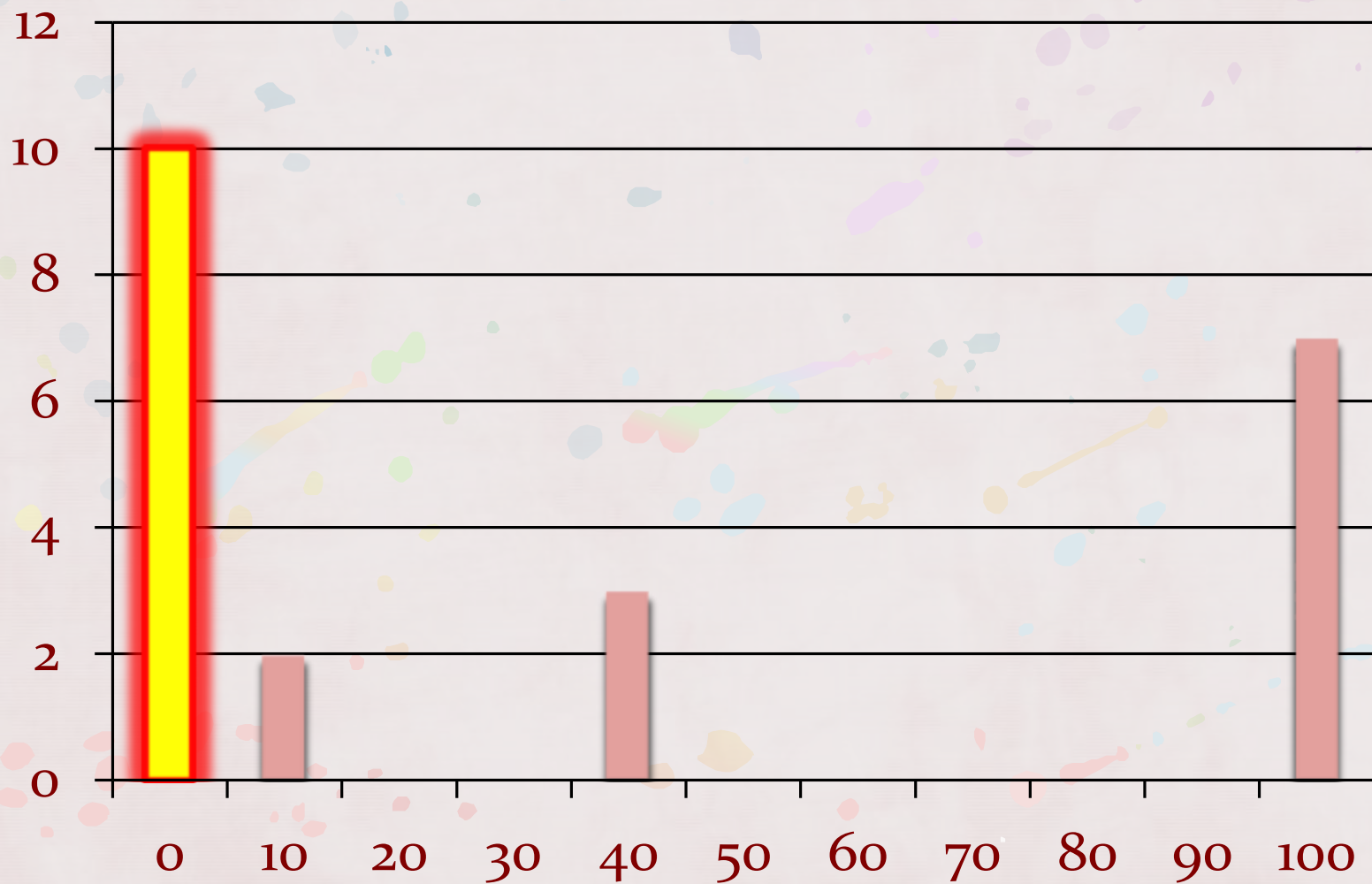
フィードバックについて

- この問題は完全フィードバックでした
 - 提出したら点数まるわかり
 - バグっててもデバッグし放題(?)
- 【重要】 完全フィードバックがある問題は

フィードバックについて

- この問題は完全フィードバックでした
 - 提出したら点数まるわかり
 - バグっててもデバッグし放題(?)
- 【重要】 完全フィードバックがある問題は
簡単
(少なくともその日の問題セットの中では)

得点分布(再再掲)



まとめ

完全フィードバック問題は比較的簡単

解く順番は自由ですが
とれる点を落とさないように

2日目からも頑張ってください

joi2012-sp-day2

テレビ放送

(Broadcasting)



Masaki Hara
@qnighy

2歳のとき強く頭を打ち、算数に目覚める。
9歳のとき強く頭を打ち、プログラミングに目覚める。
10歳のとき強く頭を打ち、Javaに目覚める。
14歳のとき強く頭を打ち、アニメに目覚める。
15歳のとき強く頭を打ち、アルゴリズムに目覚める。
17歳のとき強く頭を打ち、形式的定理証明に目覚める

<https://twitter.com/#!/qnighy/status/25315686027>

問題

- 平面上にN個の点がある
- K個の円を配置(場所・大きさは任意)し、それらの点をカバーする
- それぞれの円について、半径の二乗に比例するコストがかかる
- コストの合計値を小さくしたい
- 出力のみ提出

問題

- 平面上にN個の点がある
- K個の円を配置(場所・大きさは任意)し、それらの点をカバーする
- それぞれの円について、半径の二乗に比例するコストがかかる
- コストの合計値を小さくしたい
- 出力のみ提出

出力のみ問題！

- 一般的なアドバイスから

出力のみ問題！ (1)

- 入力を見ながらコーディングできる
 - 入力データの特徴は最大限活用しよう
 - そのためにビジュアライザが欲しい時もある

出力のみ問題！ (2)

- プログラムを提出する必要がない
 - いくつプログラムを使ってもよい
 - 手作業をしてもよい
 - 場合によっては有用(e.g. IOI2010 Maze)
 - ただし、Wrong Answerに注意

出力のみ問題！ (3)

- プログラムを提出する必要がない
 - 1秒以内に実行する必要はない
 - 長い時間かけて良い答えを得る
 - C/C++で書く必要もない
 - Ubuntuなら、bash, Perl, Pythonは確実に入っています
 - ブラウザでJavaScriptを動かすこともできます
 - ただし、コンテストの禁止事項には注意

出力のみ問題！ (4)

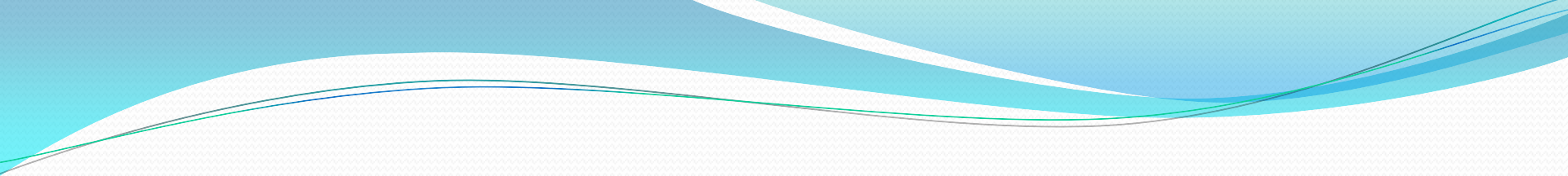
- 何回でも挑戦できます
 - 出力のみ問題では、手動でのパラメーター調整が重要になります。
 - 時間をかけるほど良い解が出るプログラムはgood
 - パラメーター調整が簡単にできるようにするとgood

出力のみ問題！ (5)

- ※5時間コンテストです！！
 - 出力のみ問題にこだわりすぎないように
 - 無制限に時間を消費するので、最後にするのが無難？
- 複雑な戦略(焼きなましやGA)は、5時間コンテストではあまり力を発揮しないかもしれません
 - パラメーター調整が難しいし。
 - 山登り法や「焼きっぱなし」がおすすめです

出力のみ問題！

- 入力データの特徴を活用する
- いくつプログラムを作ってもよい
- 好きなだけ実行してよい



閑話休題

問題

- 平面上に N 個の点がある
- K 個の円を配置(場所・大きさは任意)し、それらの点をカバーする
- それぞれの円について、半径の二乗に比例するコストがかかる
- コストの合計値を小さくしたい

入力の性質

- ビジュアライザを書く
 - JavaScript+Canvasで絵を描けます

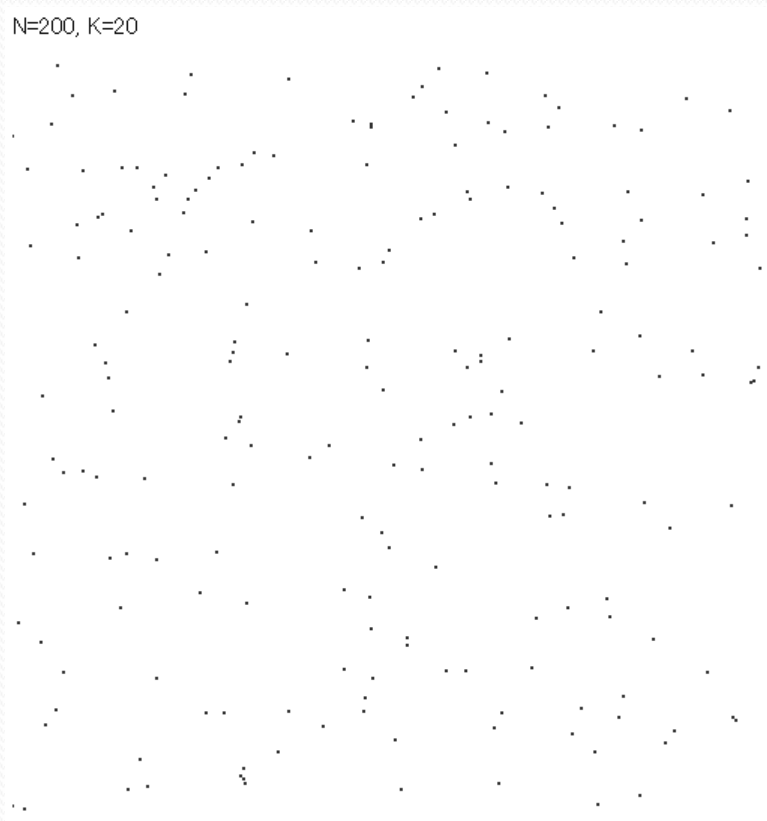
入力の性質

- ビジュアライザを書く

```
int N, K; fscanf(infile, "%d%d", &N, &K);<
static int A[500], B[500];<
for(int i = 0; i < N; i++) {<
    fscanf(infile, "%d%d", A+i, B+i);<
}<
printf("<p>N=%d, K=%d</p>¥n", N, K);<
printf("<canvas id='vis' width='%d' height='%d'></canvas>¥n", R, R);<
puts("<script>");<
puts("var vis = document.getElementById('vis');");<
puts("var ctx = vis.getContext('2d');");<
for(int i = 0; i < N; i++) {<
    puts("ctx.beginPath();");<
    printf("ctx.arc(%d, %d, 1, 0, 360);¥n", A[i]*R/1000000, B[i]*R/1000000);<
    puts("ctx.fill();");<
}<
puts("</script>");<
```

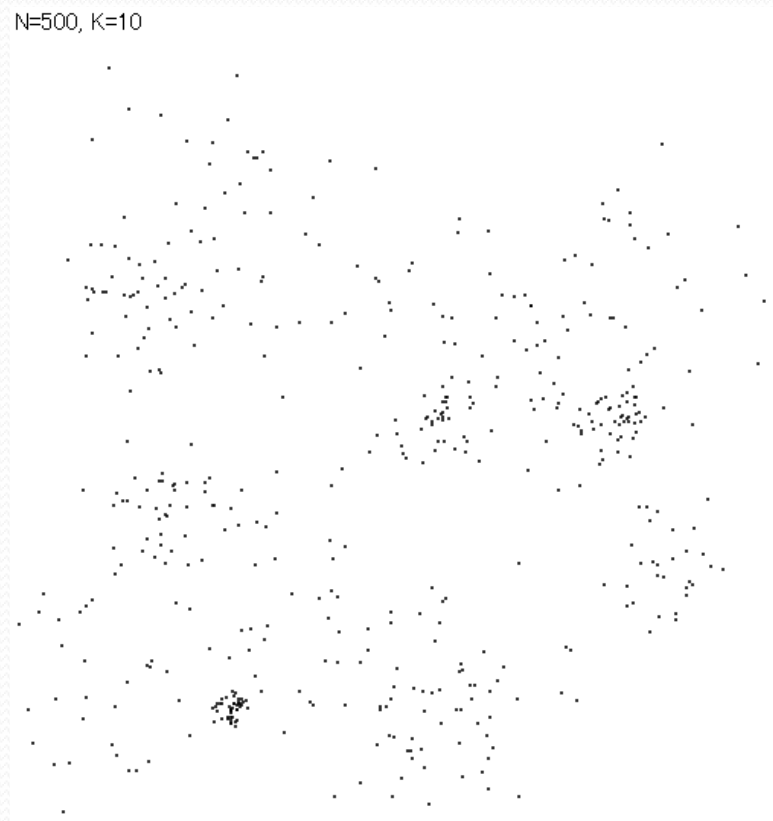
入力の性質

N=200, K=20



01.txt

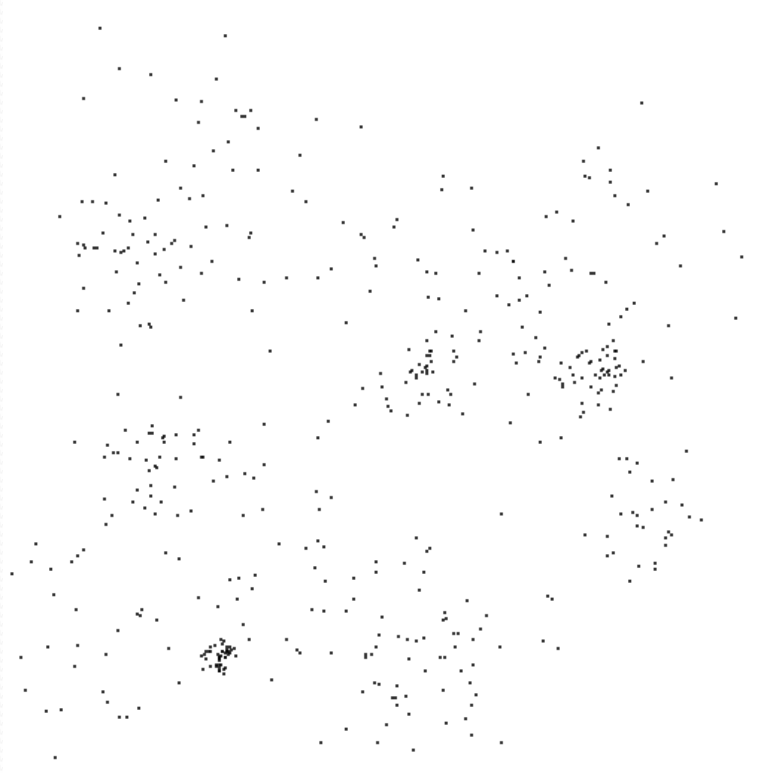
N=500, K=10



02.txt

入力の性質

N=500, K=10



03.txt

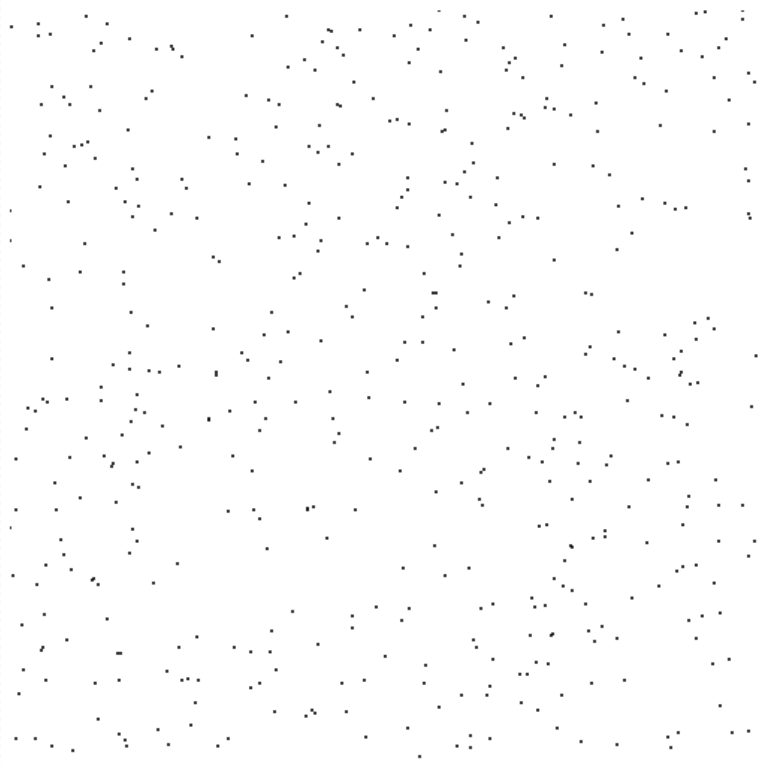
N=500, K=15



04.txt

入力の性質

N=500, K=30



05.txt

入力の性質: 考察

- 入力によって性質が違う
 - 均等なもの (o1, o5)
 - 若干ばらつきのあるもの(o2, o3)
 - 狙ってるとしか思えないもの(o4)
- アルゴリズムによって得手不得手がありそう

考察

- 考察: カバーしたい点集合に対して、最小包含円を決めることができる
- →以下の2つの問題に分かれる
 - K個の点集合を作る
 - それぞれの点集合について、最小包含円を求める

最小包含円

- 与えられた点を全て含む円で、最も半径が小さいもの
- $O(n^3)$
- 平均 $O(n)$
- その他 - 貪欲法やエセ最小包含円

最小包含円 - $O(n^3)$

- $n \leq 2$ のときは自明
- $n = 3$ のとき
 - 鈍角三角形の場合...一番長い辺が直径
 - 鋭角三角形の場合...外接円を考える
- $n \geq 3$ のとき
 - 全ての三角形を考えて、その最小包含円のうち最も大きいものを選べばよい

最小包含円 – 平均 $O(n)$

- $\text{min_disk}(P, Q)$: 点集合 P と Q を含む最小包含円を求める。
ただし Q は周上にあることが保証されている。
 - P が空なら、 Q の外接円が答え
 - P から点をひとつ選び、 p とする
 - p を除いた最小包含円 $\text{min_disk}(P - \{p\}, Q)$ を D とおく。
 - $p \in D$ なら、 D が答え
 - $p \notin D$ なら、 p は最小包含円の周上にある
 - $\rightarrow \text{min_disk}(P - \{p\}, Q + \{p\})$ が答え

最小包含円 – 平均 $O(n)$

- 点の順番がランダムになっているとする
- 最小包含円の周上に点は3個程度しかない
 - $\rightarrow \text{min_disk}(P - \{p\}, Q + \{p\})$ が呼び出される確率は $(3 - \#Q)/n$ 程度
 - したがって、平均で $O(n)$

最小包含円 – その他の方法

- 中心を決めると、必要な半径は $O(n)$ で求まる
- 貪欲法
 - 円の中心を、必要な半径が小さくなる方向に進めていく
- 適当な値で代用する
 - 円の中心 = 全ての点をカバーする矩形の中心 などとおく
 - それなりに良く近似できるし、簡単に書ける

K個の点集合を決める

- 乱択で近傍探索
- クラスタ解析
- その他の方法

K個の点集合 – 近傍探索

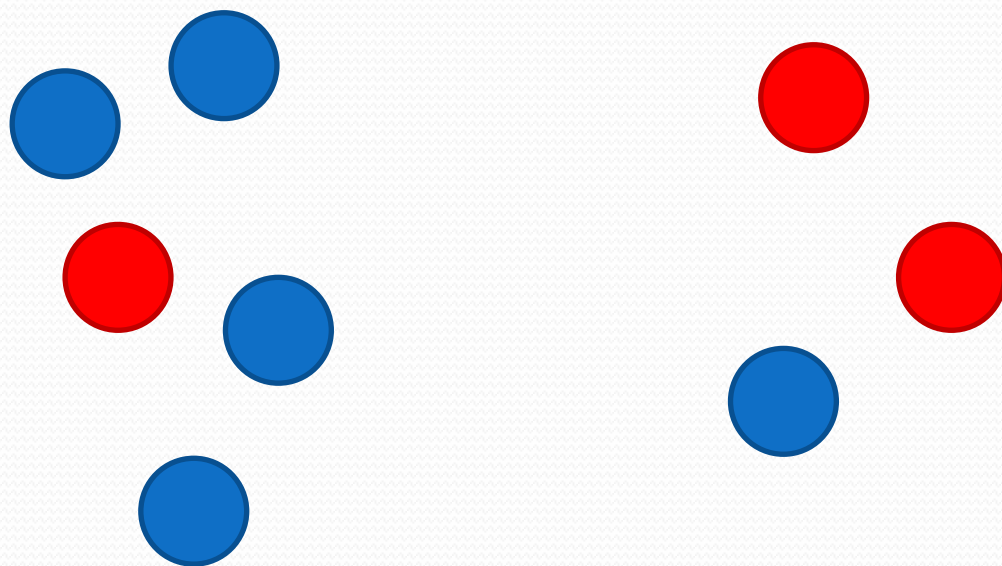
- とりあえず適当に点集合をおく(乱数とかで)
- 近傍は以下のように選ぶ
 - 適当な点を選び、適当な集合に移動させる
 - スコアに応じて、その近傍に移動する
- 均等なデータのほうが強い

K個の点集合 – クラスタ解析

- データ群を類似関係にあるグループに分類する方法
- ここでは「K平均法」を使う

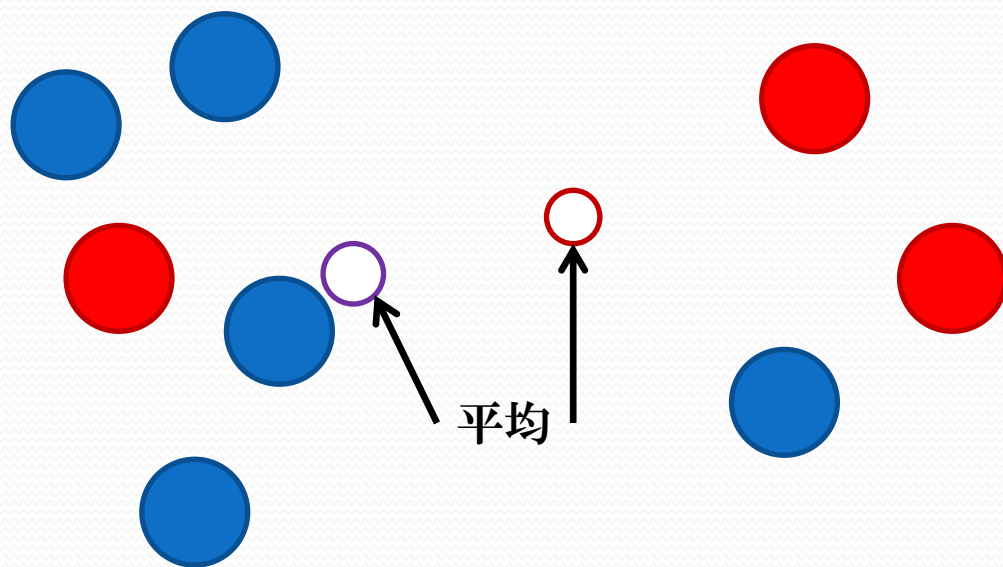
K個の点集合 – クラスタ解析

- データ群を類似関係にあるグループに分類する方法
- ここでは「K平均法」を使う



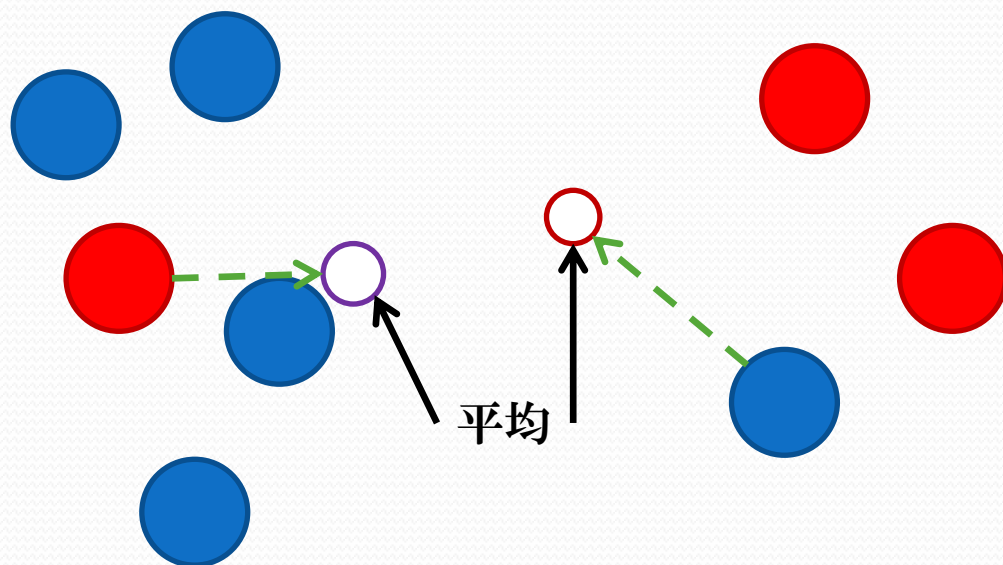
K個の点集合－クラスタ解析

- データ群を類似関係にあるグループに分類する方法
- ここでは「K平均法」を使う



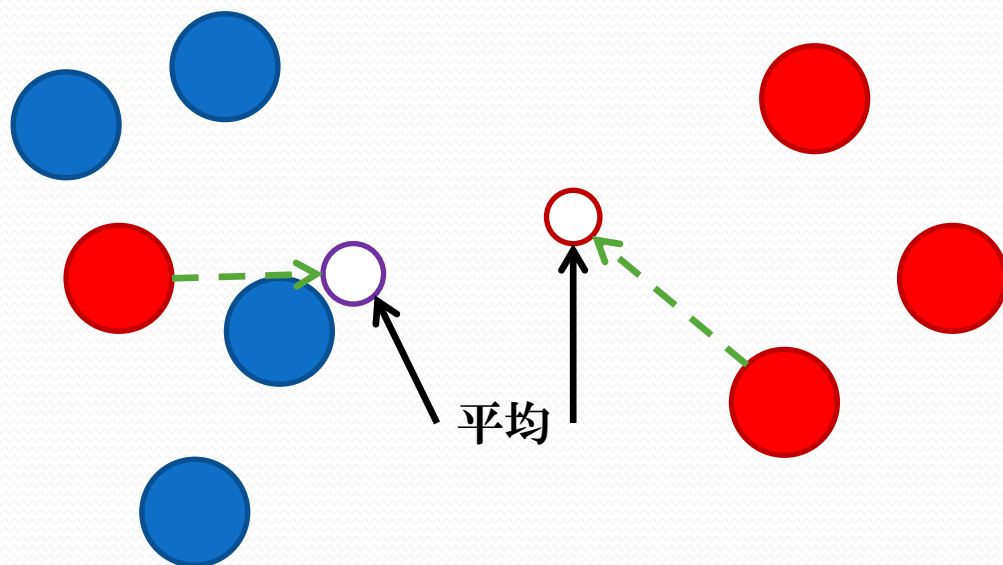
K個の点集合 – クラスタ解析

- データ群を類似関係にあるグループに分類する方法
- ここでは「K平均法」を使う



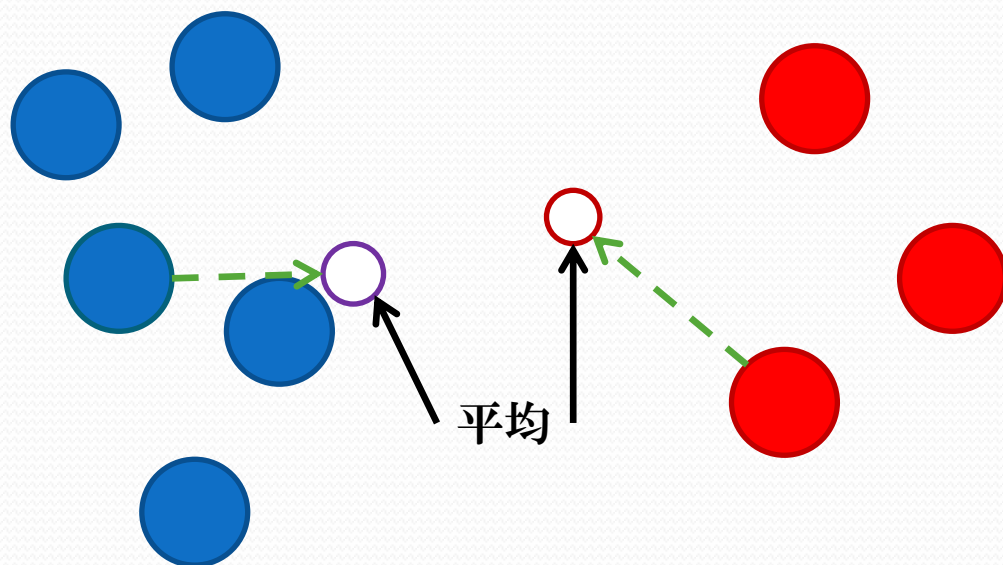
K個の点集合－クラスタ解析

- データ群を類似関係にあるグループに分類する方法
- ここでは「K平均法」を使う



K個の点集合－クラスタ解析

- データ群を類似関係にあるグループに分類する方法
- ここでは「K平均法」を使う



K個の点集合 – クラスタ解析

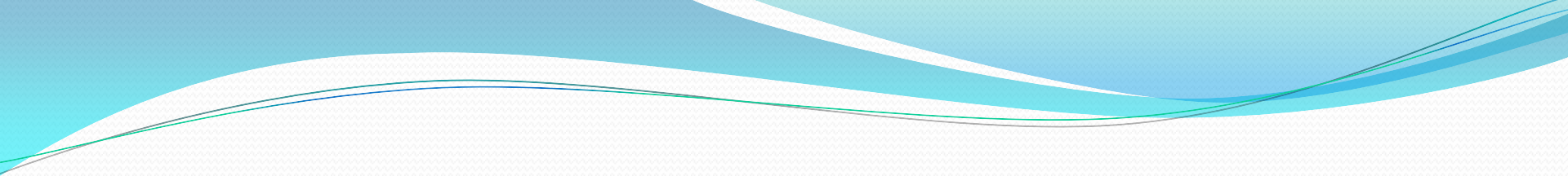
- データ群を類似関係にあるグループに分類する方法
- ここでは「K平均法」を使う
- まばらなデータに強い

K個の点集合 – その他の方法

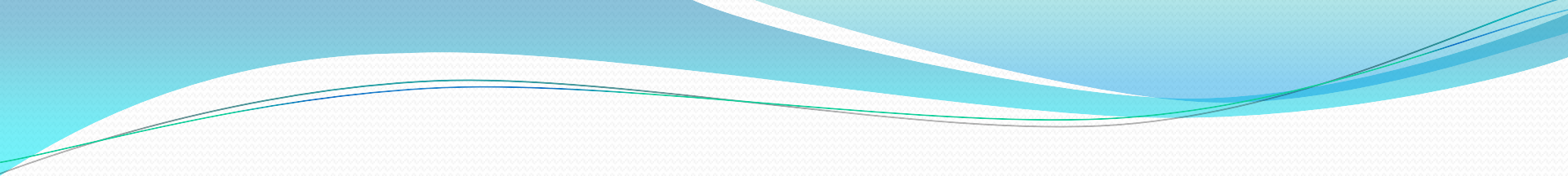
- Kruskal法と同様に、N個から併合を繰り返してK個にする

その他の方法の紹介

- 中心決め打ち
 - 貪欲法などを使って半径を決める

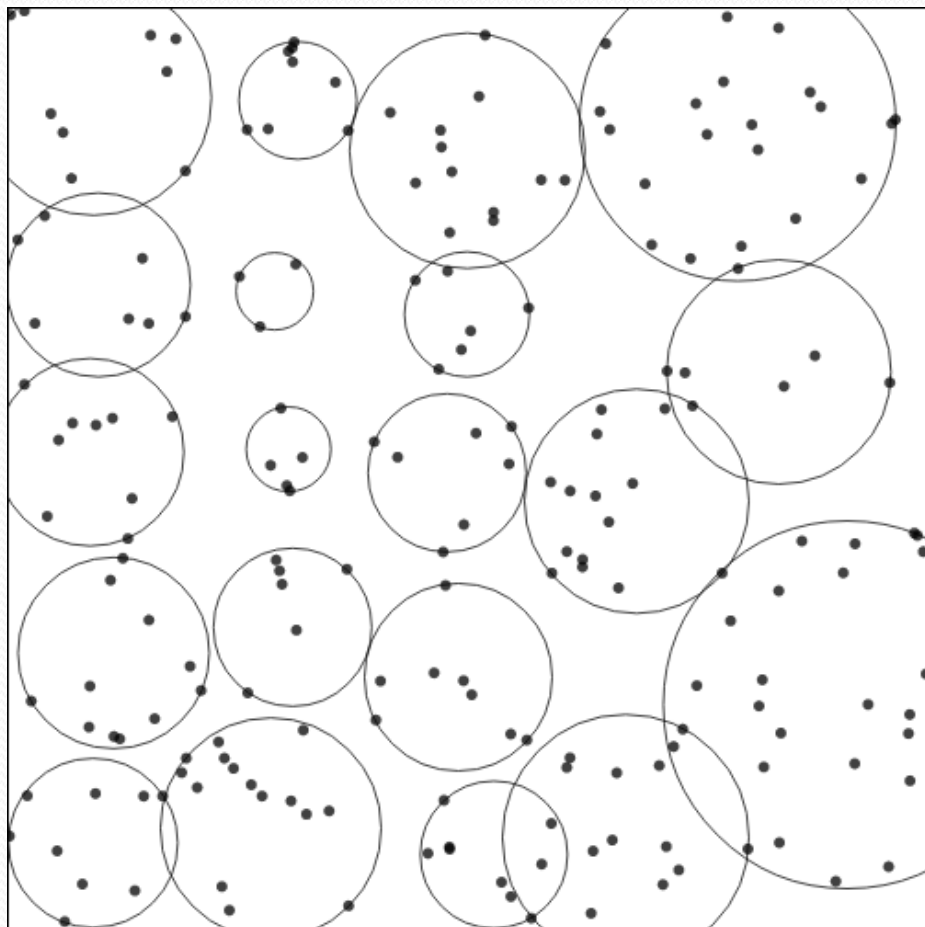


それでは



生徒の最良データの紹介

生徒の最良データの紹介

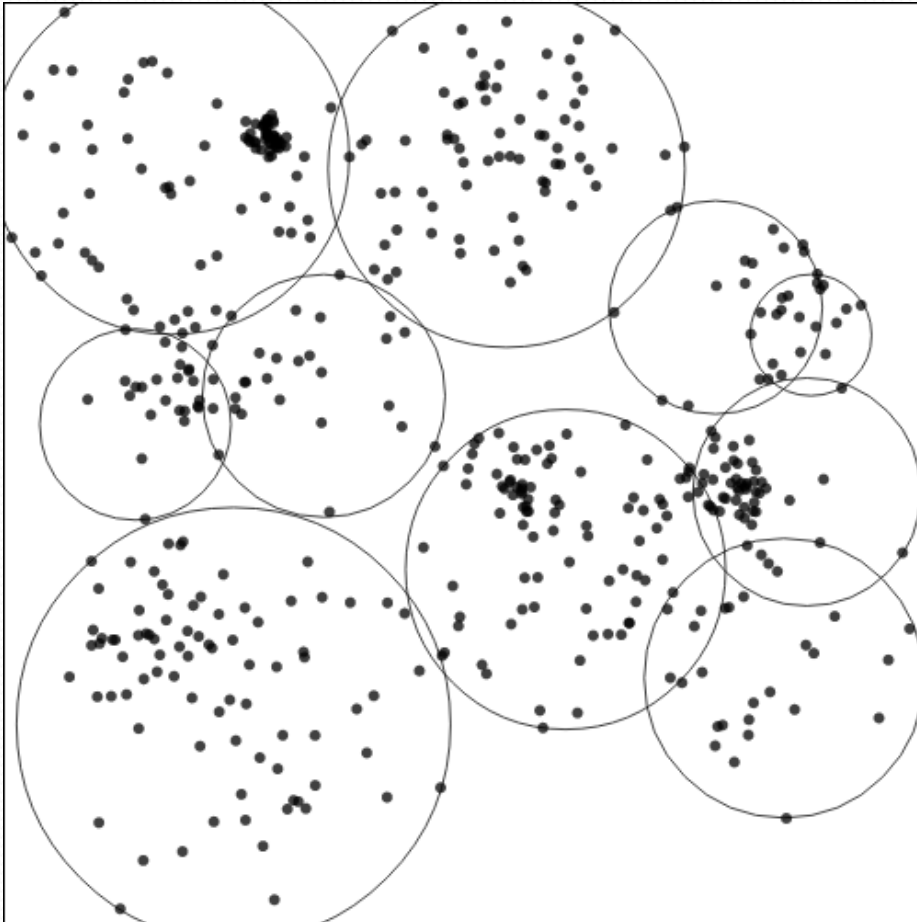


ino1

JPN₁₇

スコア 247498181034

生徒の最良データの紹介

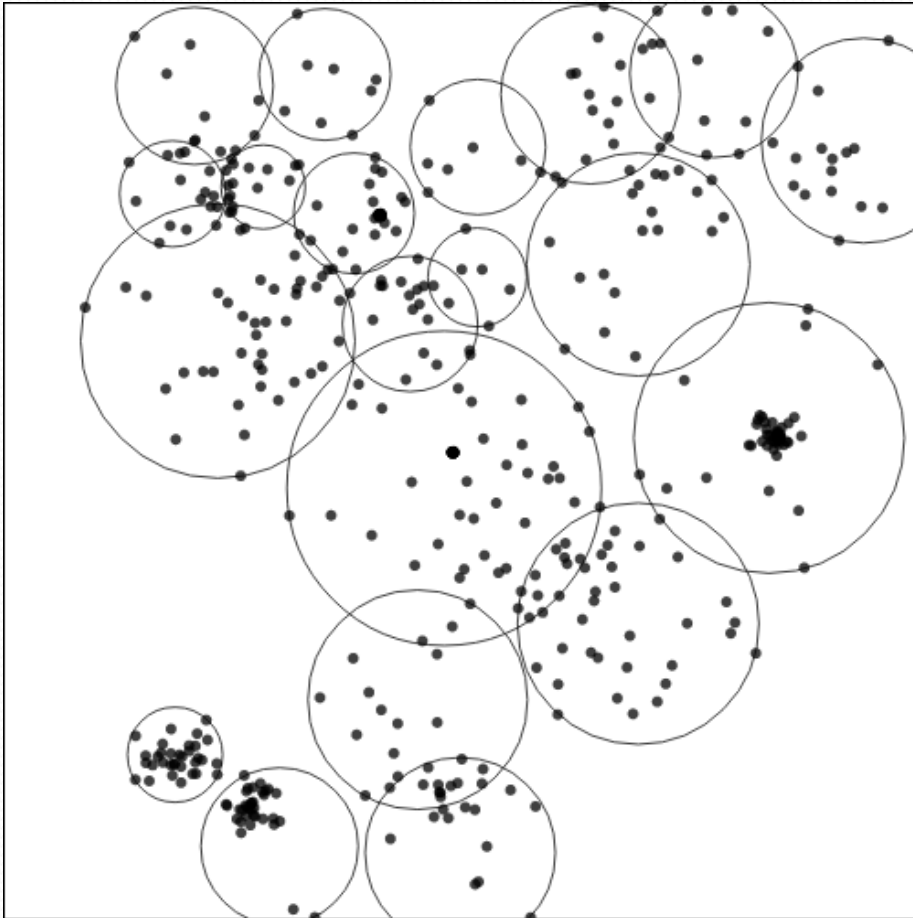


ino2

JPN18

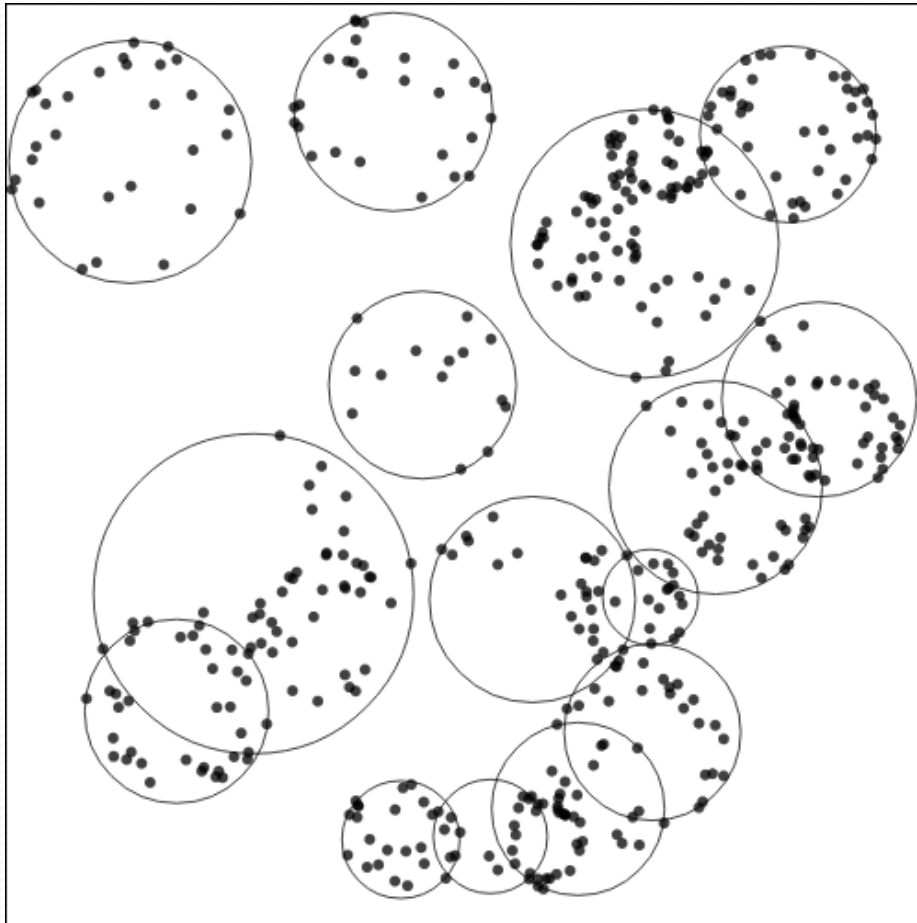
スコア 246525708455

生徒の最良データの紹介



ino3
JPN18
スコア 208558463568

生徒の最良データの紹介

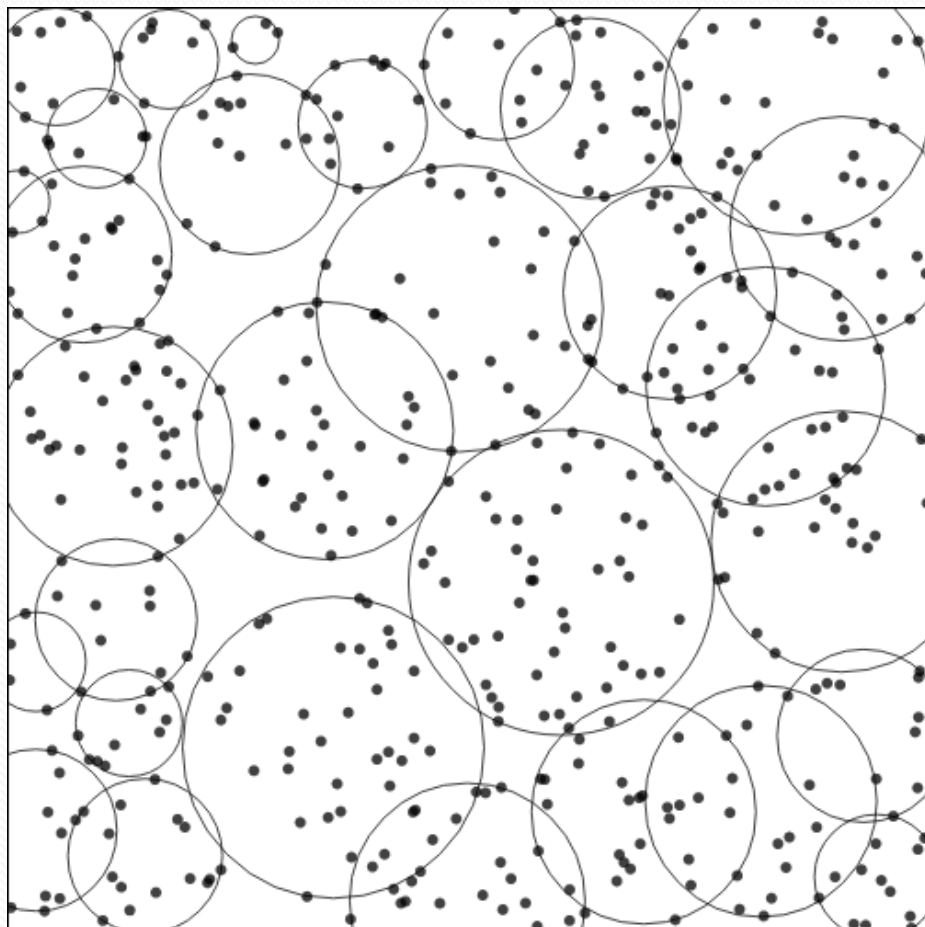


ino4

JPN18

スコア 176846776195

生徒の最良データの紹介

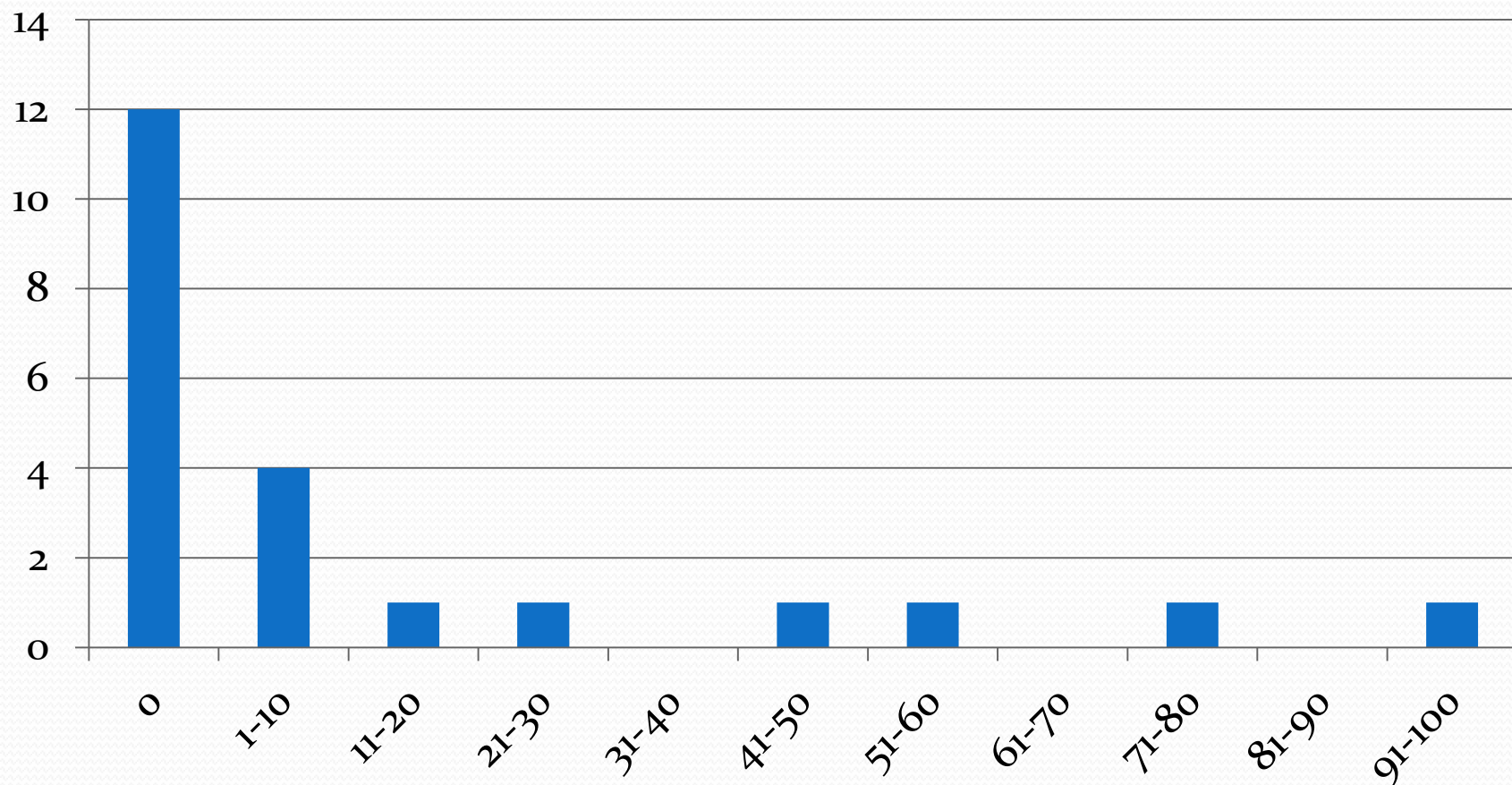


ino5

JPN18

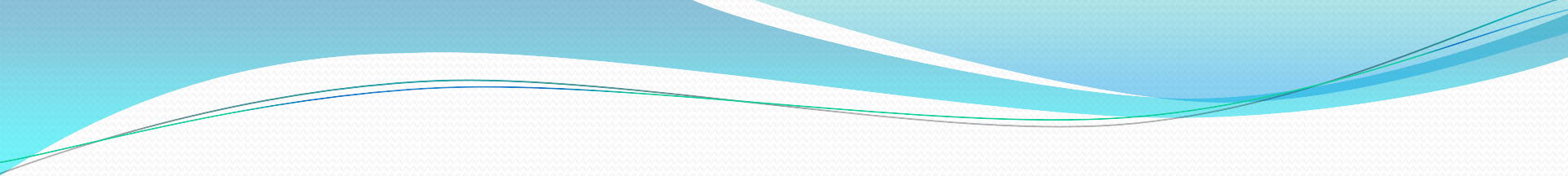
スコア 342198348502

参考：得点分布



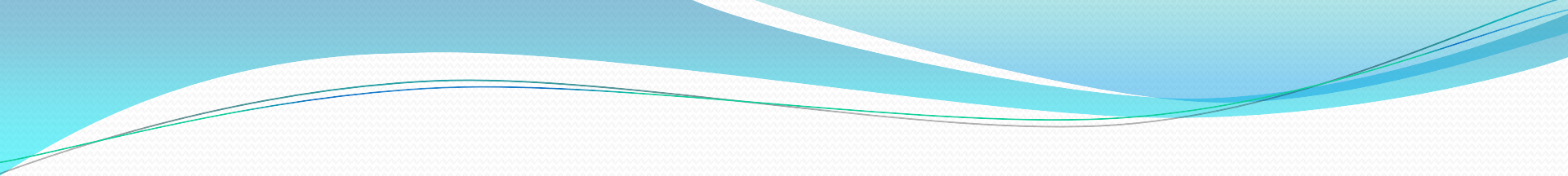
まとめ

- 出力のみ問題に特化した戦略を
 - とにかくプログラムを回しまくるとか
- 5時間という短い時間内でできることを
 - この時間内でできる戦略をしよう
- 01→適当な最小包含円+局所探索が良さそう
- 02-05→適当な最小包含円+クラスター分析が良さそう
- まだ改善の余地はあるとはいえ、5時間のコンテストでの成績としては十分ではないでしょうか
 - ただ、不正なデータで0点の人が多いいのは勿体無いですね



ご清聴ありがとうございました

今日もよく寝て万全の体調で3日目に臨みましょう。



ご清聴ありがとうございました

今日もよく寝て万全の体調で3日目に臨みましょう。

(拍手！)

春合宿2日目第2問
星座(CONSTELLATION)

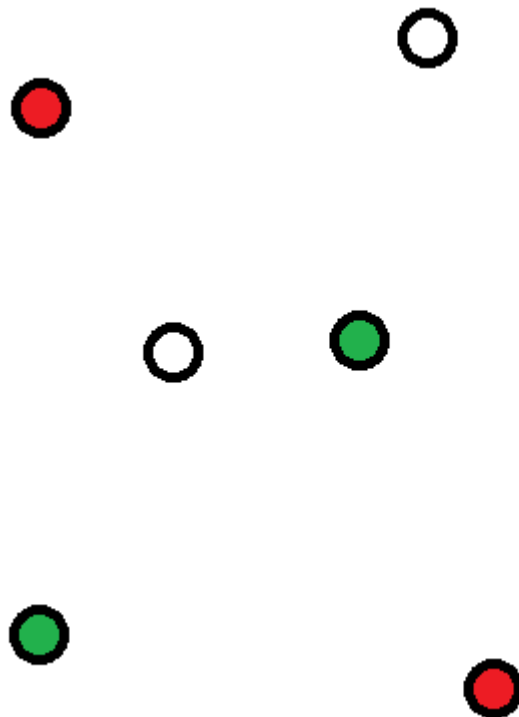
解説

平野湧一郎

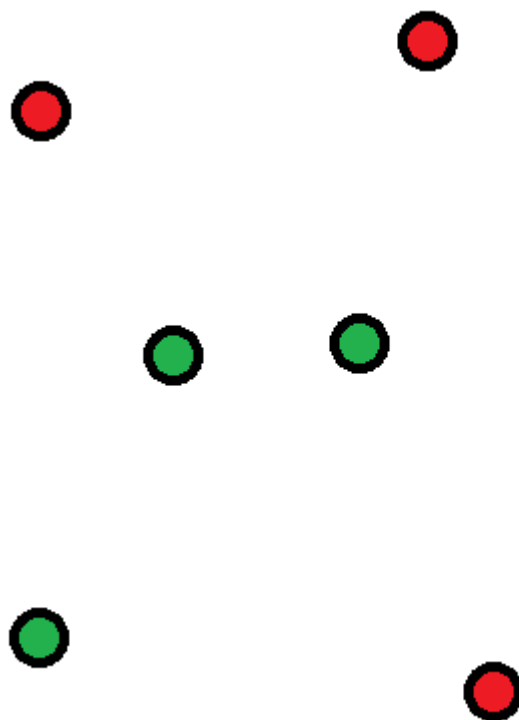
問題概要

- 平面上に星座Aに属する星・星座Bに属する星・どちらに属するかわからない星がたくさんある
- 星座Aの星どうし, 星座Bの星どうしで線分を結び, それぞれの星座に属する星どうしが連結になるようにする
- ただし, 星座Aの線分と星座Bの線分は交わってはいならない
- このような結び方が可能になるように, どちらに属するかわからない星を星座A・Bに割り当てる方法の数を求める

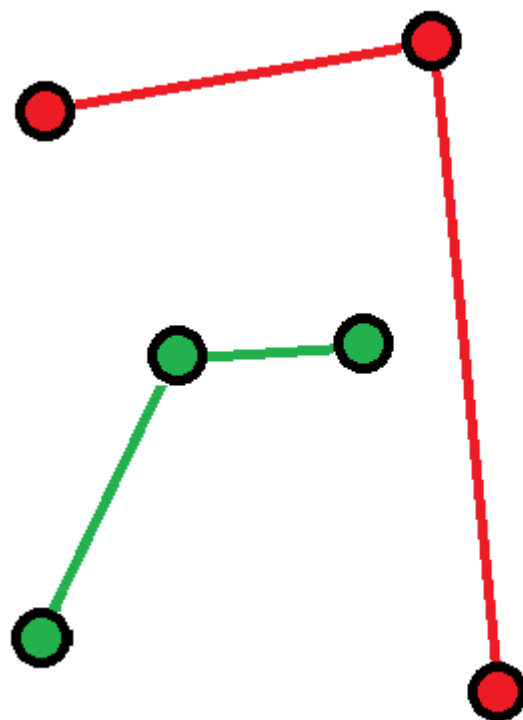
例



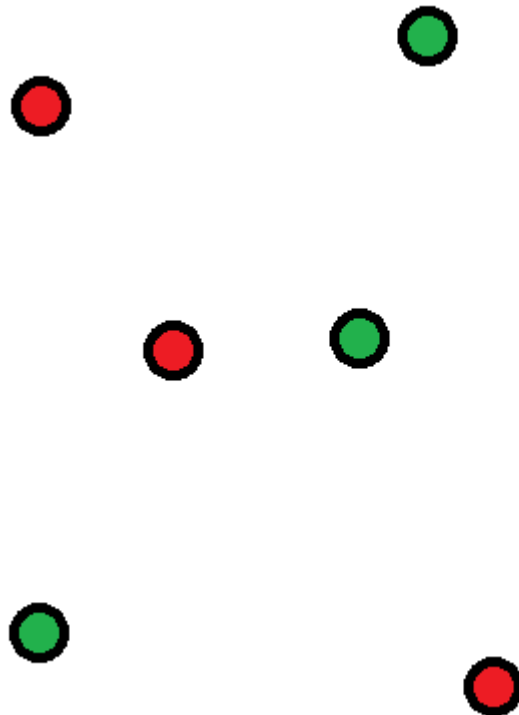
OKな例



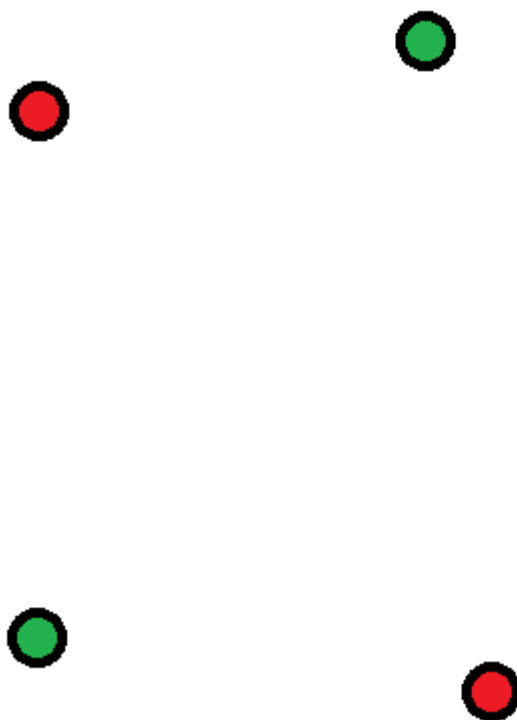
結び方の例



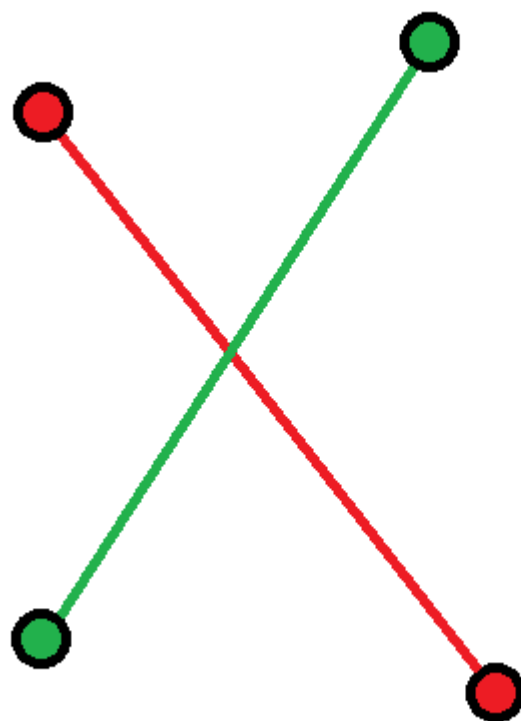
NGな例



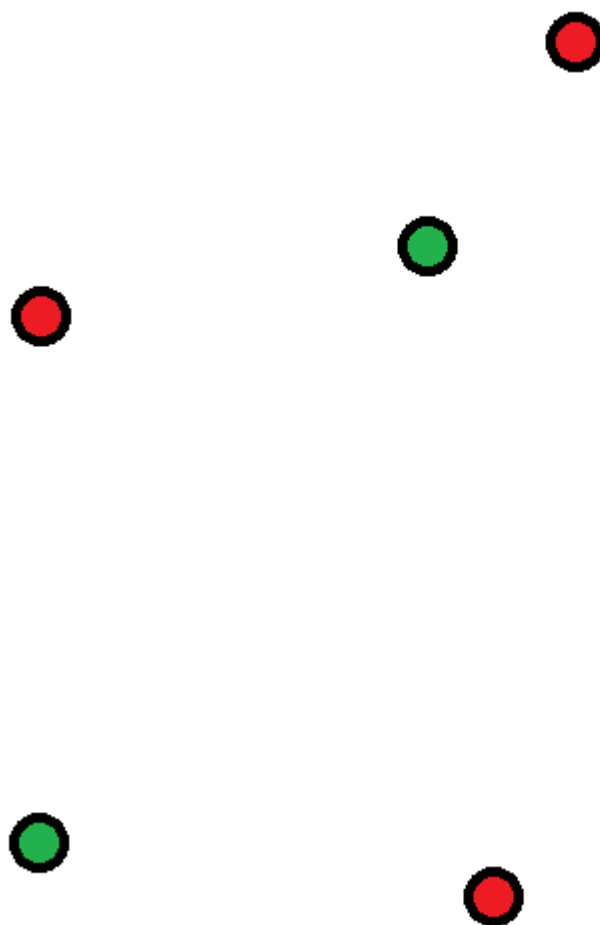
やばそうな配置



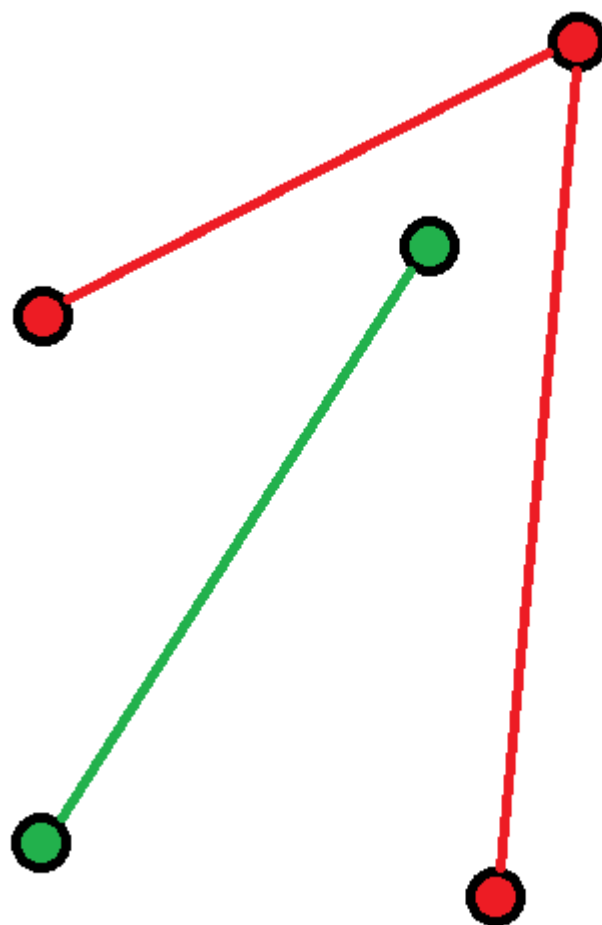
やばそうな配置



実はOKな配置



実はOKな配置



考察

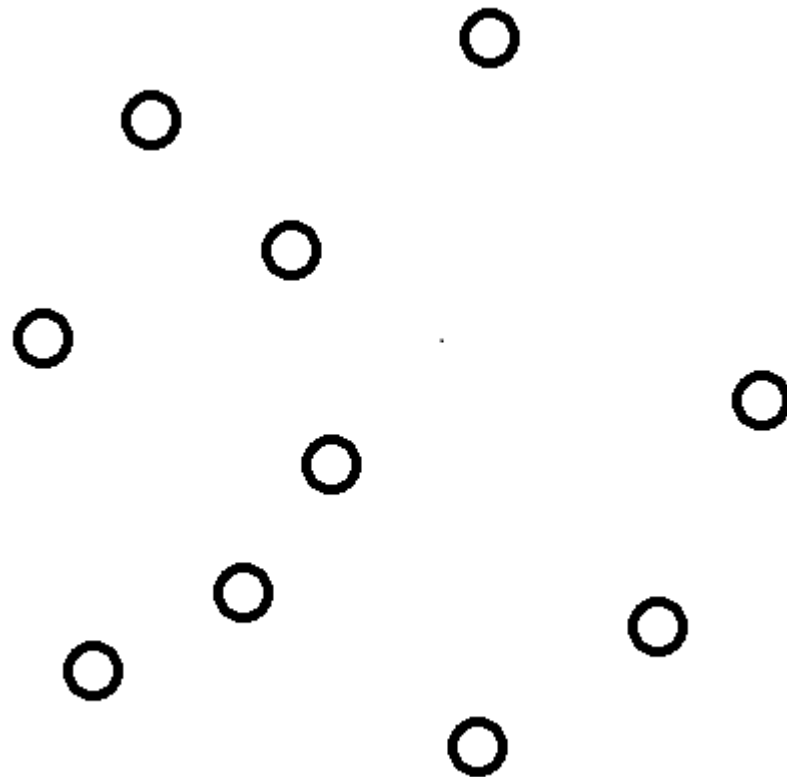
- どうやらやばそうな配置があっても、外側に他の点があればうまく迂回して結ぶことができるようだ
- 逆に言うと、一番外側がやばそうな配置だとやばそう
- 「(一番)外側」って？

凸包(Convex hull)

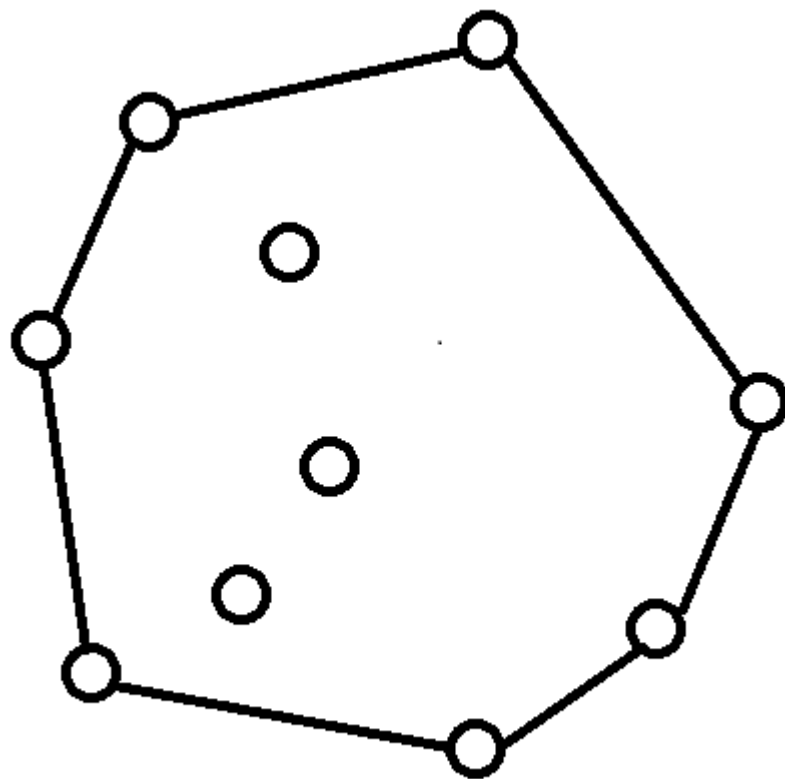
凸包とは

- ある点の集合について、それらを全て含む最小の凸多角形
- 簡単に言うと、点の位置にクギを打って全体に輪ゴムをかけたとき、ピンと張った輪ゴムのなす図形のこと

凸包の例



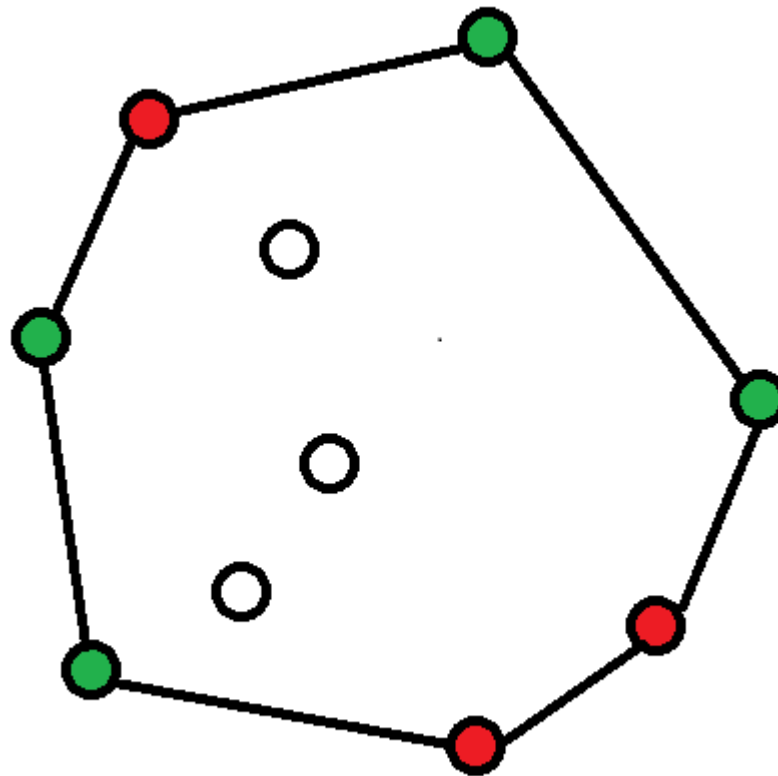
凸包の例



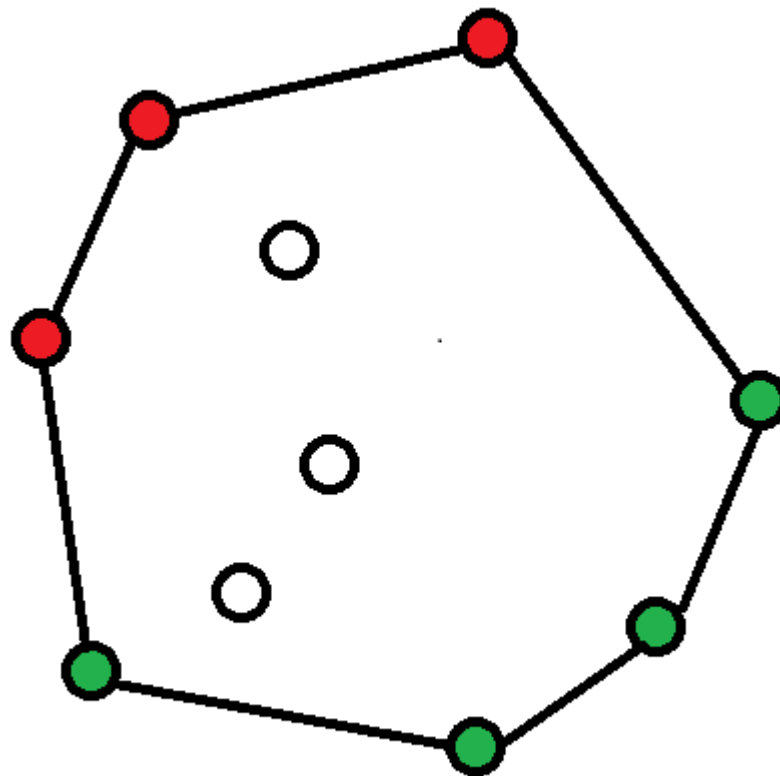
星座A・Bがうまく結べる条件

- 凸包上の点に
- 赤...赤緑...緑赤...赤緑...緑
- という配列があると**不可能**
- 逆に、凸包が
- 赤...赤緑...緑
- という配列なら**可能**
- (言い換えると、凸包を適当にぶったぎって
- 二色に分けることができればOK)
- あるいは、凸包上の点が全部同じ色でもOK

NGな例



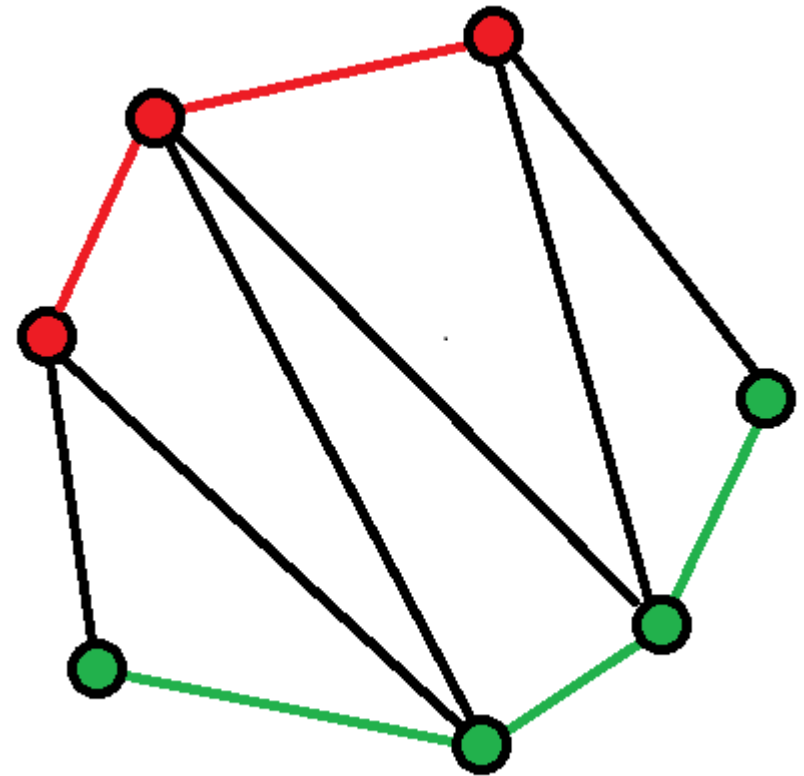
OKな例



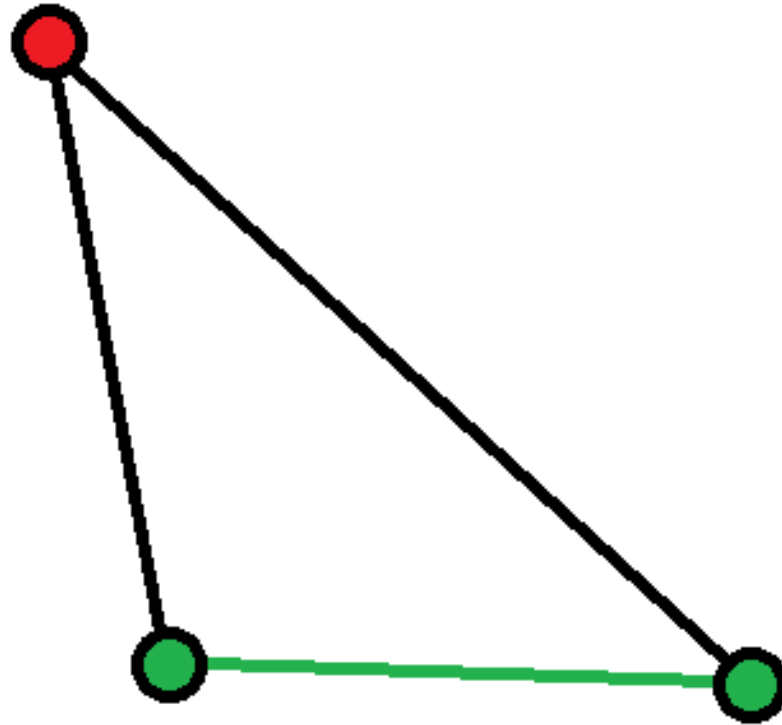
本当に凸包が二色に分かれていれば
必ずうまく結べるのか？

凸包の分割

- まず凸包上を構成する
同じ色どうしの線分は結ぶ
- 次に、凸包を三角形に分割する
- このとき、全ての三角形
の頂点が二色（赤・緑）
両方を含むようにうまく
分割する

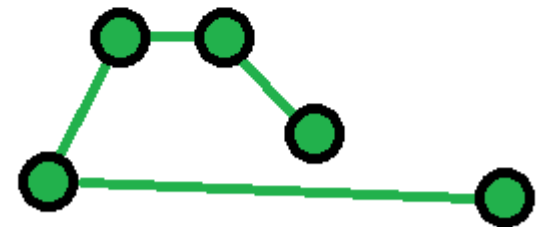


三角形の例



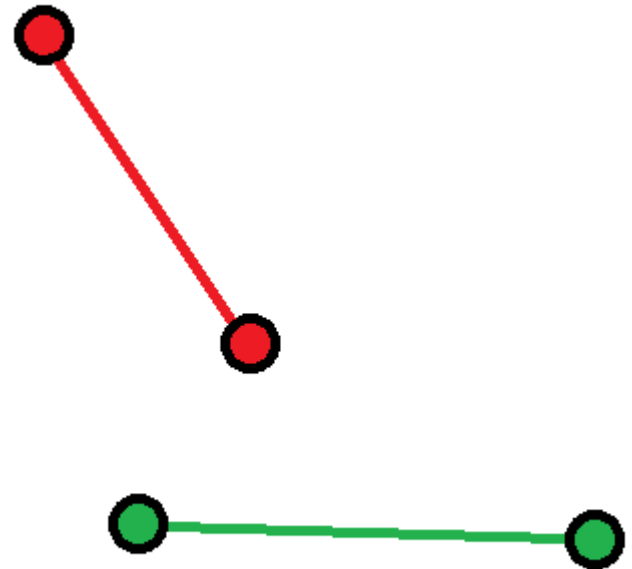
内部に赤い点がない場合

- 適当に結ぶ



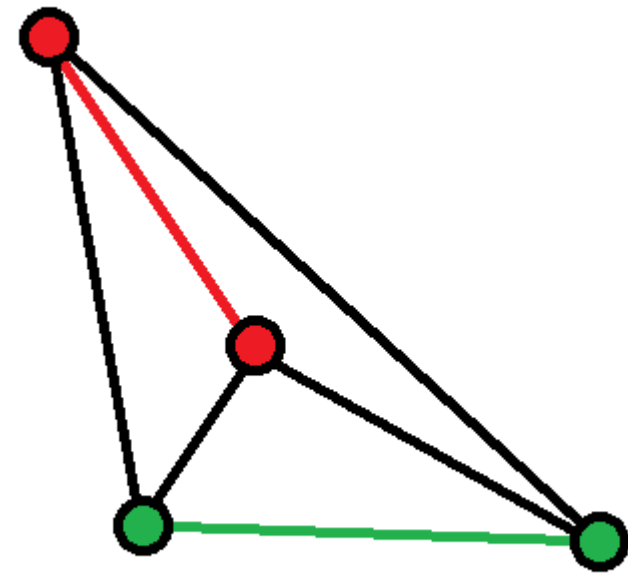
内部に赤い点がある場合

- 赤い点どうし結ぶ



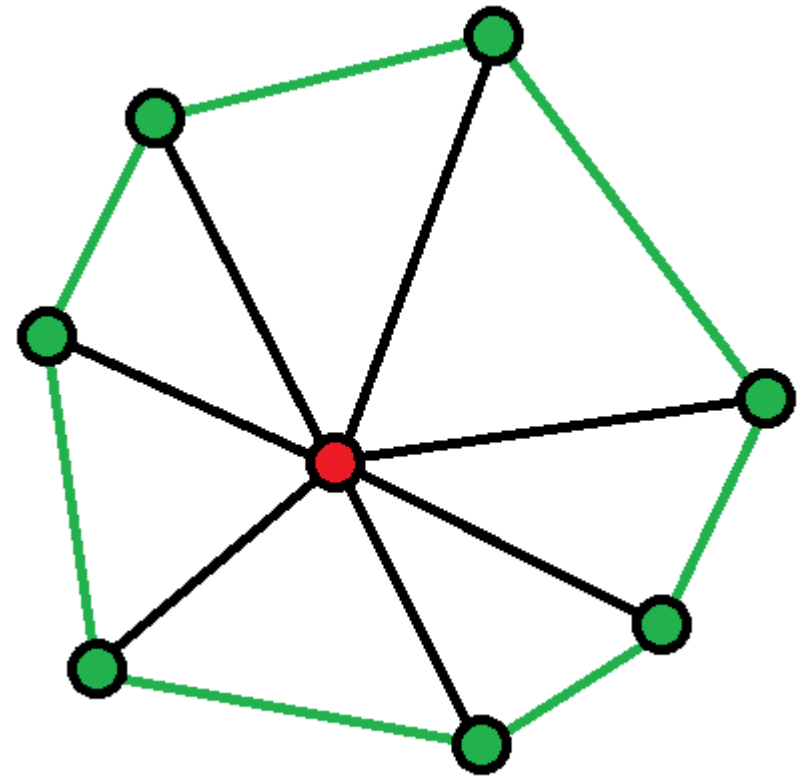
再帰的に三角形分割

- 再び3つの三角形に分割
- これら3つの三角形もまた、頂点が二色(赤・緑)を含むようになっているので、同じ議論を行える
- これを繰り返していくといつかは内部に点がなくなるので、うまく結べることができた.
- ちなみに同様の議論がIOI'06(Mexico)の点の結合(Points)で登場しています



凸包上の点全てが同じ色のとき

- 星座を構成する星の数は1以上という制約があるので、内部に必ず赤い点がある
- 凸包上の全ての点と内部の適当な赤い点を選んで三角形分割すれば、同じ議論ができる.
- 要するにこの場合でもOK



実装

凸包 ($O(N^2 \log N)$)

- まず点列をX座標でソート
- X座標最小の点は必ず凸包に含まれるので、最初
はここからスタート
- 今居る点について、最も右手にある点を順にたどっ
ていく
- 最初の点に戻ってきたら、たどってきた点の集合が
凸包
- 「最も右手にある点」を探すのにソートなどで $O(N \log N)$ かかるので、全体で $O(N^2 \log N)$

凸包 ($O(N \log N)$)

- まず点列をX座標でソート
- X座標最小の点から, 下凸包と上凸包にわけて求め, 最後にマージする
 - 下凸包を求めるときは, 点列をX座標の小さい順に見て行き, 進行方向より左側にある点はそのまま加え, 進行方向より右側にある点は, 凸性が成り立つまで最近加えた点を順に削除してからその点を加える
 - 上凸包を求めるのはこれと逆
 - 1つの点は高々1回ずつしか追加・削除されないため, ソートのコストの $O(N \log N)$ で求められる
- 詳しくはSpaghetti Sourceなどを見てください

凸包上の数え上げ

- 要するに赤と緑の境目(2箇所)がどこにあるかだけ考えればよい
 - 凸包上の点が赤と白だけなら, 連続した白の部分それぞれに対し, 緑の開始位置と終了位置が何通りあるか考えればよい
 - 凸包上に赤・緑どちらもあれば, 境目になっているところの白の部分がどこまで赤でどこから緑なのかを考えればよい
 - もし最初から赤緑赤緑という部分があれば0を出力
- 凸包上に白い点しかなければ, とりあえずどこか1箇所を赤にしてしまい, 最後に2倍すると楽です

実装上の注意

コーナーケースに注意

- 点の数が2のときの処理(凸包が構成できない)
- 「星座を構成する星の数は1以上」という条件
 - 与えられた入力に赤がまったくない場合, 「全て緑」を差し引かないと正しい答えが出ません

オーバーフローに注意

- 「でかい数で割った余りを求めよ」という問題にはオーバーフローがつきもの
- 誤って`int*int%MOD`という計算をしてしまうと
DEAD END
- 適切に`long long`にキャストしなければならない
- また、計算して値が変わった場合に必ずMODで割った余りを持たないとオーバーフローの原因になります

余談

Let's play Short Coding!!

- 私はShort Codingが大好きです
- 24日の午前中にC言語でのShort Coding大会があります
- 0日目に突貫でちょっと解説しましたが、あれでは足りないと思うのでショートコーディング攻略wiki (http://w.livedoor.jp/short_coding/)などを見ましよう
- 実際にShort Codingしてみたいという方は anarchy golf (<http://golf.shinh.org/>)などへ

お願い

- 合宿中の問題を解くときにShort Codingしないでください
- 情報の授業とかでShort Codingしないでください
- プログラミングコンテストとかでShort Codingしないでください(Short Codingのコンテストを除く)
- ルールを守って楽しくShort Codingしよう

JOI 春合宿 2012 Day2

回転 (Rotate)

秋葉 拓哉

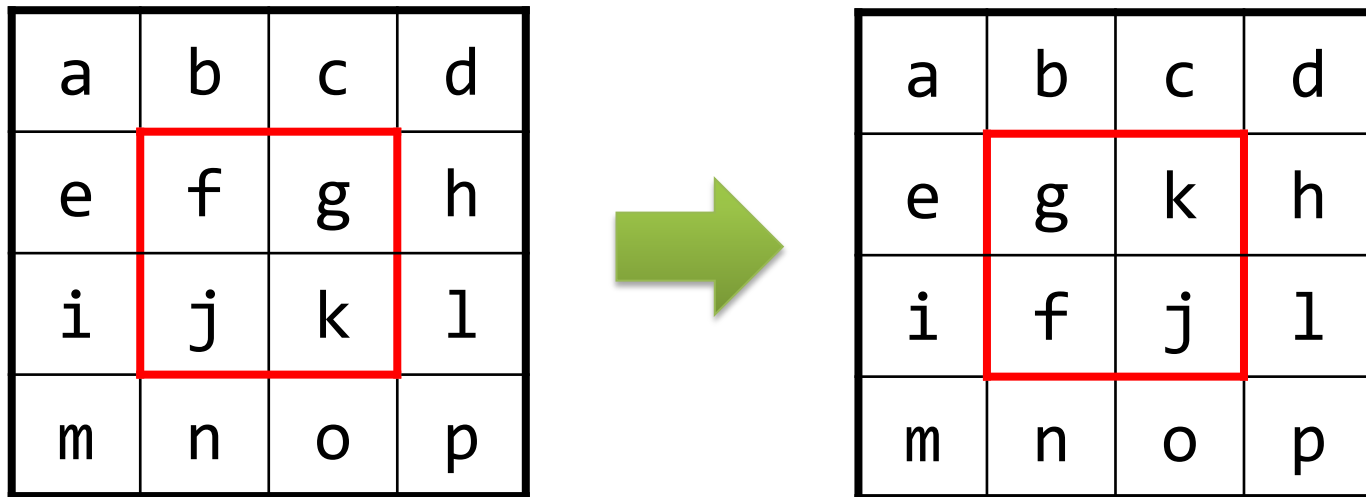
問題概要

a	b	c	d
e	f	g	h
i	j	k	l
m	n	o	p

$N \times N$ の盤面 ($N \leq 1000$)

各マスにはアルファベット

問題概要



Q 回, 正方形領域を回転 ($Q \leq 2000$)

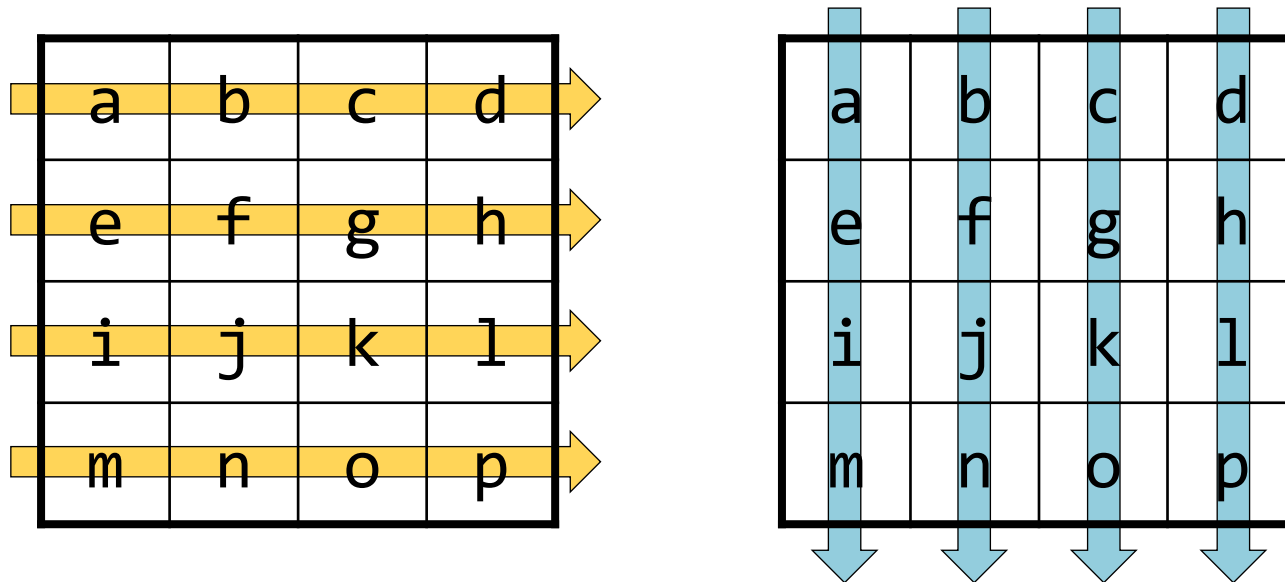
終了後の盤面を出力

愚直な解法 (10 点)

1. 二次元配列に盤面を確保
2. 毎回, 本当に回転する

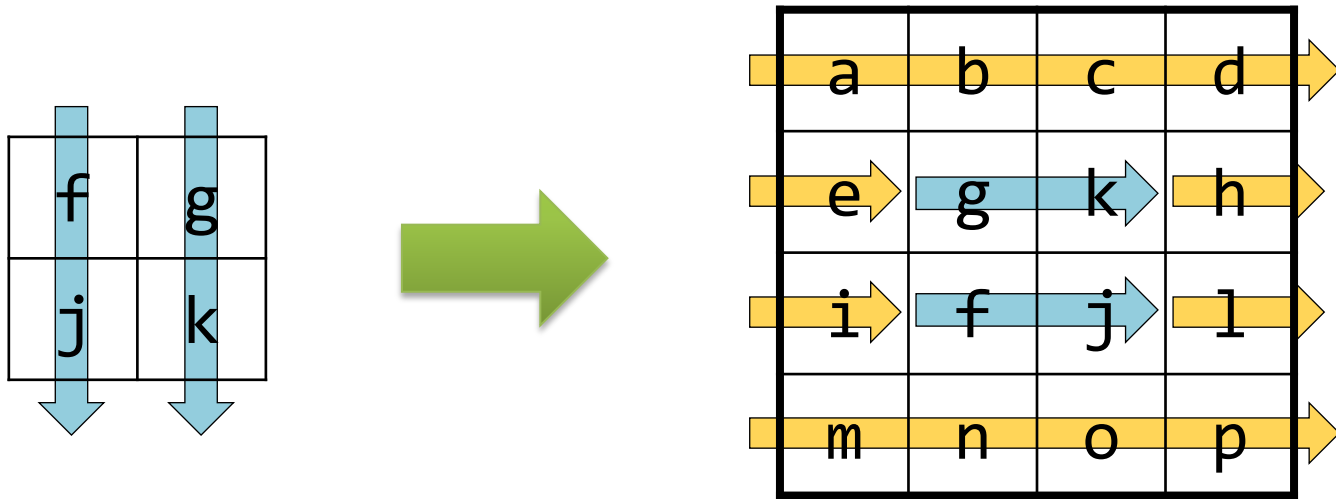
$O(QN^2)$ 時間, $N, Q \leq 100$ のセットは OK
(デバッグ用にも便利なのでどちらにせよ実装するべし)

高速な解法へのアイデア



盤面を横向きと縦向きで列で管理
列に適切なデータ構造を用いる

高速な解法へのアイデア



回転は付け替えるだけ

(4 方向分管理しておくか, 2 方向で反転をサポート)

適切なデータ構造？

列の split, merge といえば...
この問題講義でやったやつだ!!

2012/3/20 NTTデータ駒場研修所 (情報オリンピック春合宿)

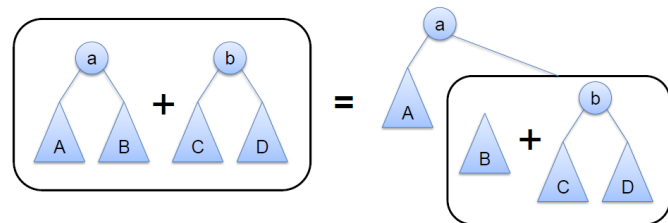
プログラミングコンテストでの データ構造 **2**

東京大学情報理工学系研究科

秋葉 拓哉

1

Treap 実装: merge (merge-split ベース)



- 優先度の高い方の根を新しい根にする
- 再帰的に merge

36

適切なデータ構造？

列の split, merge と言えば...
この問題講義でやったやつだ!!

遅い!!!

2012/3/20 NTTデータ駒場研修所 (情報オリンピック)

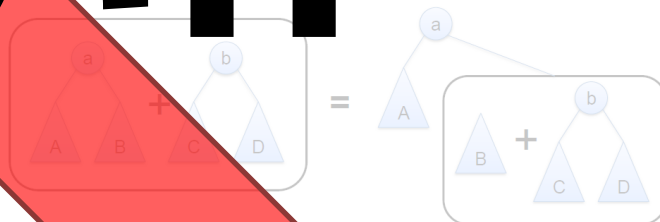
プログラミングコンテスト データ構造 2

東京大学情報理工学系研究科

秋葉 拓哉

1

Heap 実装: merge
(merge split ベース)



- 優先度の高い方の根を新しい根にする
- 再帰的に merge

36

平衡二分探索木を使うと...

- 計算量 $O(NQ \log N)$
- $N = 1000, Q = 2000 \rightarrow NQ = 2\,000\,000$
- 数百万のオーダーに $\log$ がつくのはそれだけで緊張
- 数方向について処理をするし,
- 平衡二分探索木の定数も大きい

...それで 2 秒はちょっとヤバイと見積もれる

...その前に！

平衡二分探索木は本当に必要？

- いつものじゃダメ？
 - 配列, 線形リスト
 - `Std::set`, `std::map`
 - Binary Indexed Tree, バケット法, セグメント木
- 実装が面倒なので楽に避けられたら避けたい
- 実際のところ, 本当に必要になる問題はレア

11

平衡二分探索木まとめ

- まずはもっと容易な道具を検討！
 - 配列, リスト, `std::map`, BIT, セグメント木, バケット法
 - 必要な場所だけ作るセグメント木
- 実装が楽な平衡二分探索木を選ぼう
 - 今回: Treap / Randomized Binary Search Tree
 - 他: スプレー木, Scapegoat 木, Block Linked List, Skip List
- 実装しよう
 - insert / erase ベース vs. merge / split ベース
 - 更新遅延

51

...でもやっぱその前に！

- 動的木は本当に必要？ いつものじゃダメ？
- やはり, 実装が面倒, 避けられたら避けたい
- 実際のところ, 本当に必要になる問題は**皆無**

(特別賞を狙いたいのであればこの限りではない)

56

Link-Cut 木まとめ

- まずはもっと容易な道具を検討！
 - クエリの平方分割
- 実装しよう (下に行くほど大変)
 - expose, link, cut, root, 頂点の情報に関する質問
 - evert, 頂点の情報の更新
 - 辺の情報に関する質問・更新

80

...その前に！

平衡二分探索木は本当に必要？

- いつものじゃダメ？
 - 配列, 線形リスト
 - `Std::set`, `std::map`
 - Binary Indexed Tree, バケット法, セグメント木
- 実装が面倒なので楽に避けられたら避けたい
- 実際のところ, 本当に必要になる問題はレア

11

平衡二分探索木まとめ

- まずはもっと容易な道具を検討！
 - 配列, リスト, `std::map`, BIT, セグメント木, バケット法
 - 必要な場所だけ作るセグメント木
- 実装が楽な平衡二分探索木を選ぼう
 - 今回: Treap / Randomized Binary Search Tree
 - 他: スプレー木, Scapegoat 木, Block Linked List, Skip List
- 実装しよう
 - insert / erase ベース vs. merge / split ベース
 - 更新遅延

51

...でもやっぱその前に！

- 動的木は本当に必要？いつものじゃダメ？
- やはり, 実装が面倒, 避けられたら避けたい
- 実際のところ, 本当に必要になる問題は**皆無**

(特別賞を狙いたいのであればこの限りではない)

56

Link-Cut 木まとめ

- まずはもっと容易な道具を検討！
 - クエリの平方分割
- 実装しよう (下に行くほど大変)
 - expose, link, cut, root, 頂点の情報に関する質問
 - evert, 頂点の情報の更新
 - 辺の情報に関する質問・更新

80

...その前に！

平衡二分探索木は本当に必要？

- いつものじゃダメ？
 - 配列, **線形リスト**
 - `Std::set`, `std::map`
 - Binary Indexed Tree, バケット法, セグメント木
- 実装が面倒なので楽に避けられたら避けたい
- 実際のところ, 本当に必要になる問題はレア

11

平衡二分探索木まとめ

- まずはもっと容易な道具を検討！
 - 配列, **リスト**, `std::map`, BIT, セグメント木, バケット法
 - 必要な場所だけ作るセグメント木
- 実装が楽な平衡二分探索木を選ぼう
 - **今回**: Treap / Randomized Binary Search Tree
 - **他**: スプレー木, Scapegoat 木, Block Linked List, Skip List
- 実装しよう
 - insert / erase ベース vs. merge / split ベース
 - 更新遅延

51

...でもやっぱその前に！

- 動的木は本当に必要？ いつものじゃダメ？
- やはり, 実装が面倒, 避けられたら避けたい
- 実際のところ, 本当に必要になる問題は**皆無**

(特別賞を狙いたいのであればこの限りではない)

56

Link-Cut 木まとめ

- まずはもっと容易な道具を検討！
 - クエリの平方分割
- 実装しよう (下に行くほど大変)
 - expose, link, cut, root, 頂点の情報に関する質問
 - evert, 頂点の情報の更新
 - 辺の情報に関する質問・更新

80

高速な解法 (100 点)

ポイント

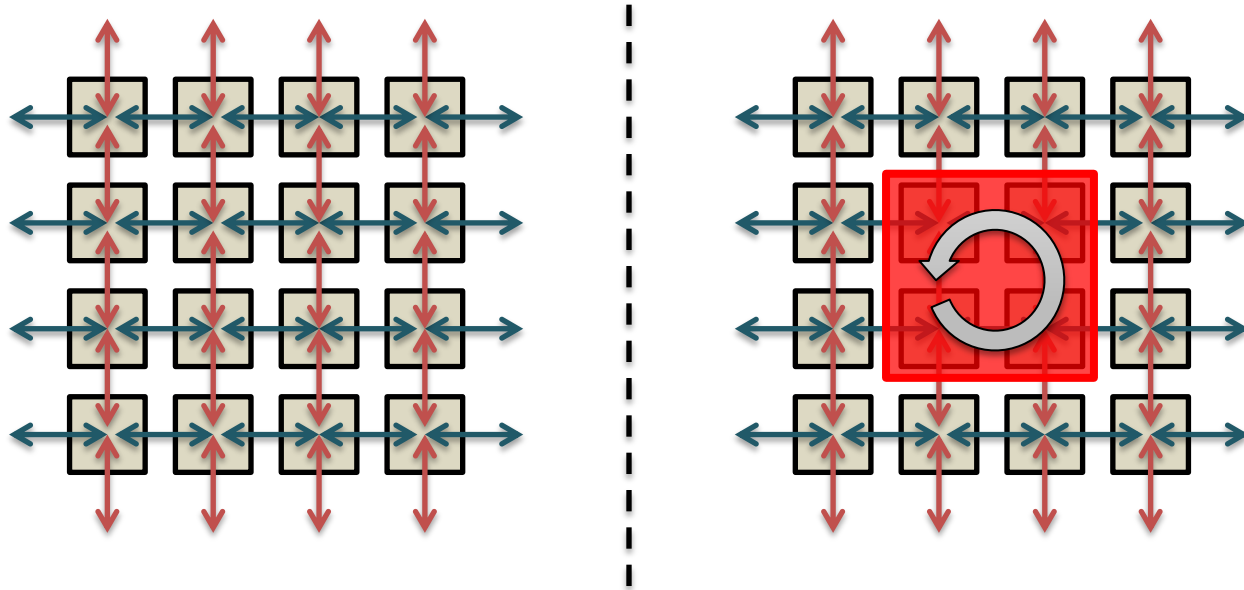
一度に $O(N)$ はかけてよい



アイディア

実はリストでできる!!

高速な解法 (100 点)



右にも下にも行けるリストでやる

数方向分持っておいて取り替えたり,
入ってきたら次に出ていく方向を管理したりすれば良い

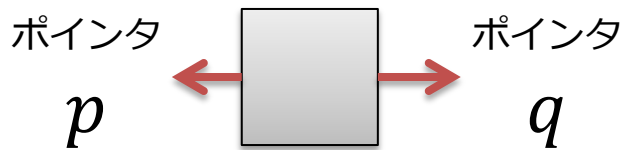
高速な解法 (100 点)

何故リストでも良い？

- むしろ、何故、平衡二分探索木を使いたかったか？
→ 切りたい場所をすぐ見つけられ、すぐ切れるから
- でも、一回に $O(N)$ はかかっても良い
 - よって、切る場所を見つけるのに全体で $O(N)$ かかって良い
 - 切る場所まで端からたどることが許される

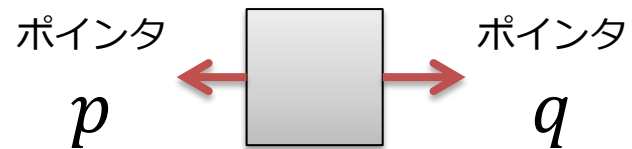
おまけ: Xor-Linked List

普通の Double-Linked List



p と q の両方を覚えてる

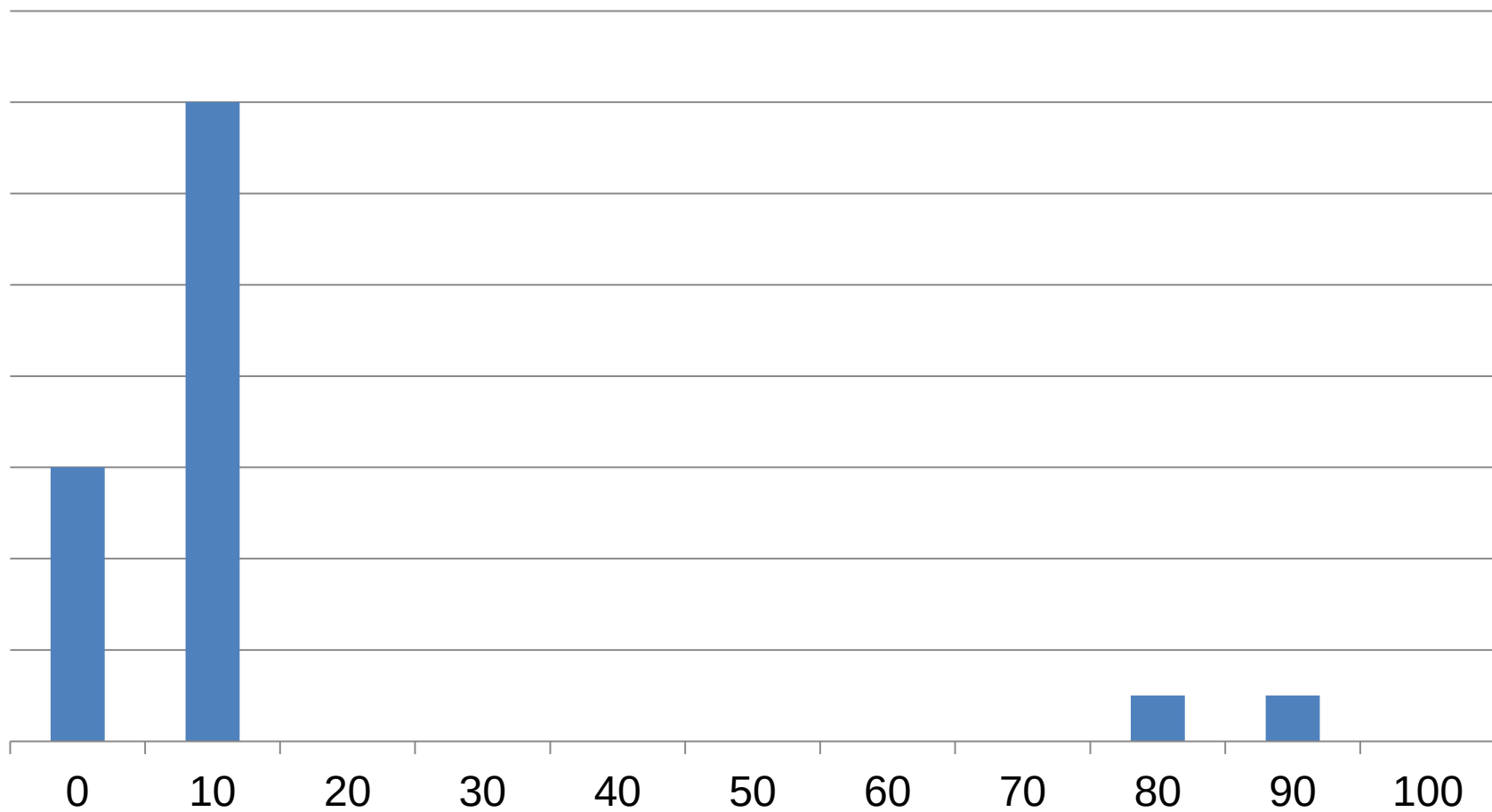
Xor-Linked List



$p \otimes q$ を覚えておく

- $p \otimes q$ を覚えておけば, 実は十分な事が多い
 - p から来たら, $(p \otimes q) \otimes p = q$ より, q がわかる
 - q から来たら, $(p \otimes q) \otimes q = p$ より, p がわかる
- 領域が減るだけでなく, 左右反転が容易にでき便利
 - 張り替えるだけで良い

得点分布



JOISP2012 Day3

Fortune Telling (占い)

解説



問題の概要

- ⇒ カードが M 行 $\times$ N 列に並んでいる
- ⇒ 長方形領域をひっくり返すのを K 回繰り返す
- ⇒ 最後に表になっている枚数を求める
($M, N: 10^9$ $K: 10^5$)



愚直な解法

$M \times N$ の配列を用意してシミュレーション
(または上/左から1行ずつ見ていく)

→ 時間: $O(MNK)$, 空間: $O(MN)$
or $O(\min(M, N))$



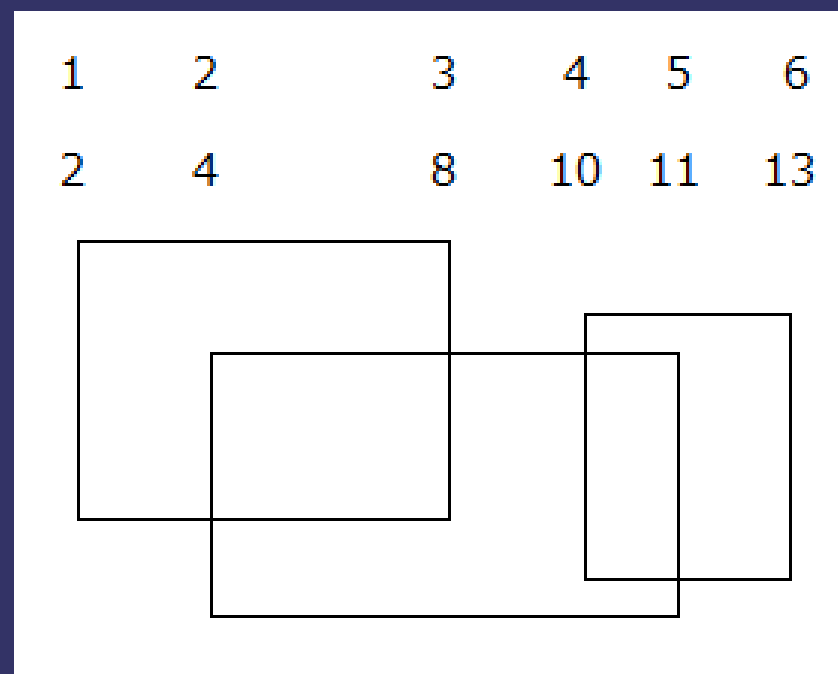
パソコンが壊れる



テクニック1：座標圧縮

使う座標だけを持って
座標をつけ直すと座標が
高々2Kくらいに収まる
(某本参照)

i	0	1	2	3	4	5	6	7
x[i]	0	2	4	8	10	11	13	N

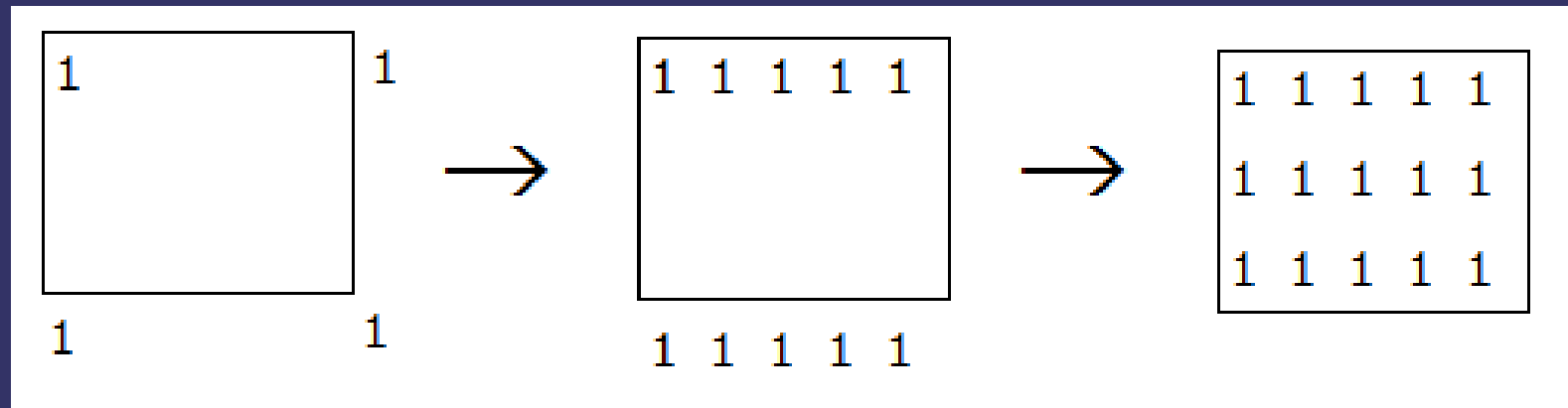


部分点解法

たくさんの長方形に (mod 2で) 1を足すには,
4カ所に1を書いて最後にまとめて

各行で累積和 → 各列で累積和 すればよい.

((2*K)^2の配列をintで取るとMLEなので注意)



部分点解法'

実は上の行から処理すれば

メモリには1行だけ持っておけばいいので

空間計算量は $O(K)$ でもできる

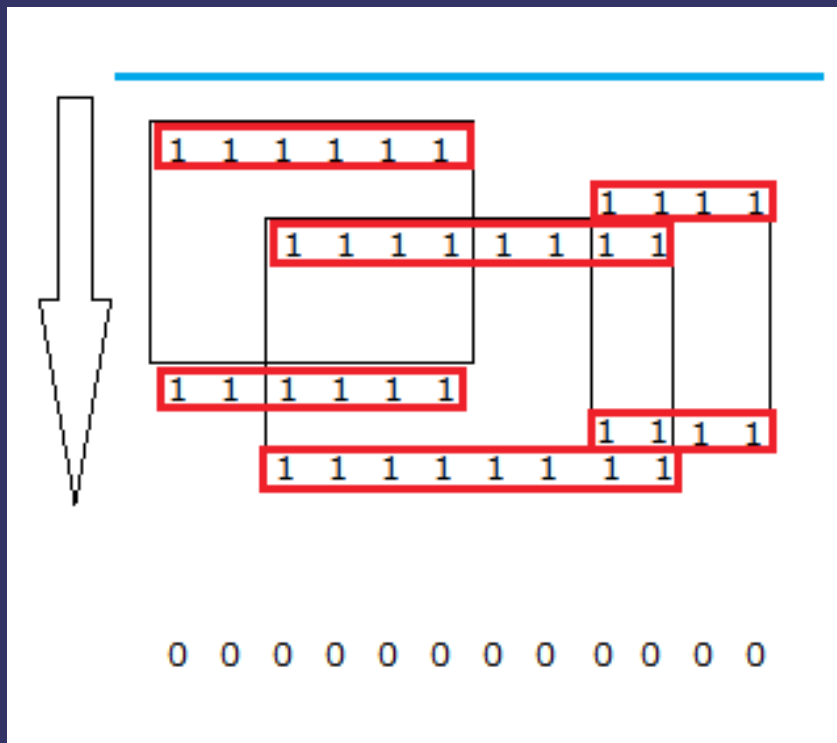
(1を書く場所たちを上から→左から順にソートしておく)

時間 : $O(K^2)$ 空間 : $O(K^2)$ or $O(K)$

→ 30点



やるべきこと

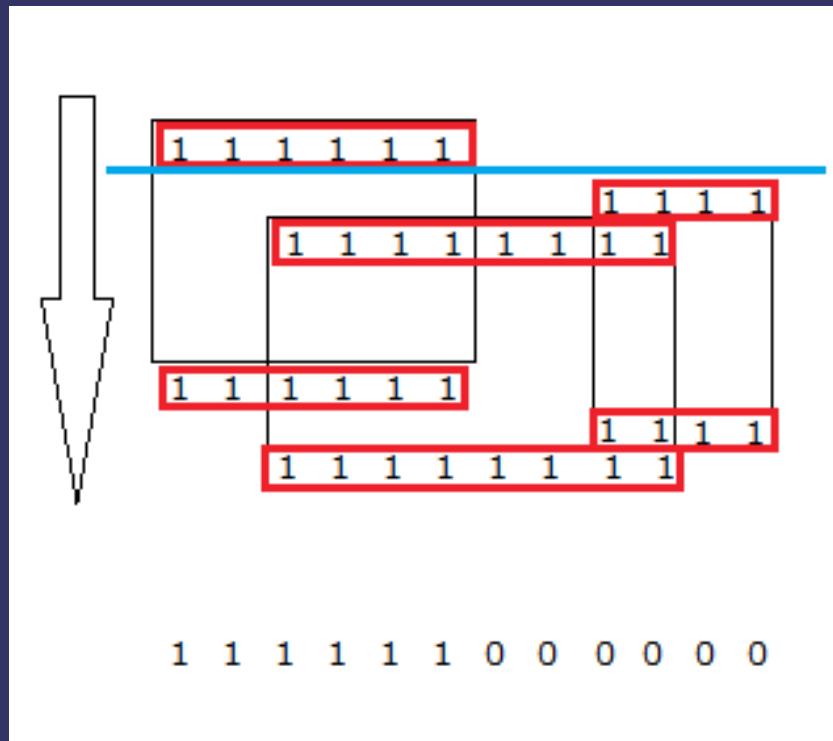


上から1行ずつ見ていって

- ・ 長方形の上/下辺が来たら区間をflipする
- ・ 各行で0がいくつあるか見る



やるべきこと

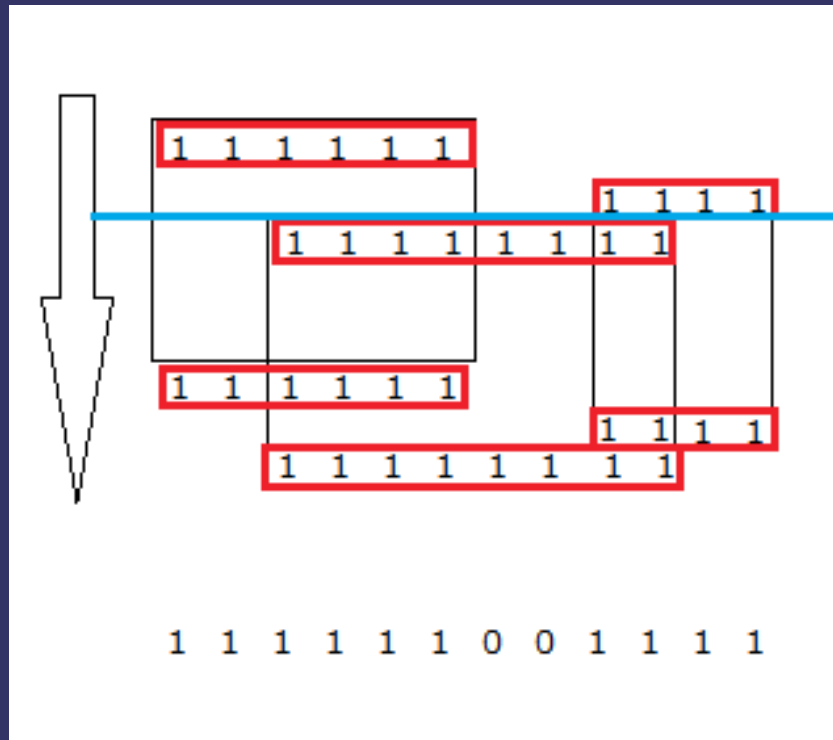


上から1行ずつ見ていって

- ・ 長方形の上/下辺が来たら区間をflipする
- ・ 各行で0がいくつあるか見る



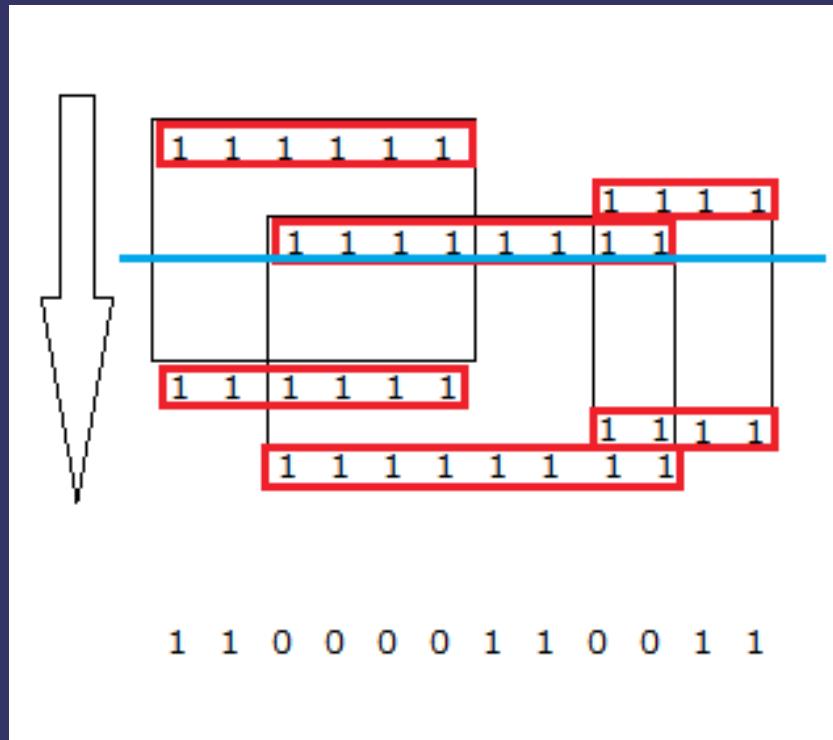
やるべきこと



- 上から1行ずつ見ていって
- ・ 長方形の上/下辺が来たら区間をflipする
 - ・ 各行で0がいくつあるか見る



やるべきこと



- 上から1行ずつ見ていって
- ・ 長方形の上/下辺が来たら区間をflipする
 - ・ 各行で0がいくつあるか見る



100点を取るには

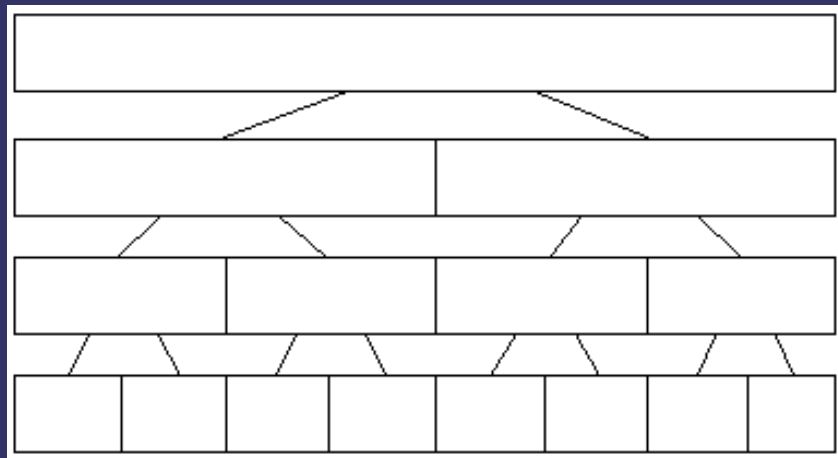
0,1(表/裏)の列があるとき,

- ・ 指定した区間をflipするクエリ
- ・ 全体で0(表)が何個あるか見る

の操作が高速に出来るデータ構造がほしい.



テクニック2：セグメント木



- ノードに区間が対応している二分木
- ノードに様々なデータを持たせると色々できる
- 某本参照



テクニック2：セグメント木

この問題では各ノードに

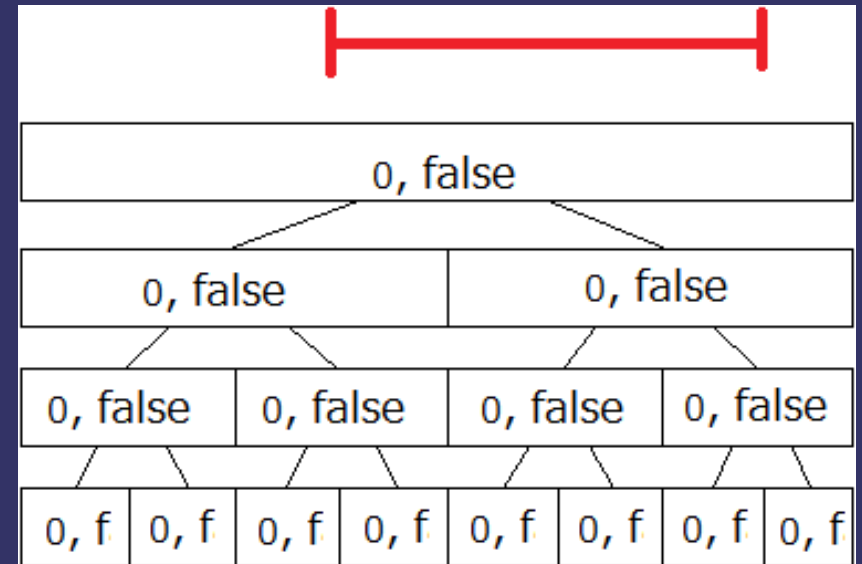
- ・ 対応する区間にある裏の数
- ・ 子孫の反転処理を遅延していることを表すフラグ

を持たせると出来る.



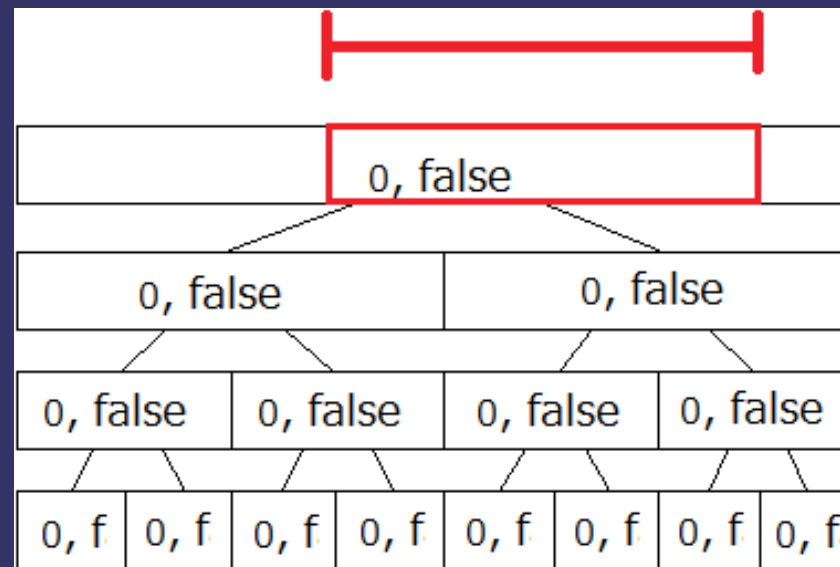
クエリの処理の仕方

区間[4,7]を反転するク
エリ



クエリの処理の仕方

add(根, 4, 7) 的な関数

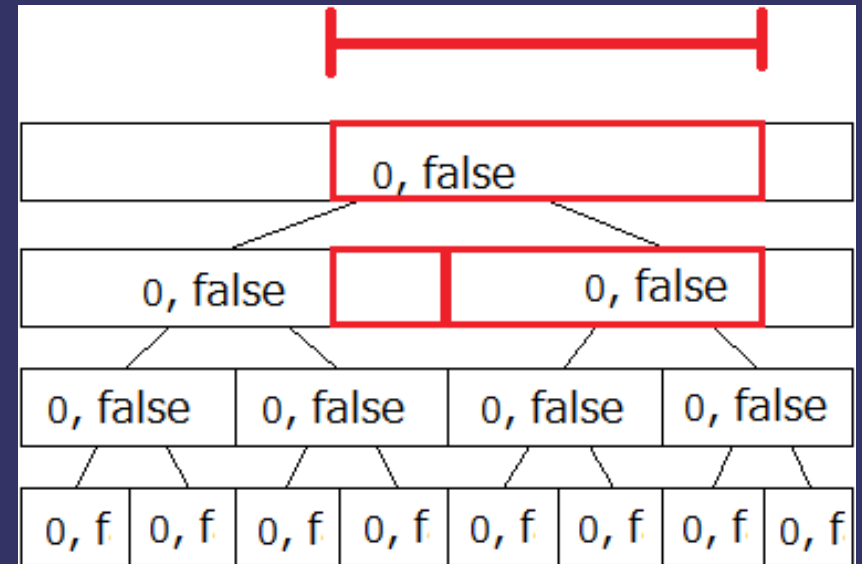


クエリの処理の仕方

add(根, 4, 7)は

- ・ add(左の子, 4, 4)
- ・ add(右の子, 5, 7)

を呼び出す



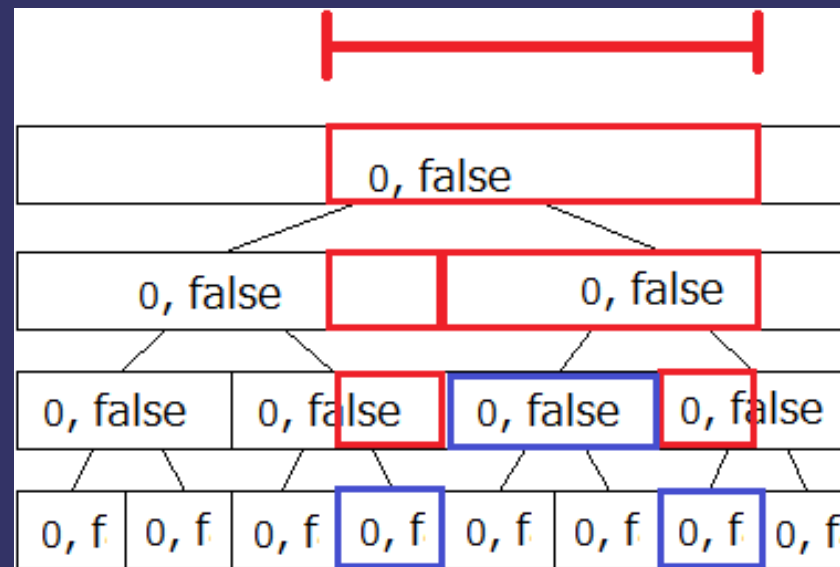
クエリの処理の仕方

同様に繰り返す.

ただし, ノードに対応する
区間全体を反転する
ときはそこで止める

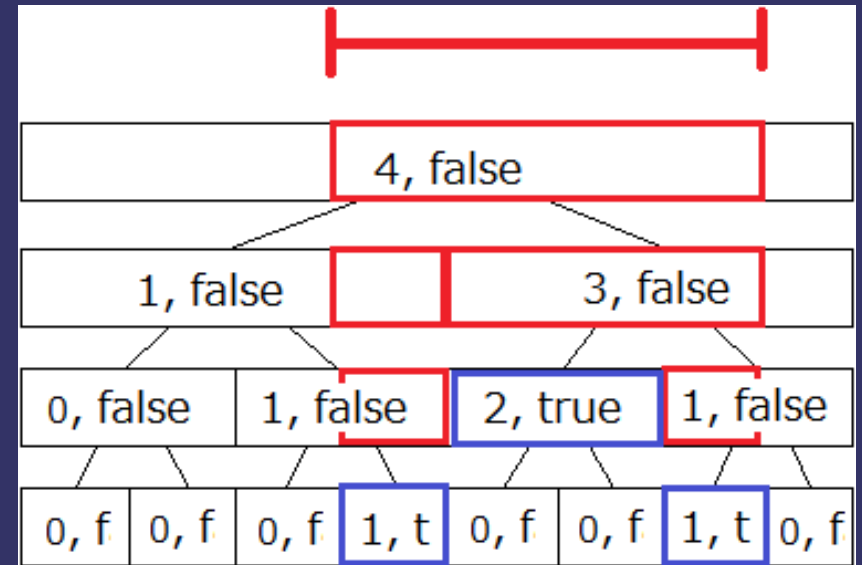


呼び出しは $O(\log K)$ 回



クエリの処理の仕方

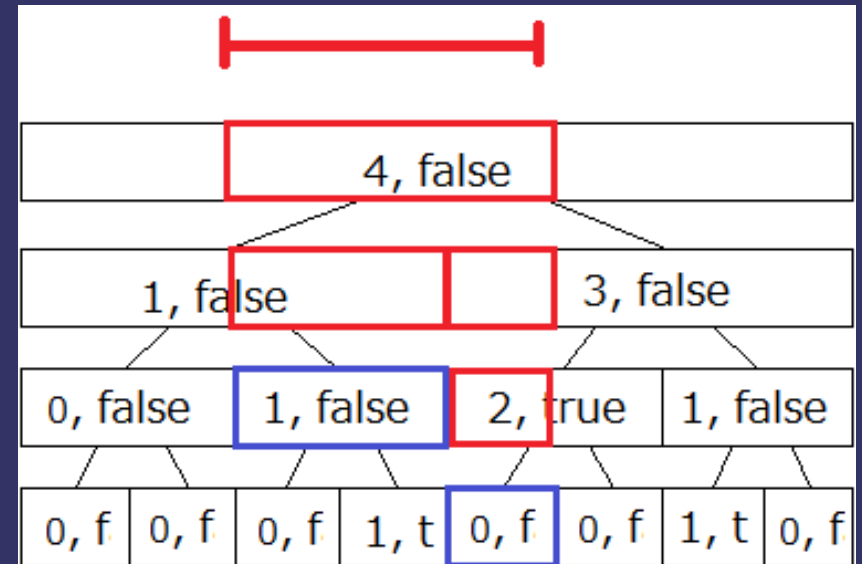
- ・ 区間全体を反転するところではフラグをtrueにしておく.
- ・ あとはボトムアップに



クエリの処理の仕方

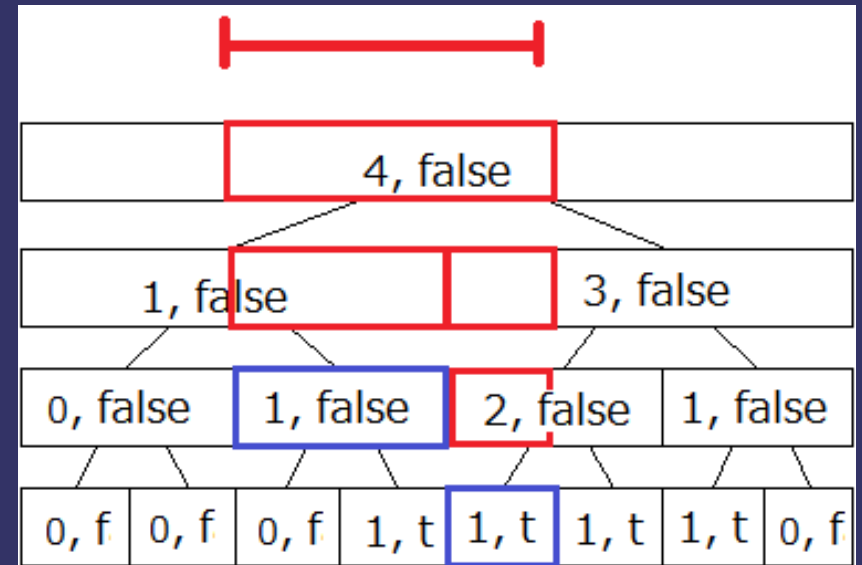
次のクエリ[3, 6]

さっきと同様にやるが、



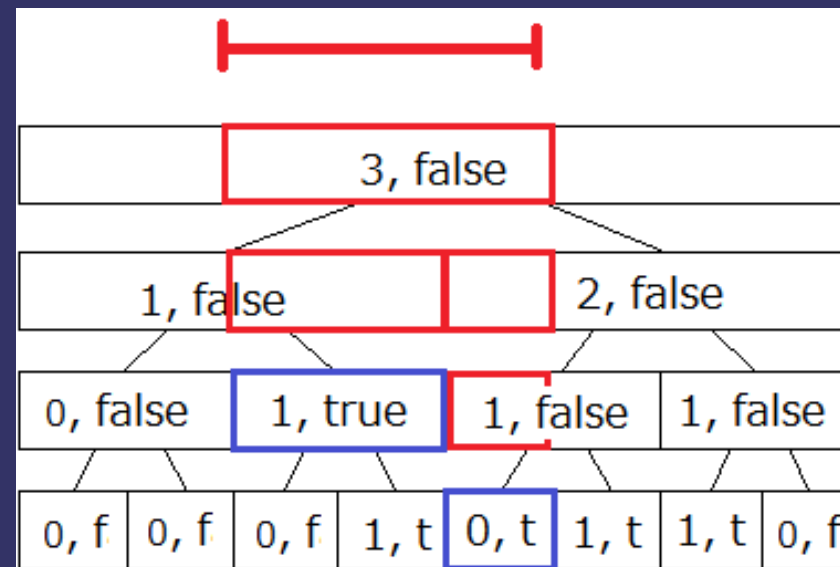
クエリの処理の仕方

- 途中でフラグがtrueになっているノードがあったら先に子にフラグを伝播しておく

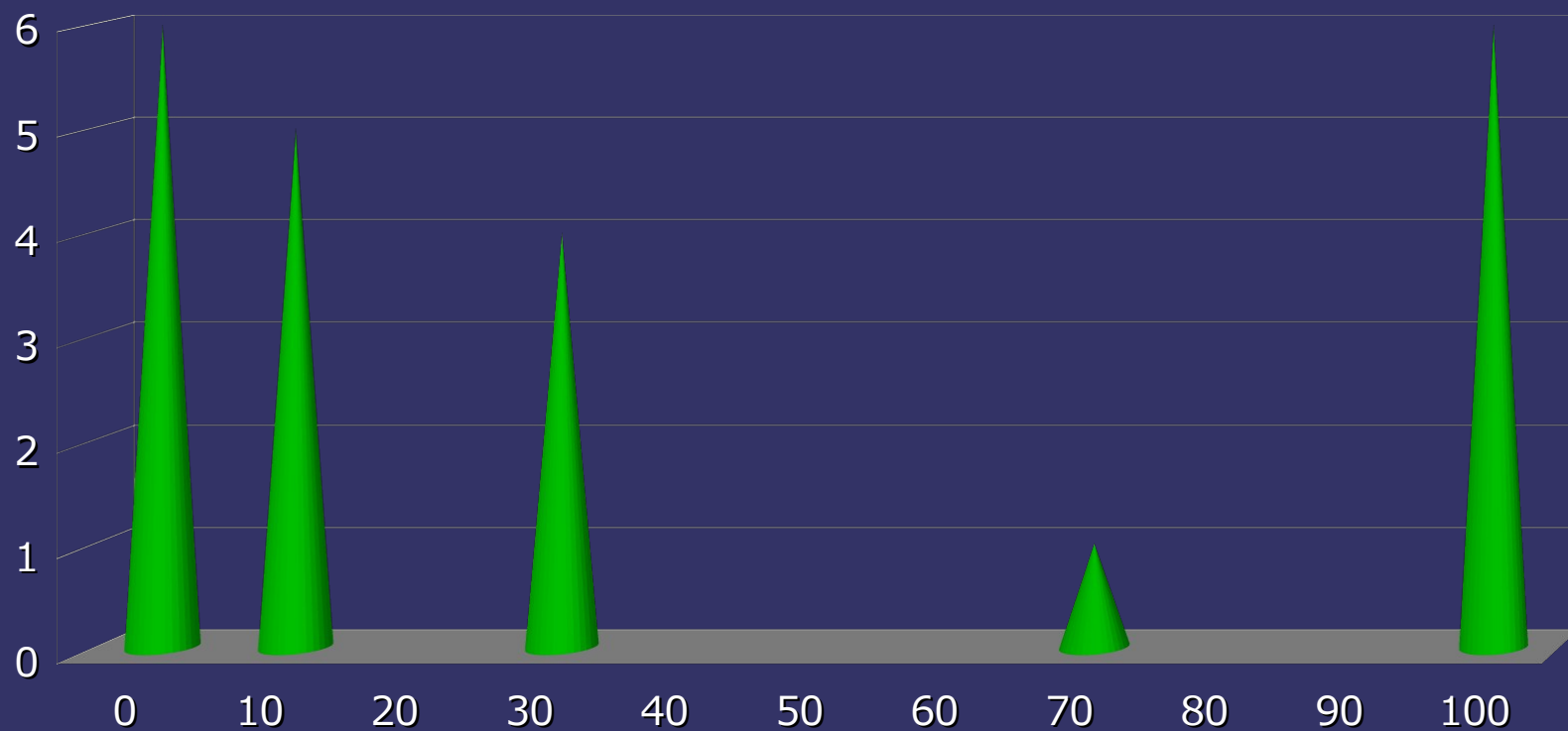


クエリの処理の仕方

- あとはさっきと同様に
ボトムアップに各ノード
に対応する区間の裏
の個数を求める
- 各操作 $O(\log K)$
→ 全体 $O(K \log K)$



得点分布

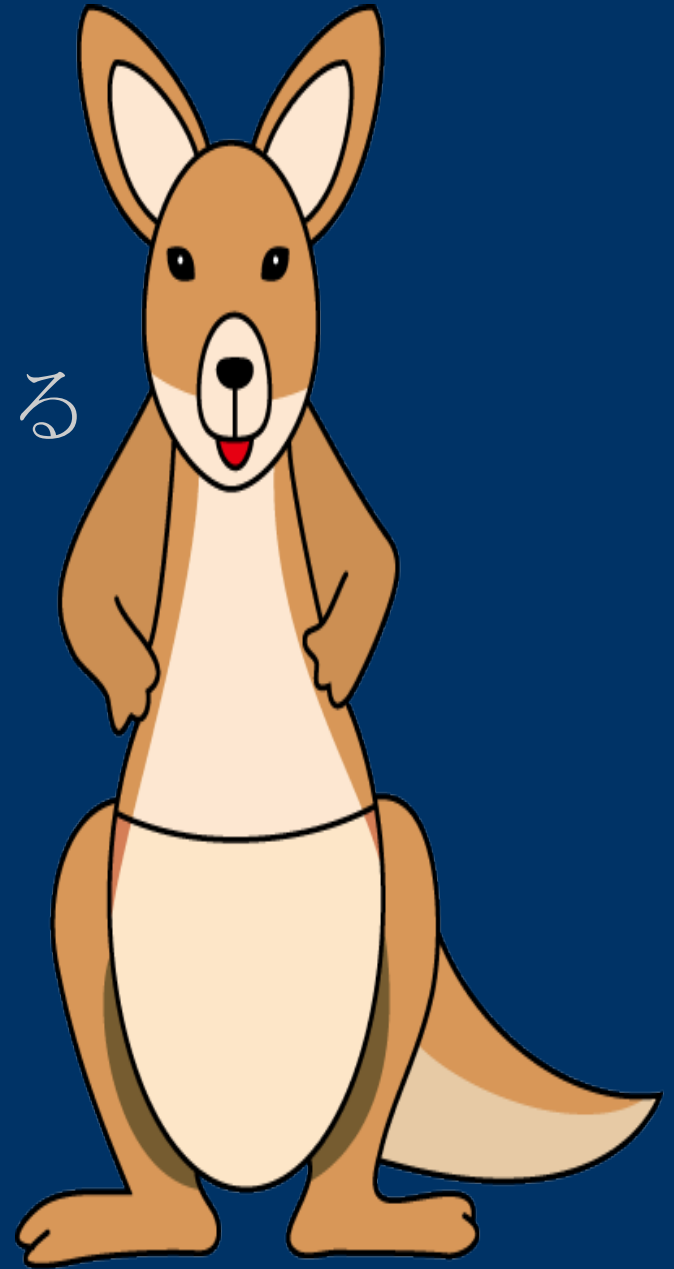
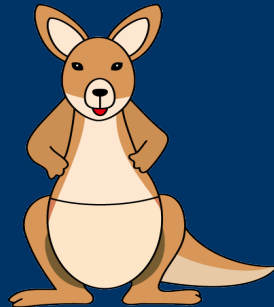
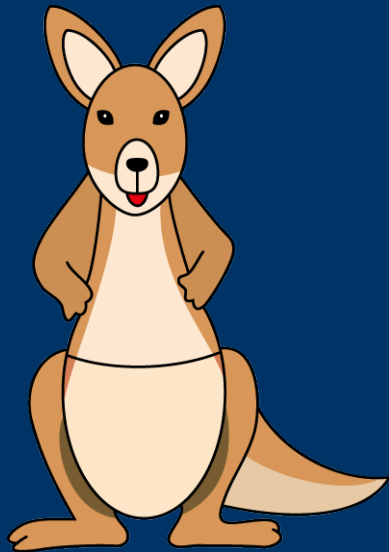


Kangaroo



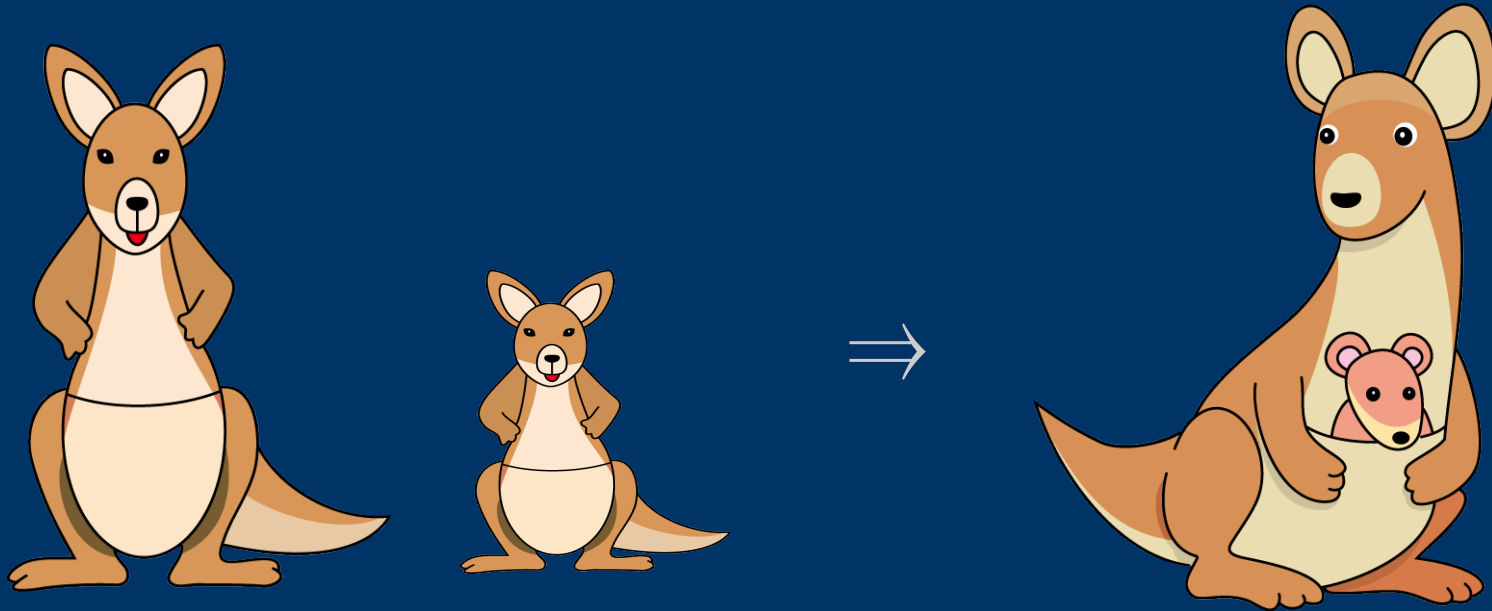
Problem Statement

本体とポケットからなる
カンガルーが N (≤ 300) 匹いる



Problem Statement

カンガルーは自分より大きい
ポケットに入ることができる



Problem Statement

操作できなくなる
まで繰り返すと
き、最後の状態は
何通りあるか



Sample Input 1

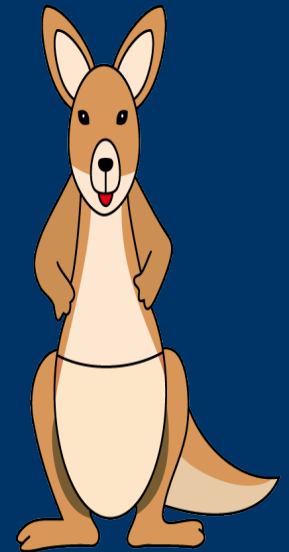
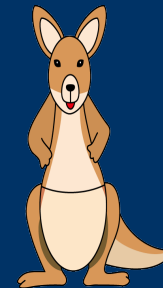
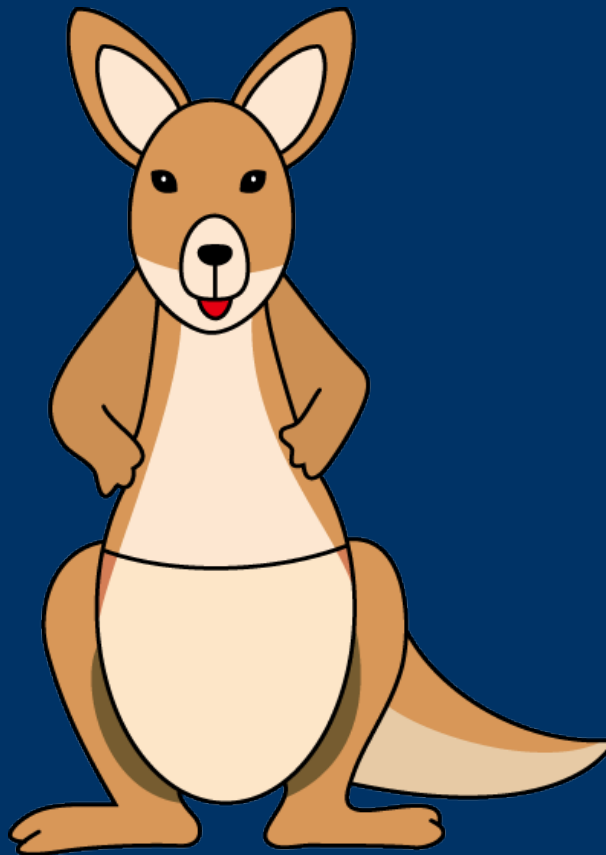
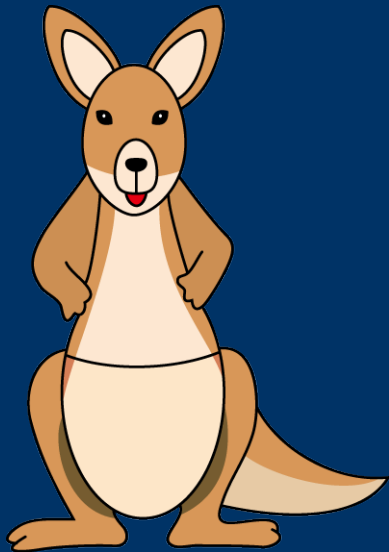
(6, 5)

(4, 3)

(3, 1)

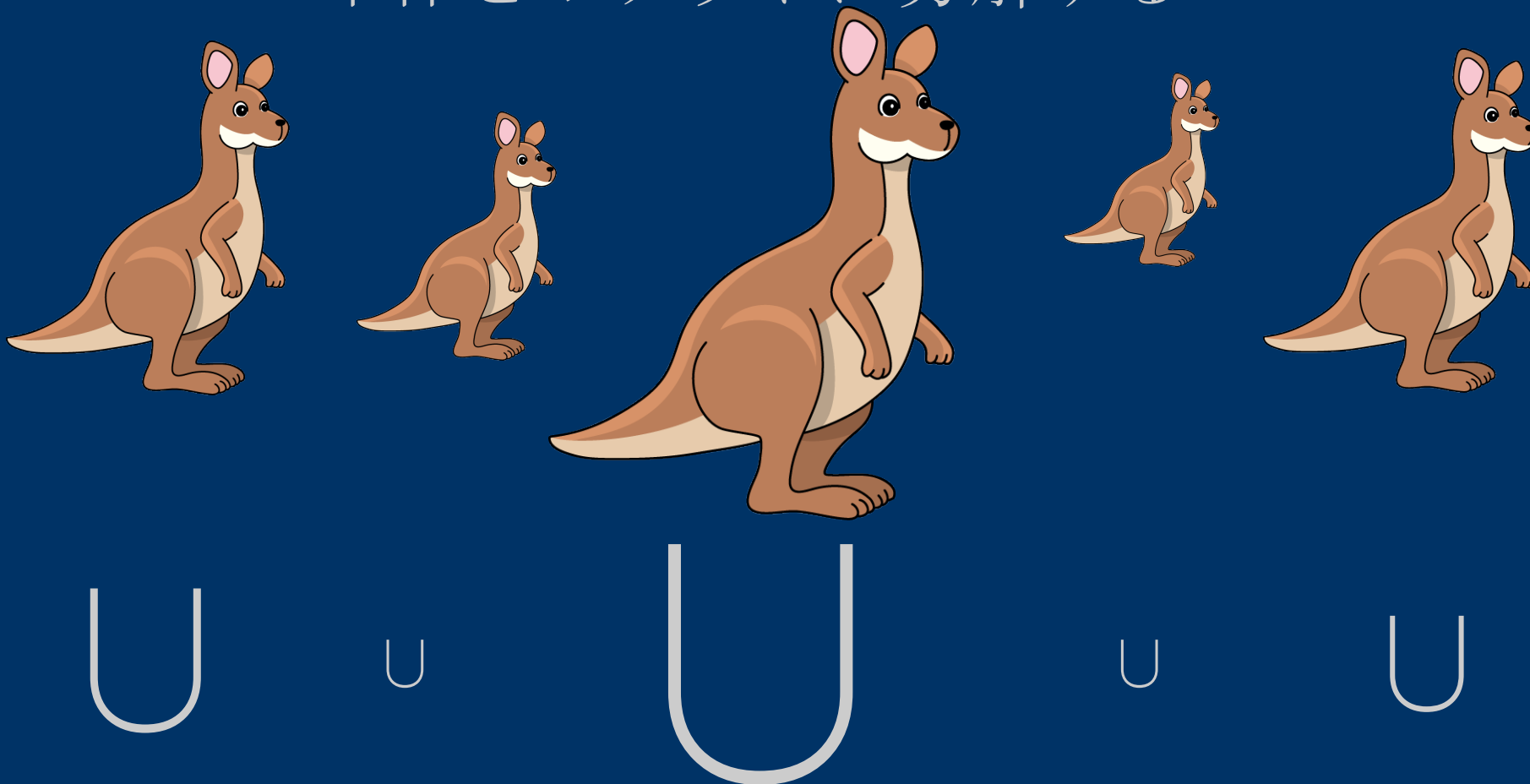
(4, 2)

(2, 1)



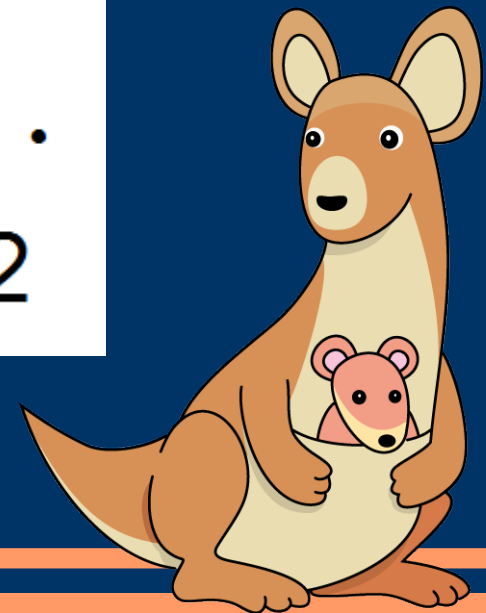
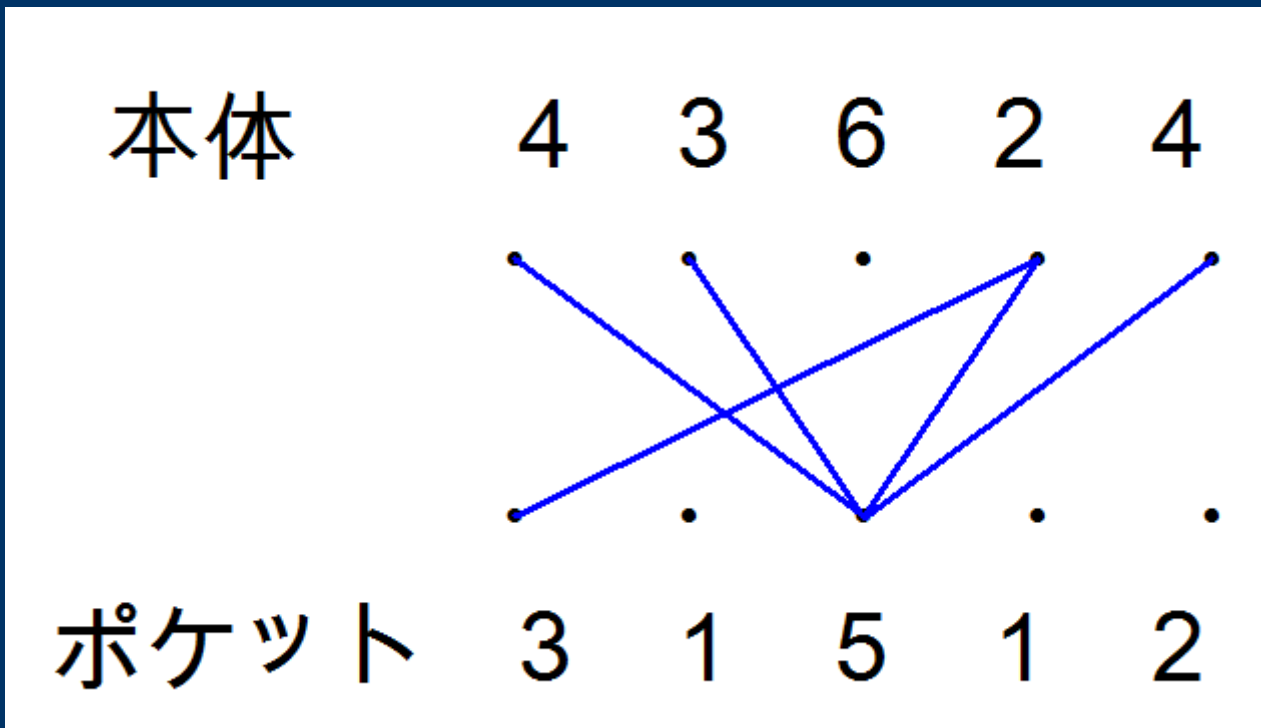
Sample Input 1

本体とポケットに分解する



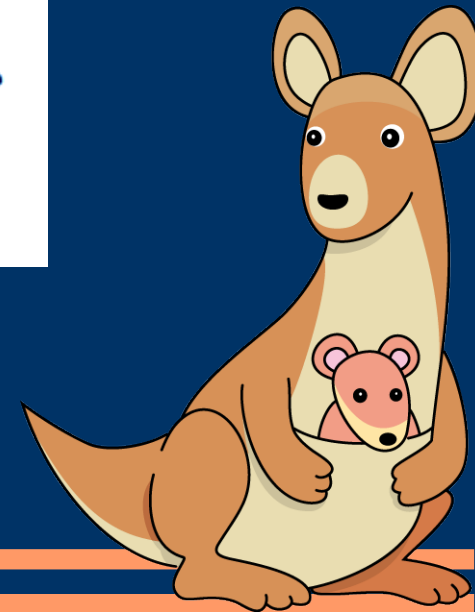
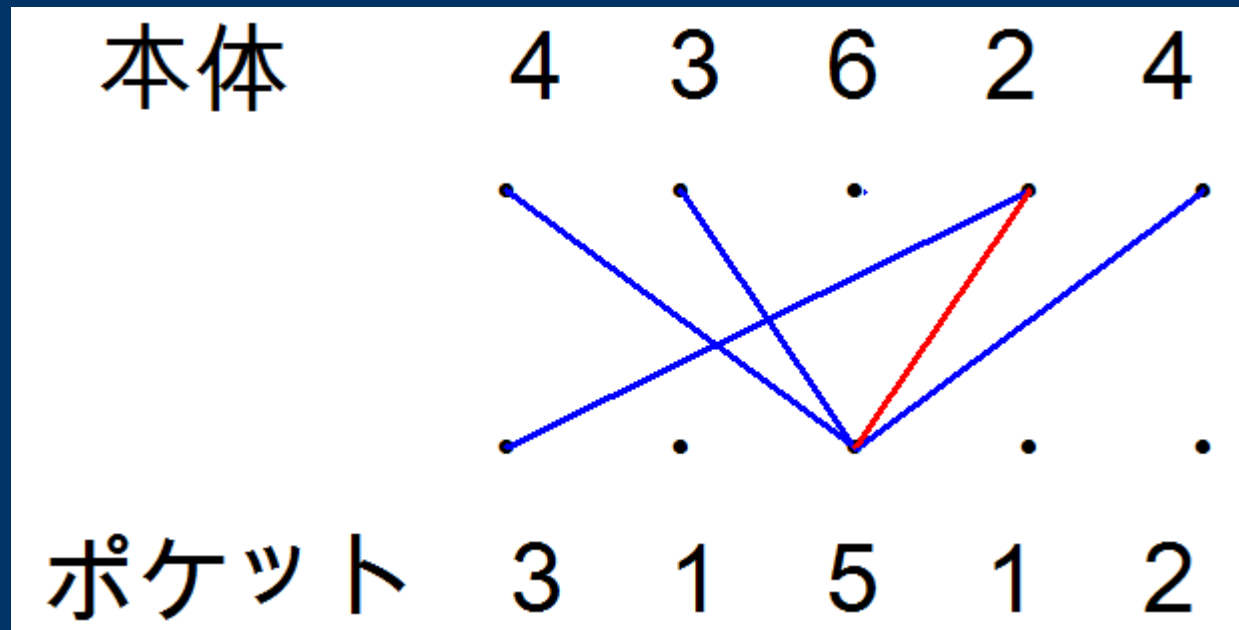
Sample Input 1

本体が他のポケットに入れるとき線で結ぶ



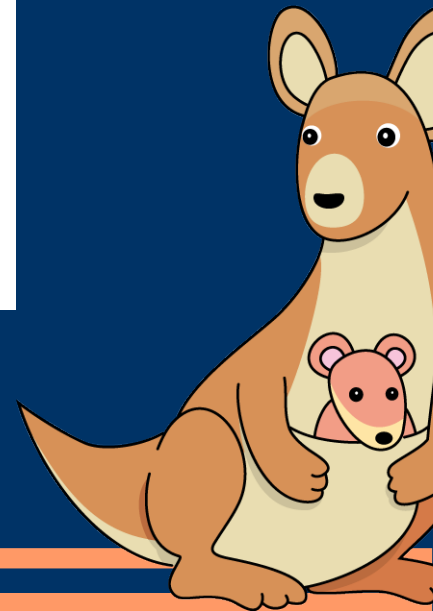
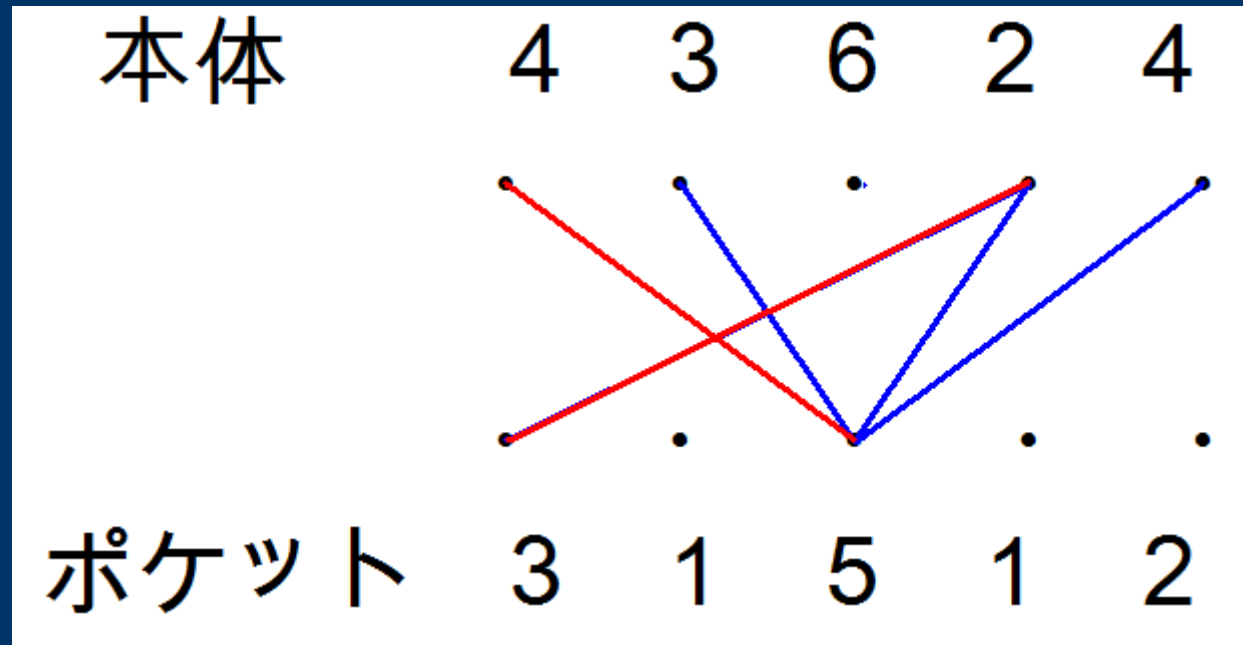
Sample Input 1

カンガルー4 がカンガルー3 のポケットに入っている.



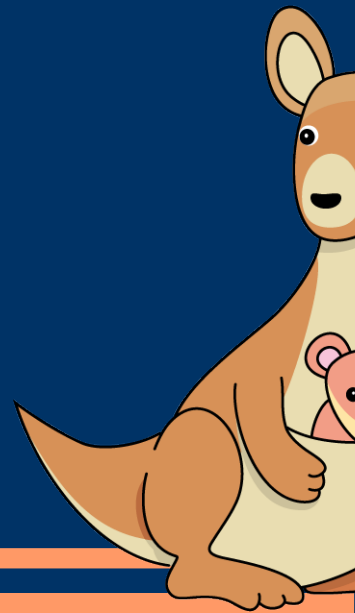
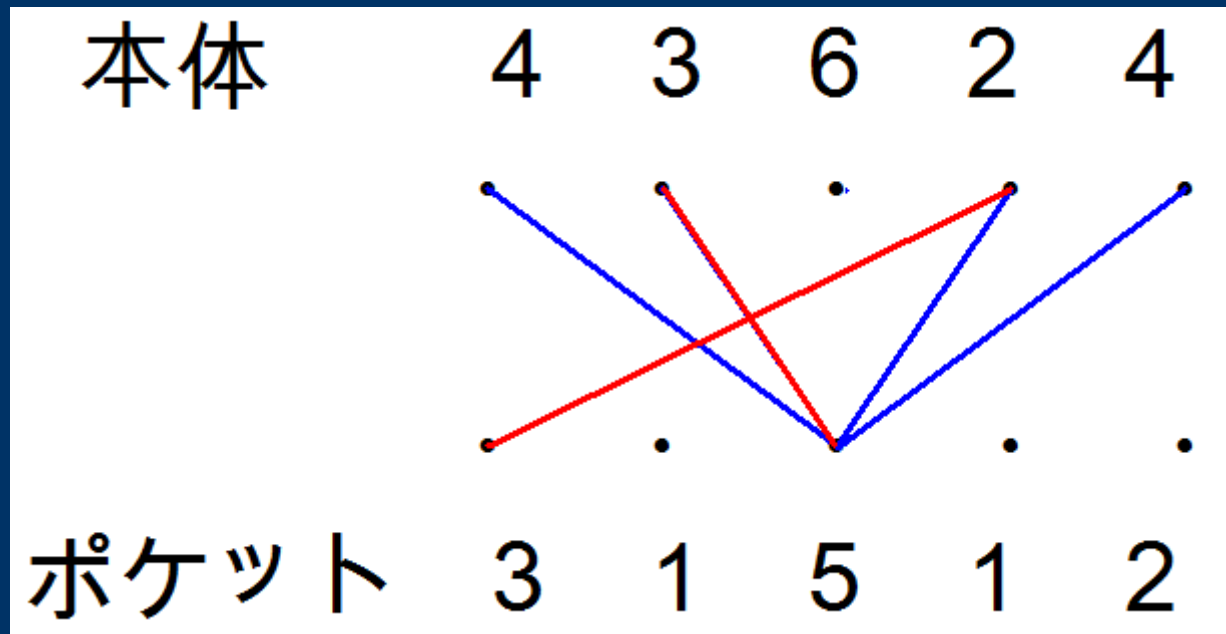
Sample Input 1

カンガルー4はカンガルー1のポケットに入っており，カンガルー1はカンガルー3のポケットに入っている．



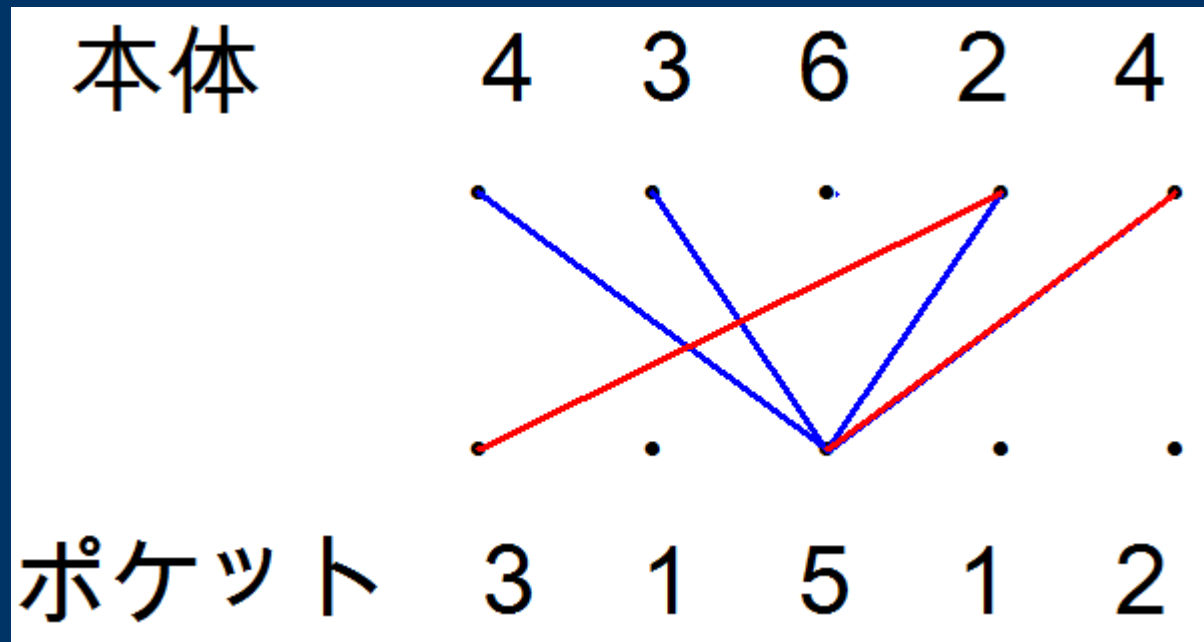
Sample Input 1

カンガルー4はカンガルー1のポケットに入っており，カンガルー2はカンガルー3のポケットに入っている．



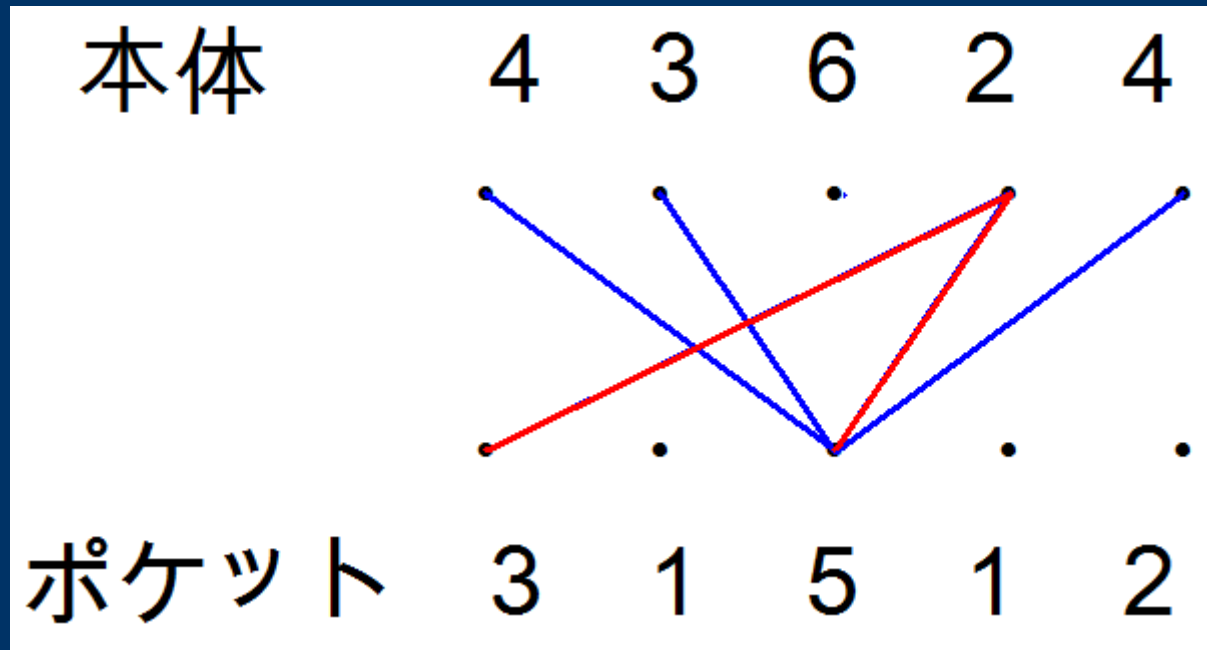
Sample Input 1

カンガルー4はカンガルー1のポケットに入っており，カンガルー5はカンガルー3のポケットに入っている．



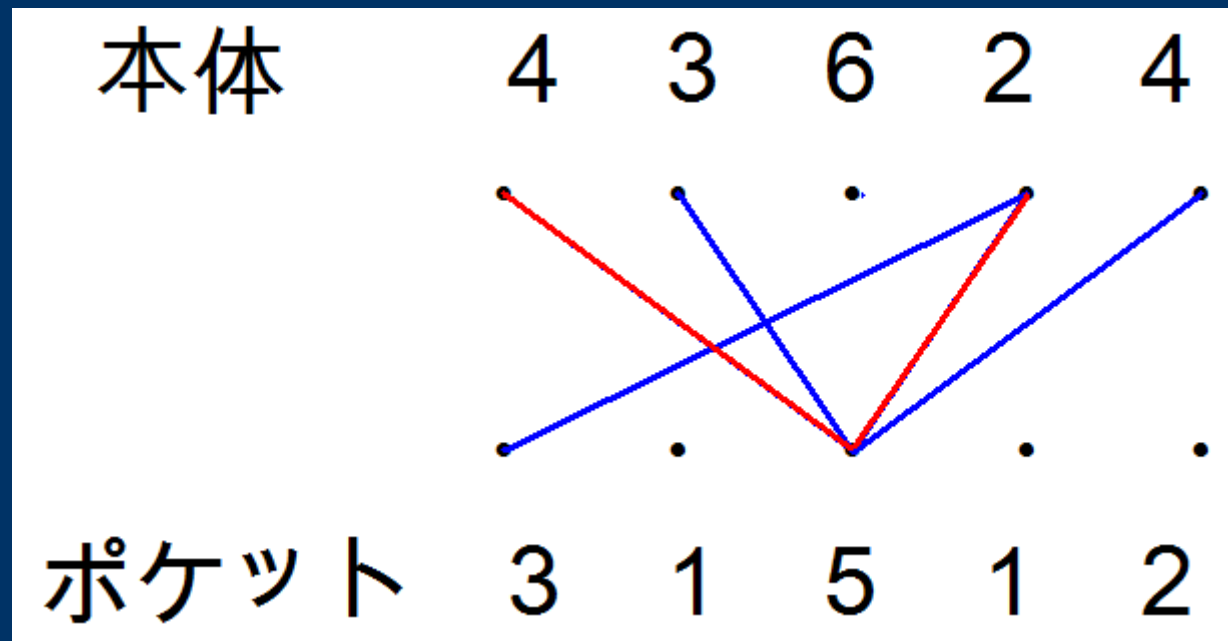
Sample Input 1

だめな例（カンガルー 4 の本体が 2 つのポケットに入っている）



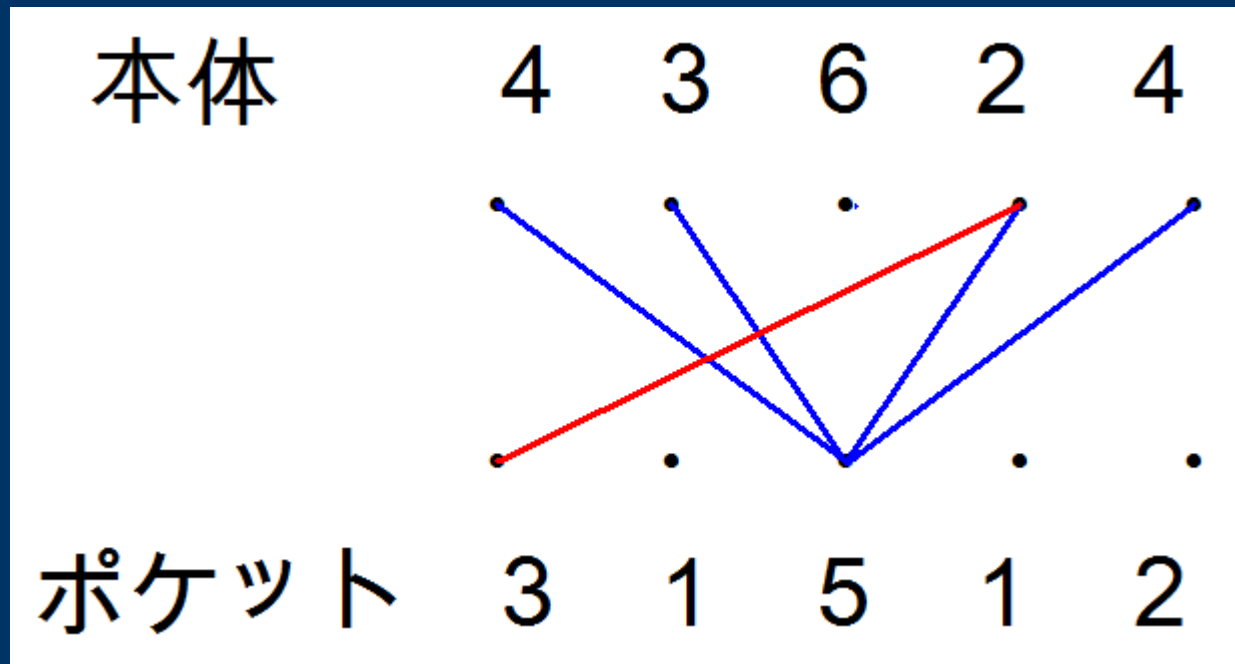
Sample Input 1

だめな例 (カンガルー 3 のポケットに複数のカンガルーが入っている)



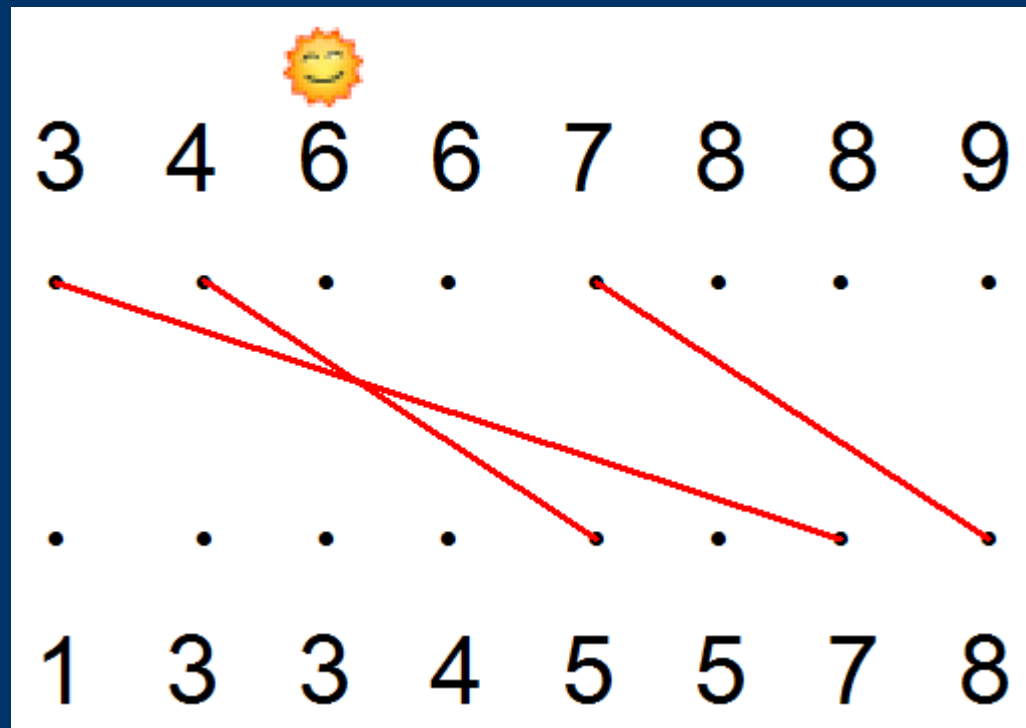
Sample Input 1

だめな例（操作が終わっていない）

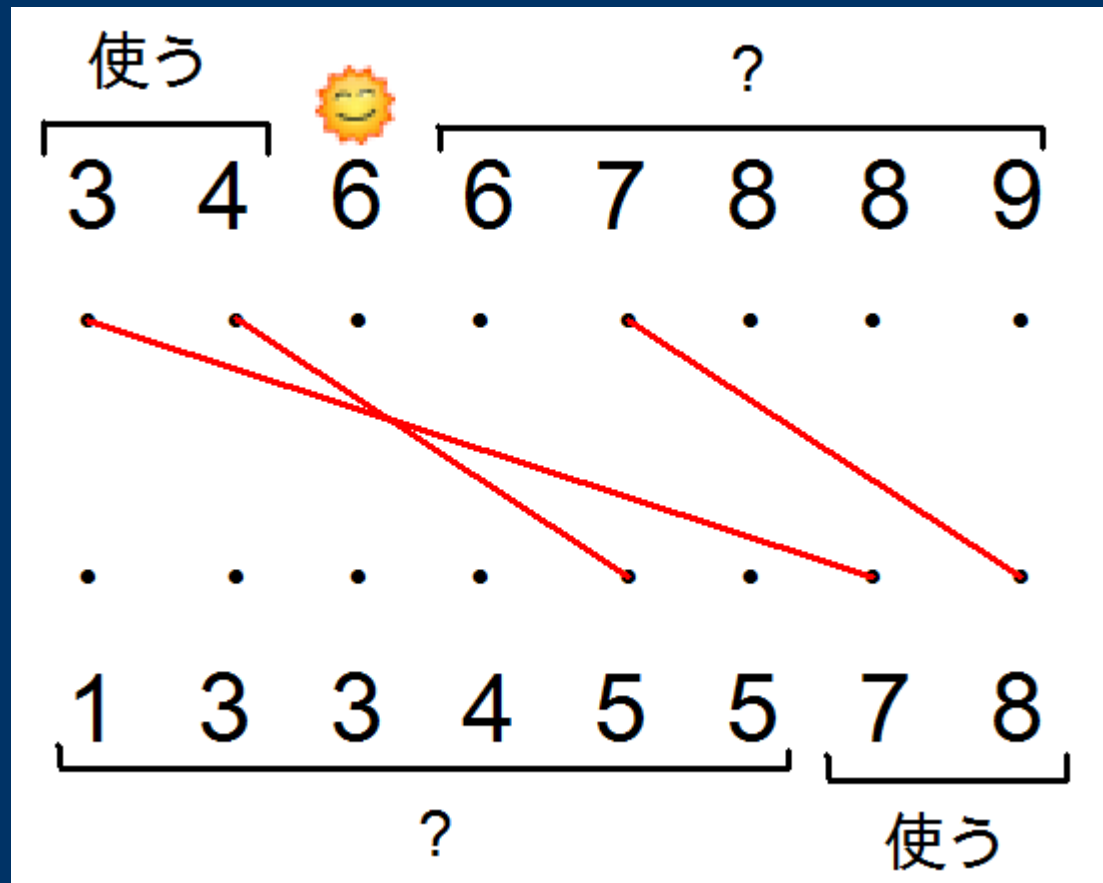


Solution

本体とポケットをそれぞれソートし、最も小さい使われていない本体によって場合分けする

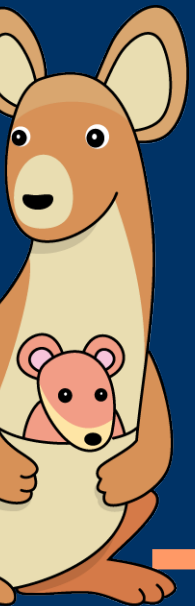
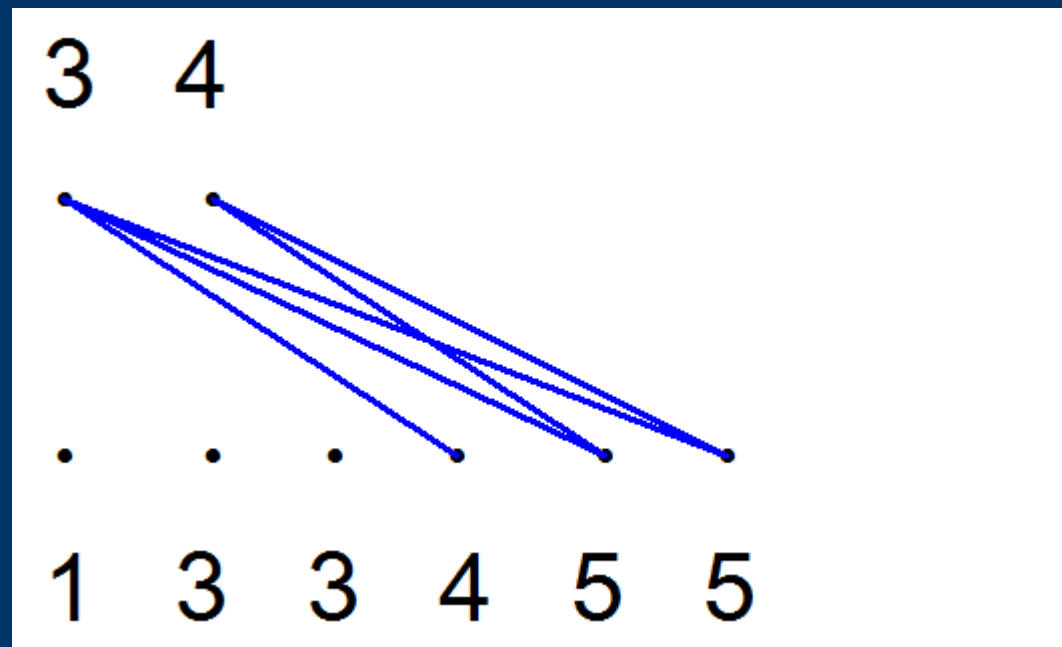


Solution



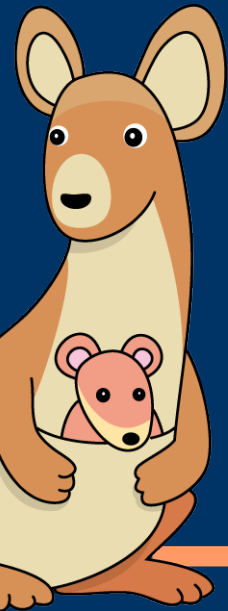
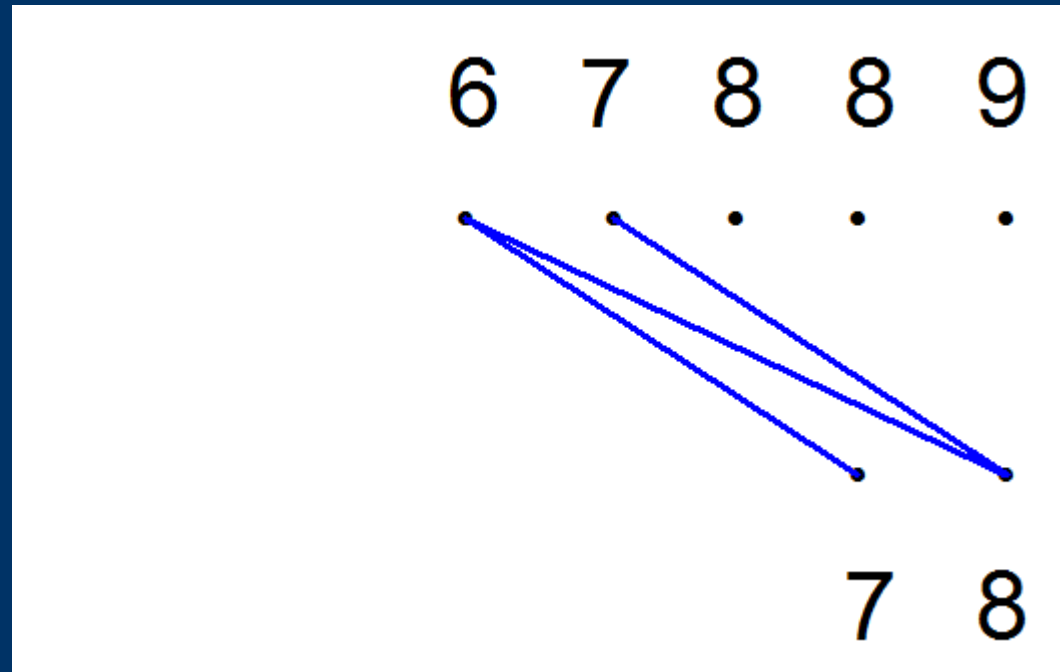
Solution

左側から k 組を選ぶ方法が $a[k]$ 通りある
とする (簡単な DP により求められる)



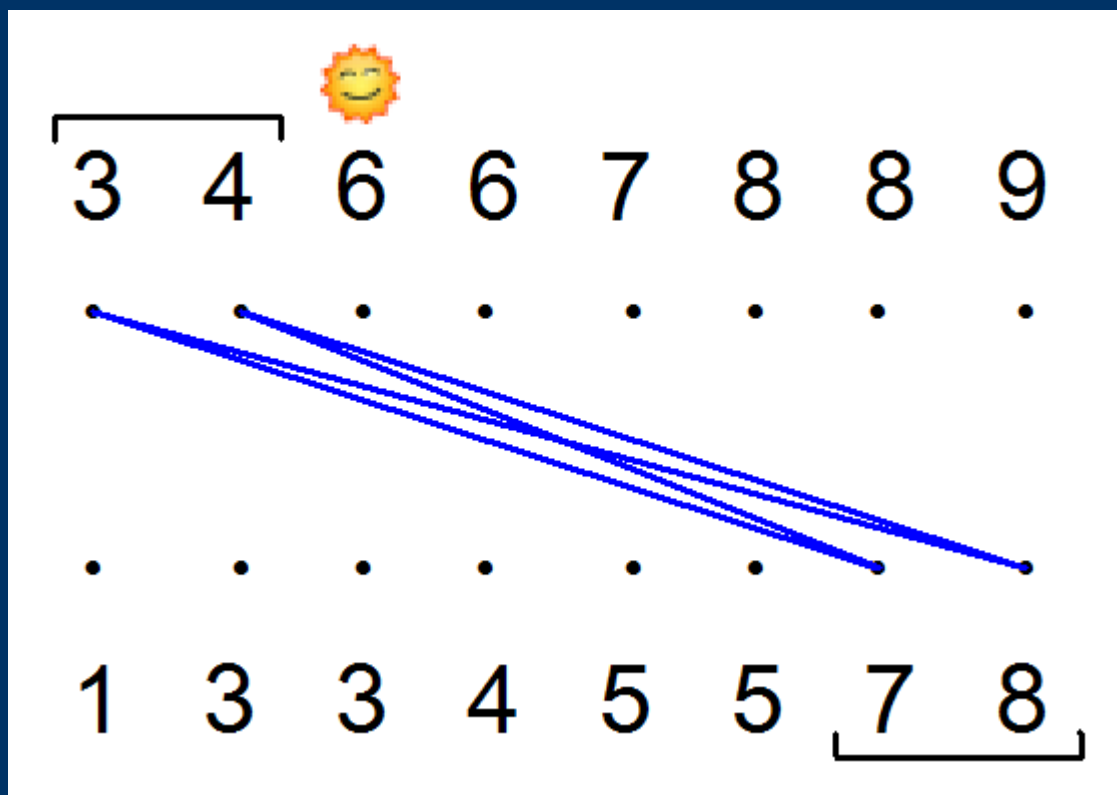
Solution

右側から k 組を選ぶ方法が $b[k]$ 通りある
とする (簡単な DP により求められる)



Solution

上段左側と下段右側からなる組の個数を決めると、 a と b の積によって何通りあるか求められる

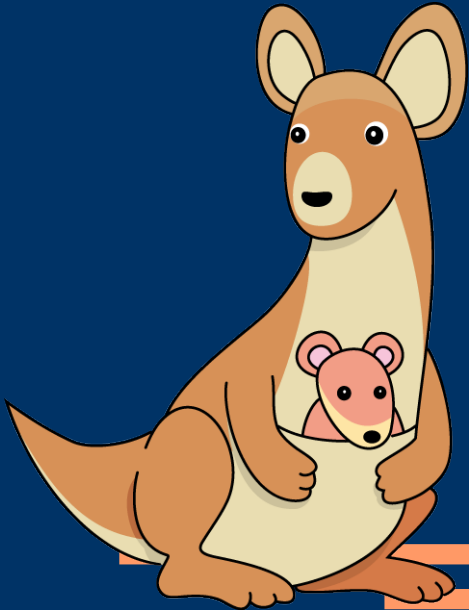


Solution

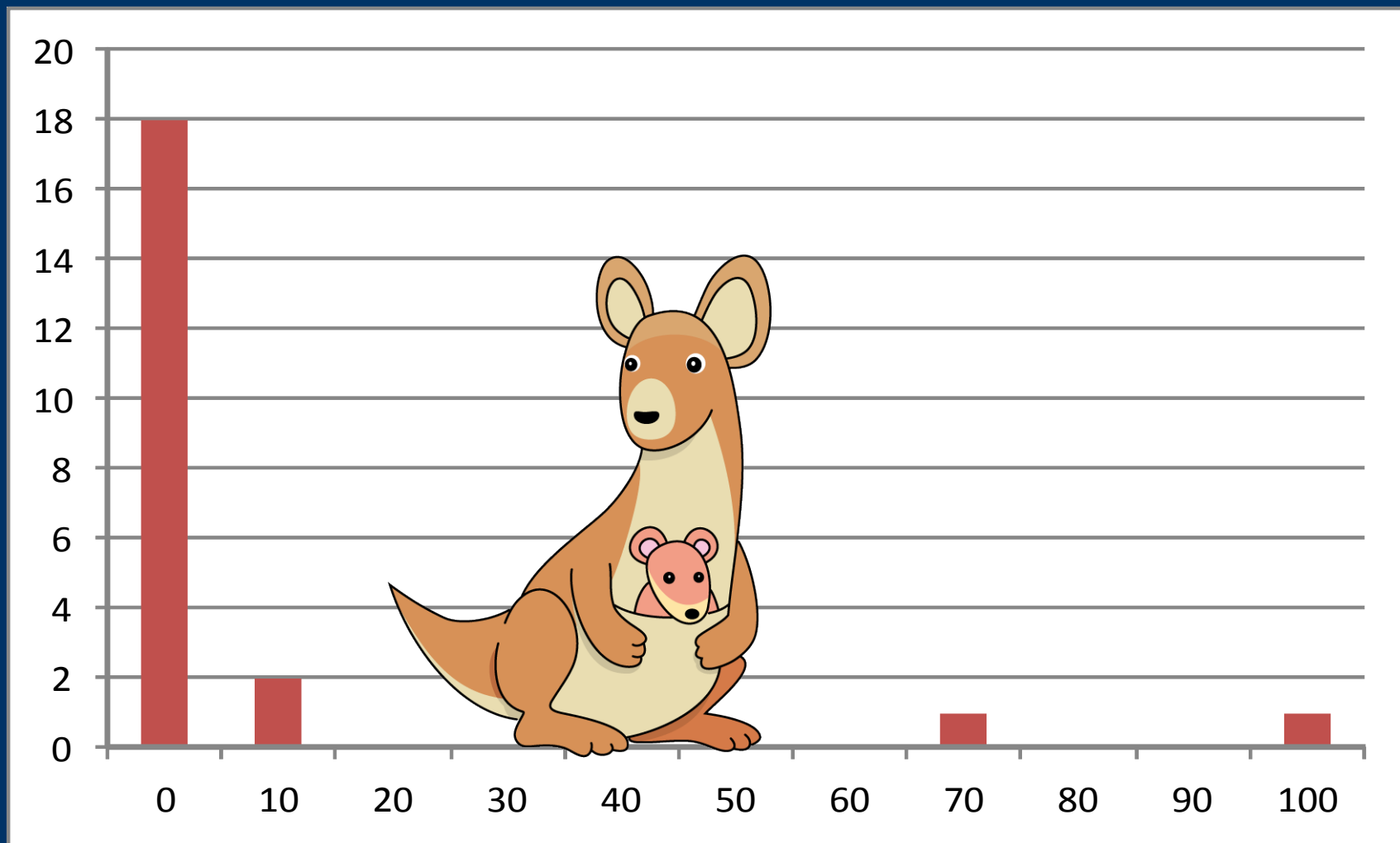
計算量

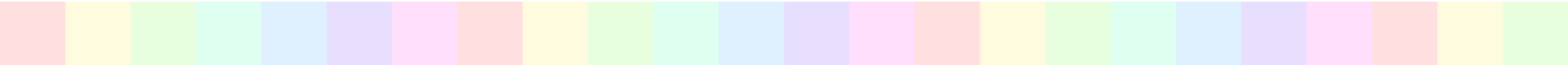
 の場所の決め方 : N
DP : $O(N^2)$

$O(N^3)$



得点分布

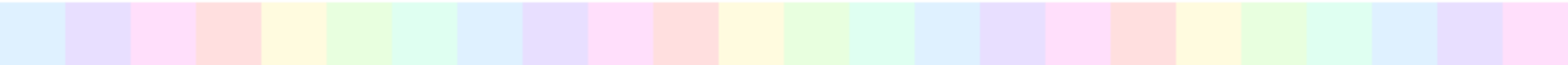




倉庫番 (Sokoban)

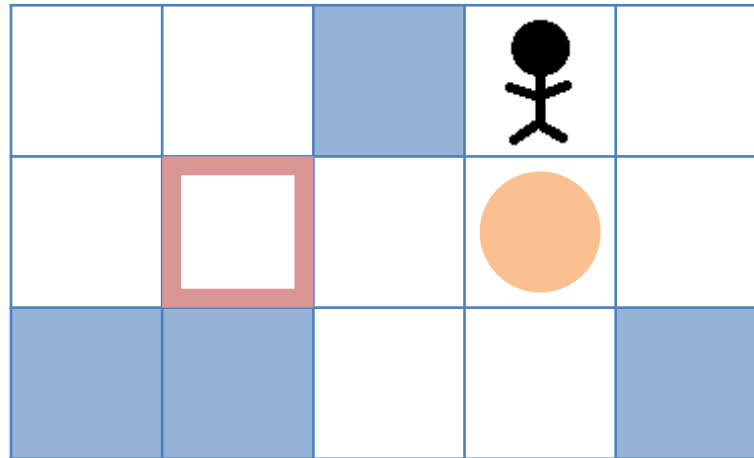
JOI 春合宿 2012 Day 3

解説： 保坂 和宏



問題概要

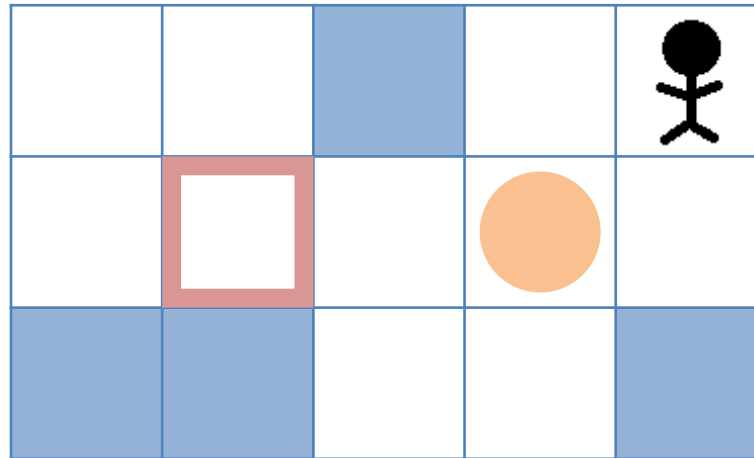
- 倉庫番



– 箱を押して目標地点に動かす

問題概要

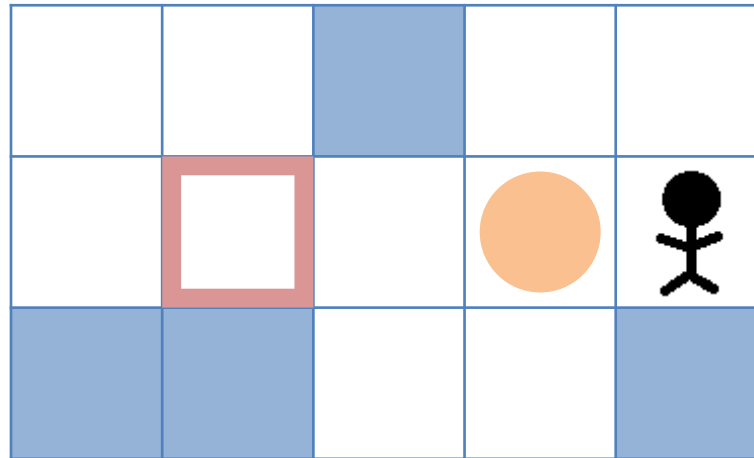
- 倉庫番



– 箱を押して目標地点に動かす

問題概要

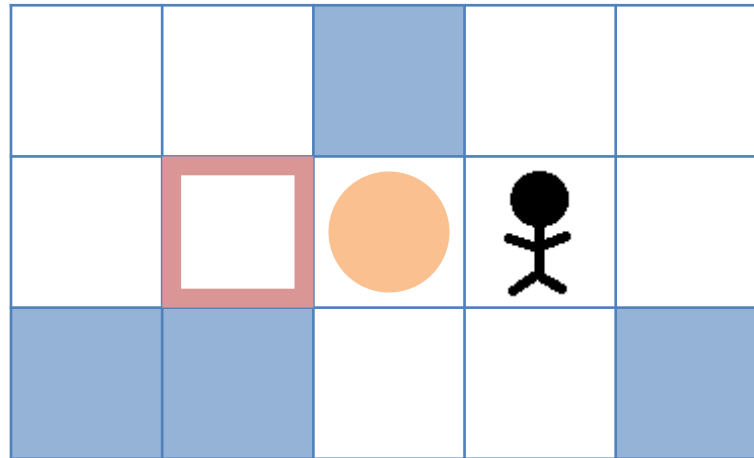
- 倉庫番



– 箱を押して目標地点に動かす

問題概要

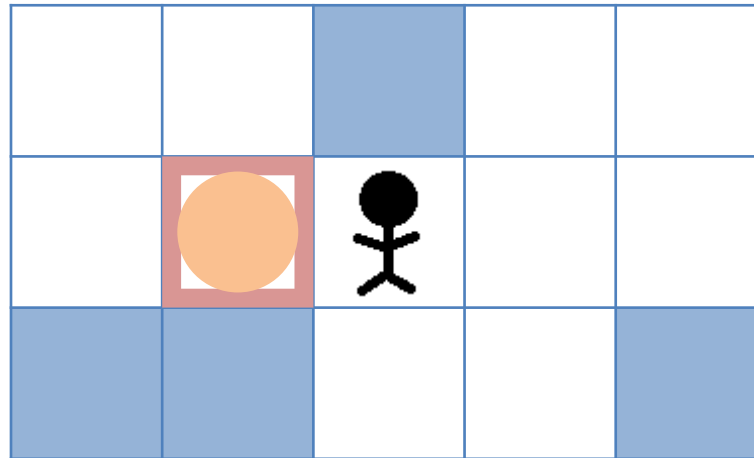
- 倉庫番



– 箱を押して目標地点に動かす

問題概要

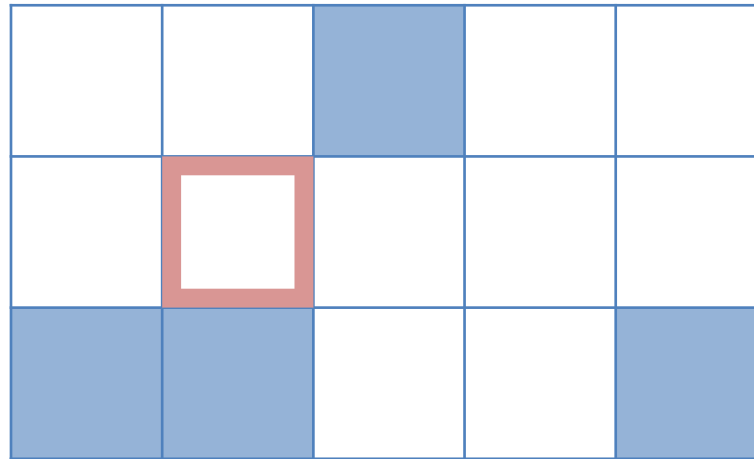
- 倉庫番



– 箱を押して目標地点に動かす

問題概要

- 倉庫番の問題が何通りあるか数える



- 壁と目標地点が与えられ, 人と箱を置く
- 盤面のサイズ: $M, N \leq 1,000$

単純な解法

1. 人と箱を置く場所をすべて試す
2. 実際に倉庫番の問題を解く

- 問題は $O(M^2N^2)$ 通り
- 解くのは, 人と箱の位置を状態として探索すれば $O(M^2N^2)$
- 全体で $O(M^4N^4)$
 - 0 点 ~ 10 点 ?

単純な解法

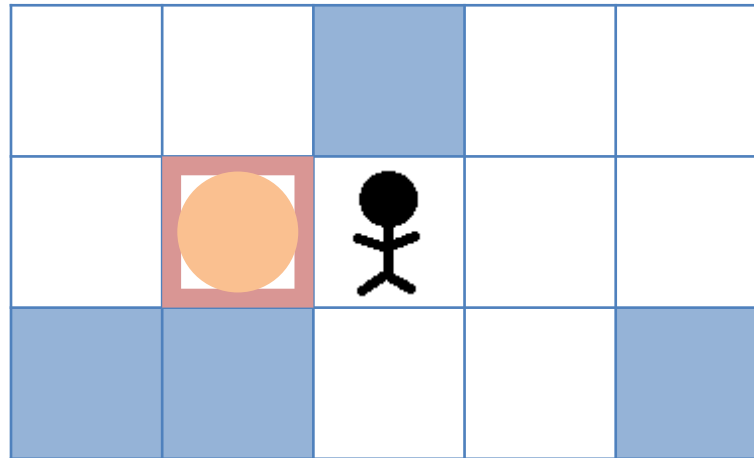
- 単純な解法のどこが無駄か？
→同じ状態に対して何回も計算
- 解けるような人と箱の位置の組を効率的に列挙したい

部分点解法

- ある人と箱の位置から解けるということは、目標地点から箱を引っぱってきてその人と箱の位置を作れることと同じ

部分点解法

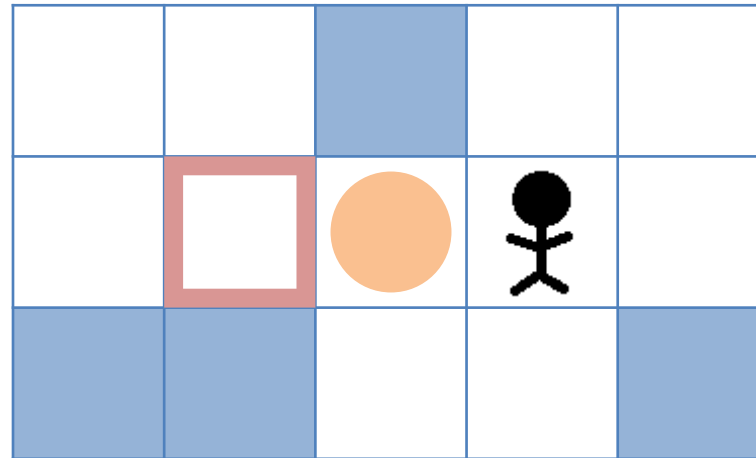
- 倉庫番の逆



- 目標地点から箱を引っぱる

部分点解法

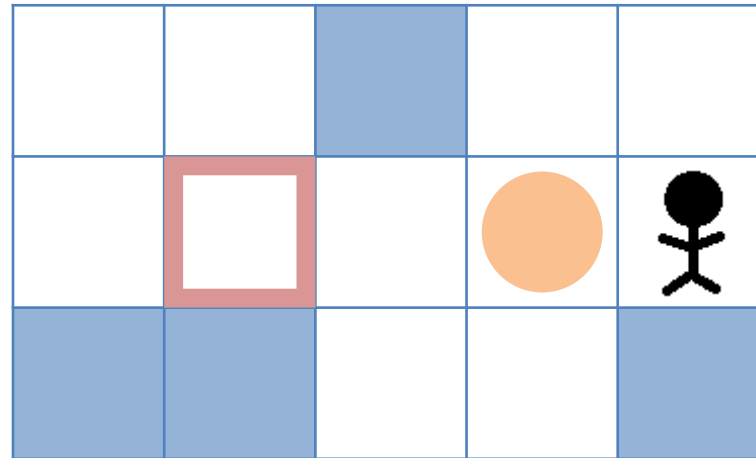
- 倉庫番の逆



– 目標地点から箱を引っぱる

部分点解法

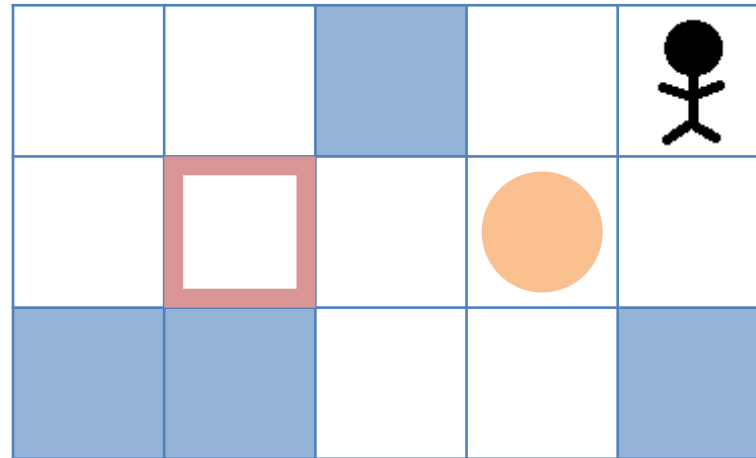
- 倉庫番の逆



– 目標地点から箱を引っぱる

部分点解法

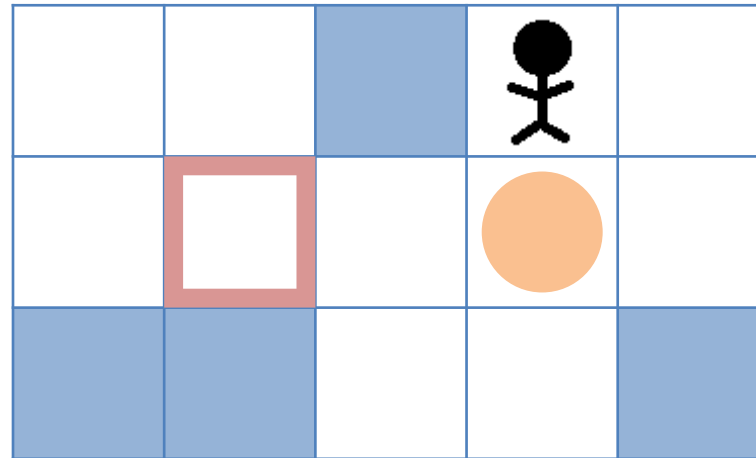
- 倉庫番の逆



– 目標地点から箱を引っぱる

部分点解法

- 倉庫番の逆



– 目標地点から箱を引っばる

部分点解法

- 倉庫番の逆
 - 初期状態：箱は目標地点，人は箱に隣接
 - 移動：
 - 人が動く
 - 人が箱を引っぱって動く
- 幅優先探索などで $O(M^2N^2)$
 - 10 点 ～ 20 点？

アイデア

- 倉庫番の逆
 - 初期状態：箱は目標地点，人は箱に隣接
 - 移動：
 - 人が動く
 - 人が箱を引っぱって動く
- 人は壁と箱で区切られた連結成分内を自由に動ける

アイデア

- 状態
 - 箱のあるマスと人のいるマス
 - 各状態に対して答に 1 を足す
- ↓
- 箱のあるマスと人のいる連結成分
 - 各状態に対して答に連結成分のサイズを足す

アイデア

- 目標地点と繋がっていないマスは無視
 - 壁にしてしまえばよい
 - 全体が連結になる
- 連結な状態から箱を置いて 1 マスを塞いでも連結成分は高々 4 個
 - 状態数が $O(MN)$

アイデア

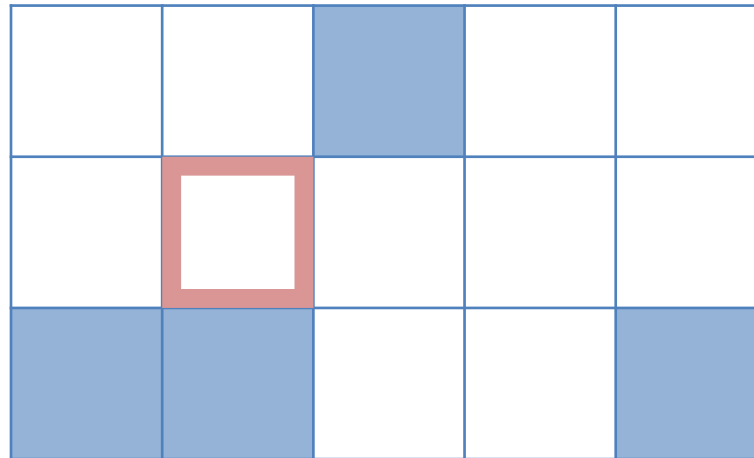
- グリッドグラフを考える
 - 次数が高々 4
 - 箱を 1 個置く = 頂点を 1 個取り除く
 - 頂点を 1 個取り除いたときの状況を知りたい
 - 箱の 4 方向のマスの接続関係と, それぞれの連結成分のサイズ
 - 関節点 (取り除いたら非連結) に関連している? !

接続関係

- 箱の周囲 4 マスの接続関係
 - グラフの各頂点 u に対してそれを取り除いたときの隣接頂点たちの接続関係
- より一般に「頂点 u を取り除いたときに頂点 a と b は同じ連結成分に属するか」というクエリに答える方法を紹介
 - 今回はこれを行わずとも解けるようです

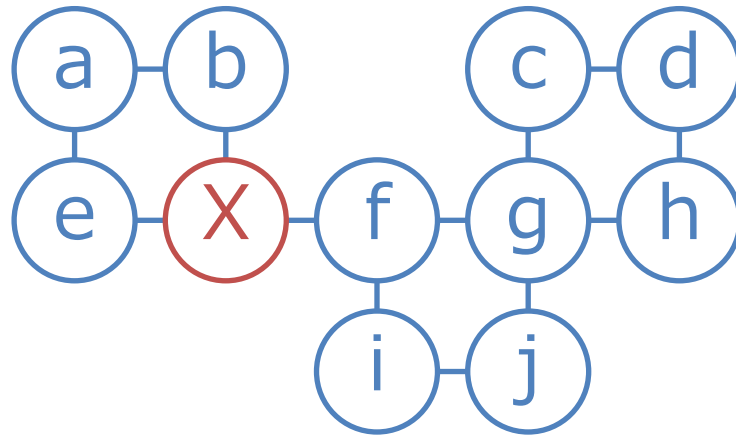
DFS 木

- DFS 木を作る
 - 根はどこでもよいので目標地点からでも

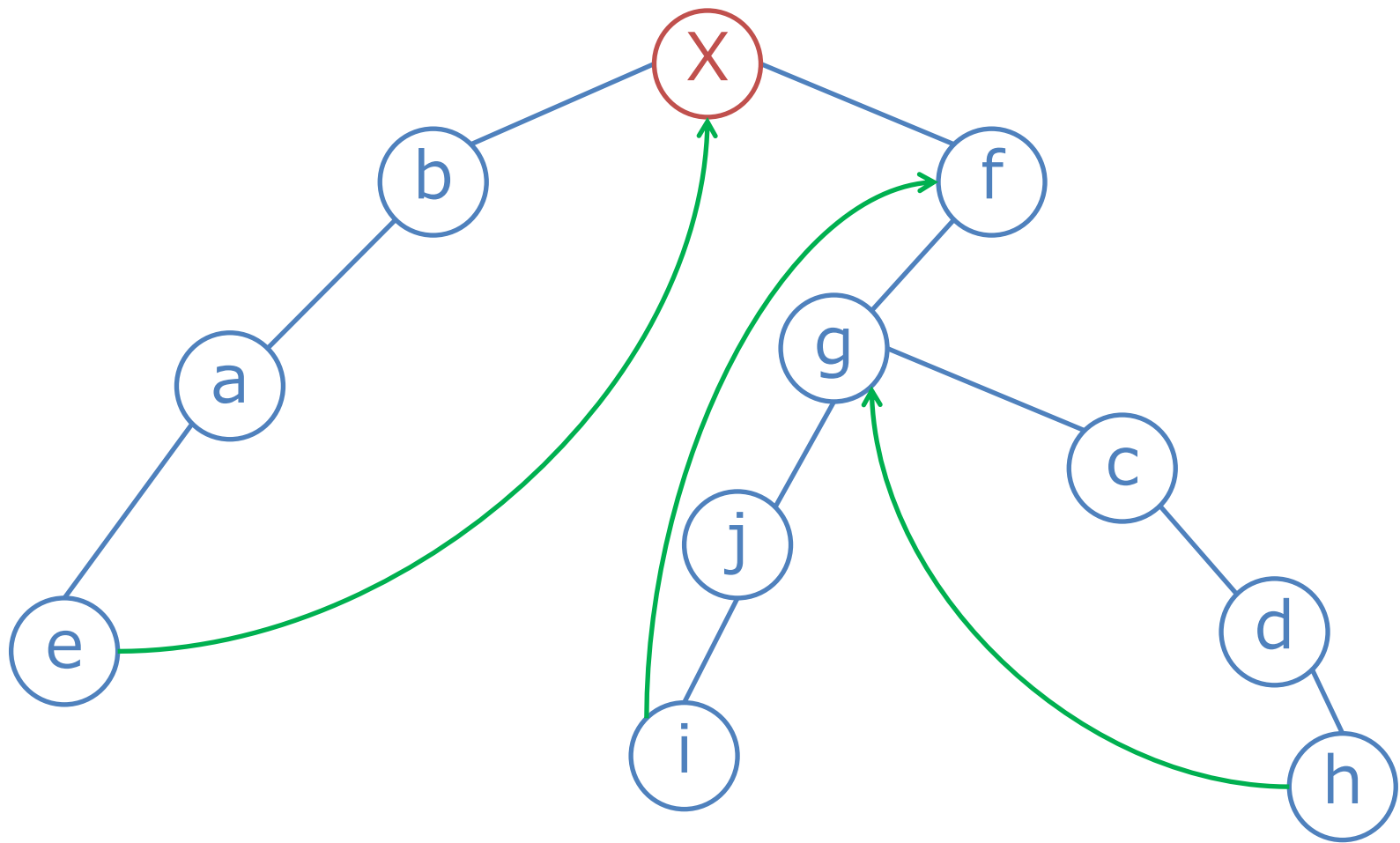


DFS 木

- DFS 木を作る
 - 根はどこでもよいので目標地点からでも



DFS 木



DFS 木

- DFS 木による 2 種類の辺
 - 木の辺
 - 後退辺
 - 祖先へ向かう
- 頂点を訪れた順に番号を付ける
- Lowlink を求める

DFS 木

- $dis[u]$: 頂点 u を訪れた時刻 (discover)
- $fin[u]$: 頂点 u を去った時刻 (finish)
- $low[u]$: 頂点 u の "Lowlink"
 - 頂点 u から「木の辺は子孫方向に何度でも」
「後退辺は 1 回だけ」用いて到達できる頂点
のうちの dis の最小値

DFS 木

function $DFS(u)$

$dis[u] := k, \quad k := k + 1.$

$low[u] := dis[u].$

for each $v : (u \text{ に隣接している頂点}):$

if (頂点 v をまだ訪れていない):

$DFS(v).$

$low[u] := \min(low[u], low[v]).$

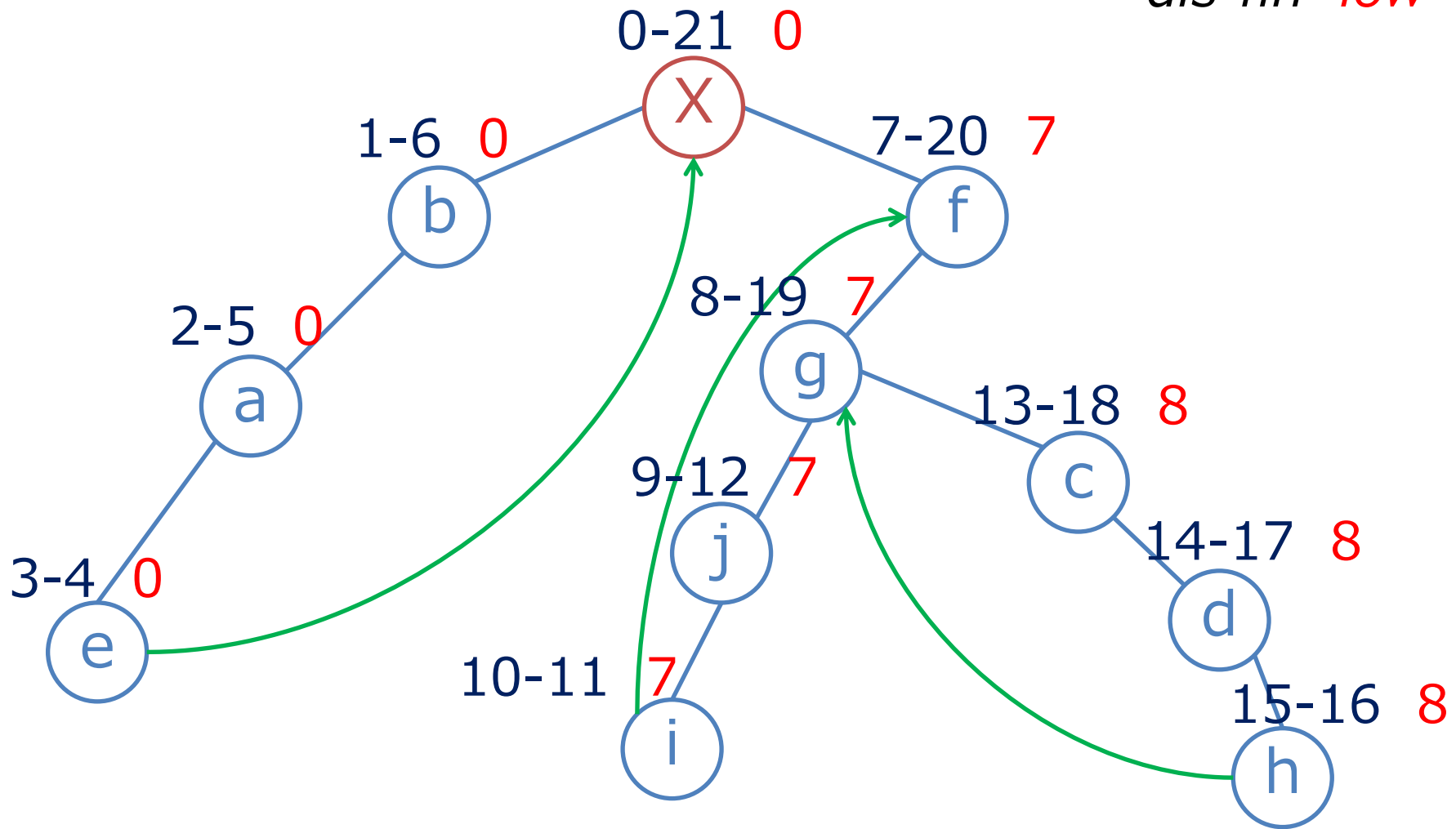
else if (v が u の親でない):

$low[u] := \min(low[u], dis[v]).$

$fin[u] := k, \quad k := k + 1.$

DFS 木

dis-fin *low*



便利関数

頂点 v が頂点 u の子孫か？

- $dis[u] \leq dis[v]$ かつ $fin[v] \leq fin[u]$

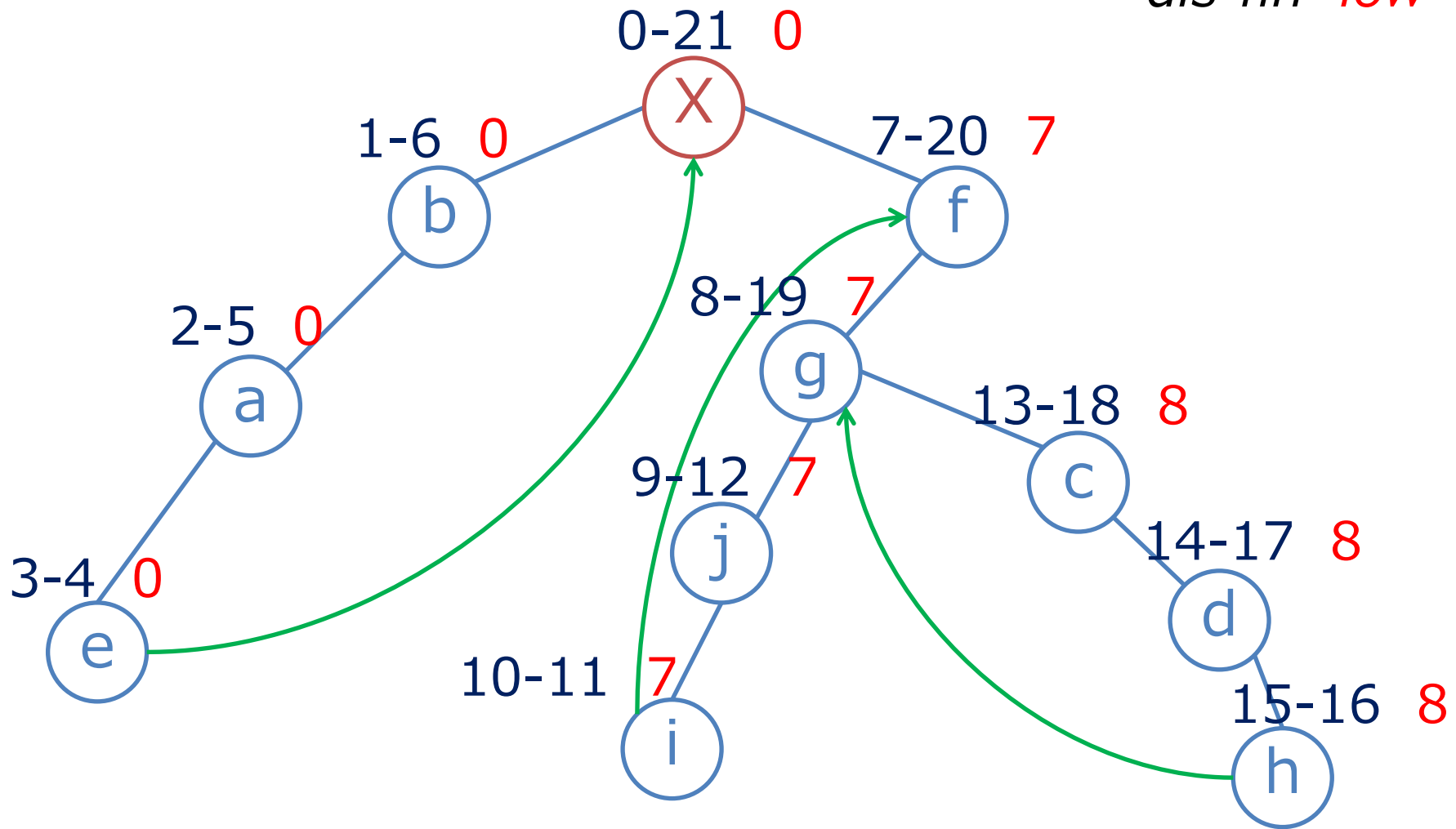
便利関数

頂点 v が頂点 u の子孫かつ $u \neq v$ のとき,
 u の子 w であって v が w の子孫であるものは？

- u の子すべてに対し先程の関数を使う
 - $O(u \text{ の次数})$ (今回はこれで OK)
 - 二分探索すると $O(\log (u \text{ の次数}))$ も可能

DFS 木

dis-fin *low*



接続関係

頂点 u を取り除いたときに頂点 a と b は同じ連結成分に属するか？

Case 1. a も b も u の子孫でないとき

- Yes
 - 影響がない

接続関係

Case 2. a は u の子孫で, b は u の子孫でないとき

- a を子孫にもつ u の子を v_a とする
- $low[v_a] < dis[u]$ なら Yes
 - a が v_a 側から脱出できる

接続関係

Case 3. a も b も u の子孫であるとき

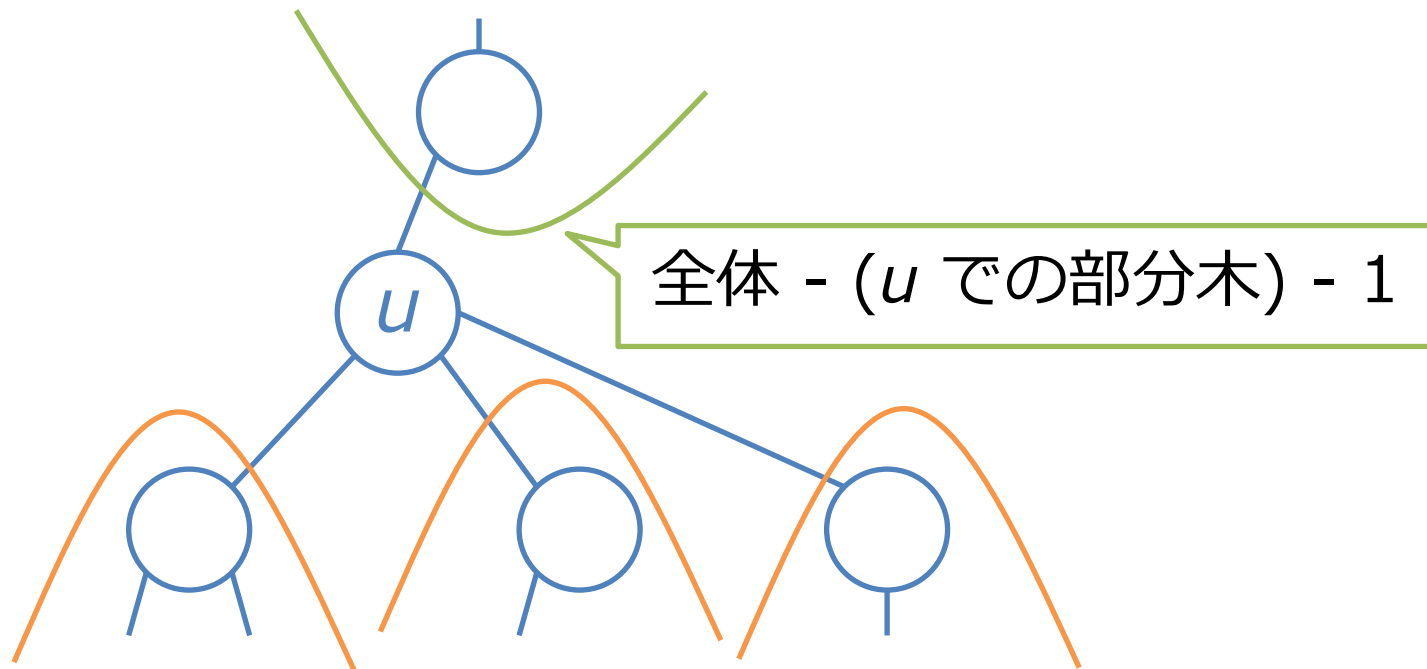
- a, b を子孫にもつ u の子を v_a, v_b とする
- 次のいずれかなら Yes
 - $V_a = V_b$
 - 脱出の必要なし
 - $low[v_a] < dis[u]$ かつ $low[v_b] < dis[u]$
 - a が v_a 側から, b が v_b 側から脱出できる

接続関係

- 箱の周囲 4 マスの接続関係
 - グラフの各頂点 u に対してそれを取り除いたときの隣接頂点たちの接続関係
 - 4 頂点の各組に対して同じ連結成分に属するか判定すればよい

連結成分のサイズ

- 各部分木のサイズを計算しておけばよい
 - u に木の辺で繋がっている頂点について部分木のサイズを足す



解法

- 状態：箱のあるマスと人のいる連結成分
- 遷移：箱の引っぱり方 (高々 4 通り)
- 各状態にたどり着くたびに、連結成分のサイズを答に足す
- $O(MN)$
 - 100 点
 - 4^2 倍という定数には注意

注意点

- 「プレイヤーと箱はそれぞれ壁でなく目標地点と異なるマスに配置しなければならない
ならず、プレイヤーと箱は異なるマスに配置しなければならない」
 - 箱が目標地点にいるときは答に足さない
 - 連結成分のサイズに目標地点は含めない
 - 目標地点を根にしておくとなんが楽

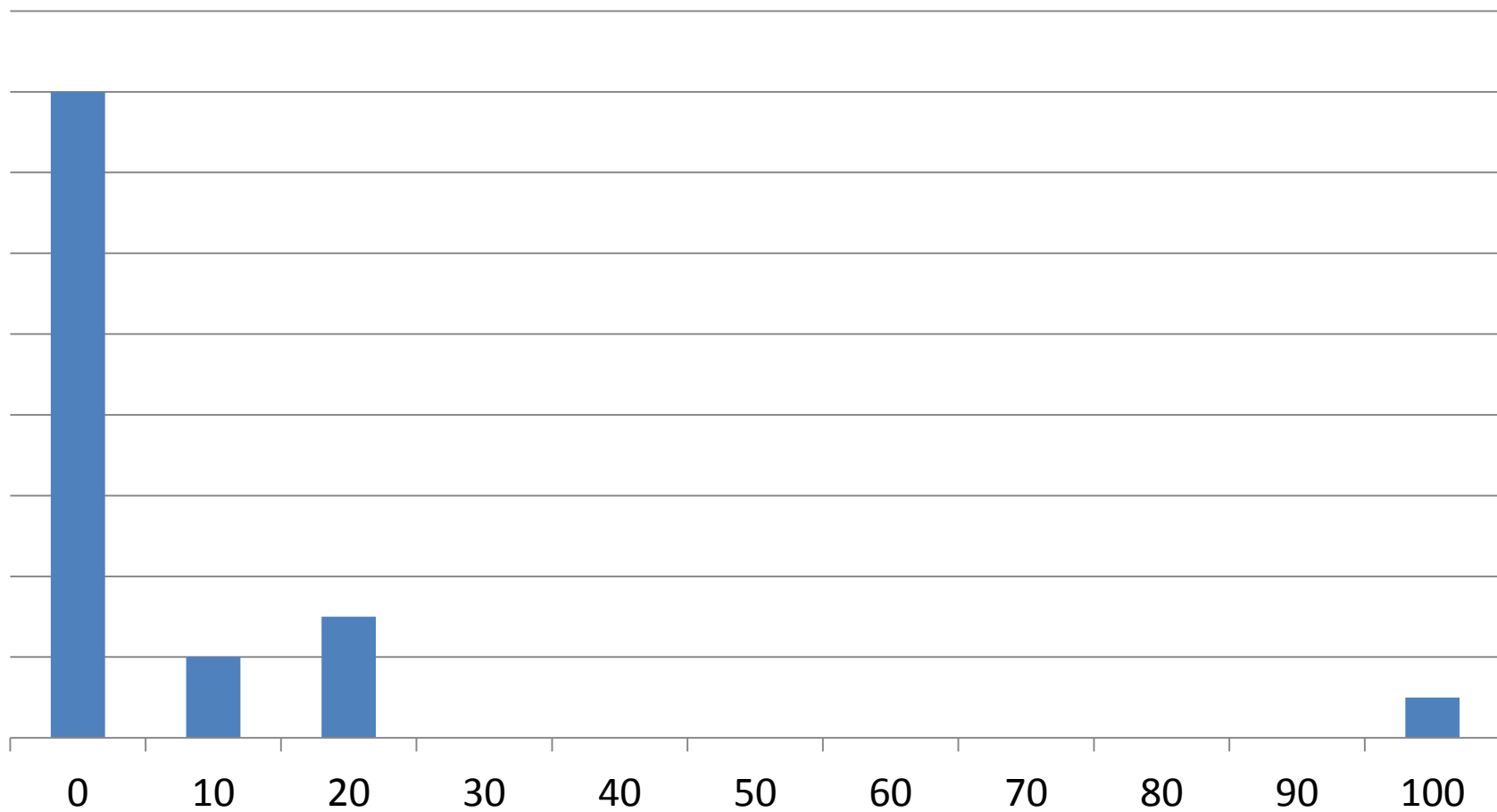
注意点

- MN 頂点のグラフに対して DFS を行う
 - スタックオーバーフロー
 - 今回のジャッジ環境では問題なかった
 - 手元で実行して怪しい挙動をしたときに焦らない
 - 自前のスタックを用意すれば再帰を避けられる
 - やや面倒

おまけ

- 倉庫番 (解く方) は一般には箱がたくさん
 - 盤面の状態が入力サイズの指数個
- **NP** に属するかどうかは未解決
 - 最短手数が入力サイズの指数になる場合あり
- **PSPACE** 完全であることが知られている
 - 入力サイズの多項式のメモリで解ける
 - そのような問題のなかで最も難しい
 - 参考：グラフの s - t 連結性判定, 対数メモリ

得点分布



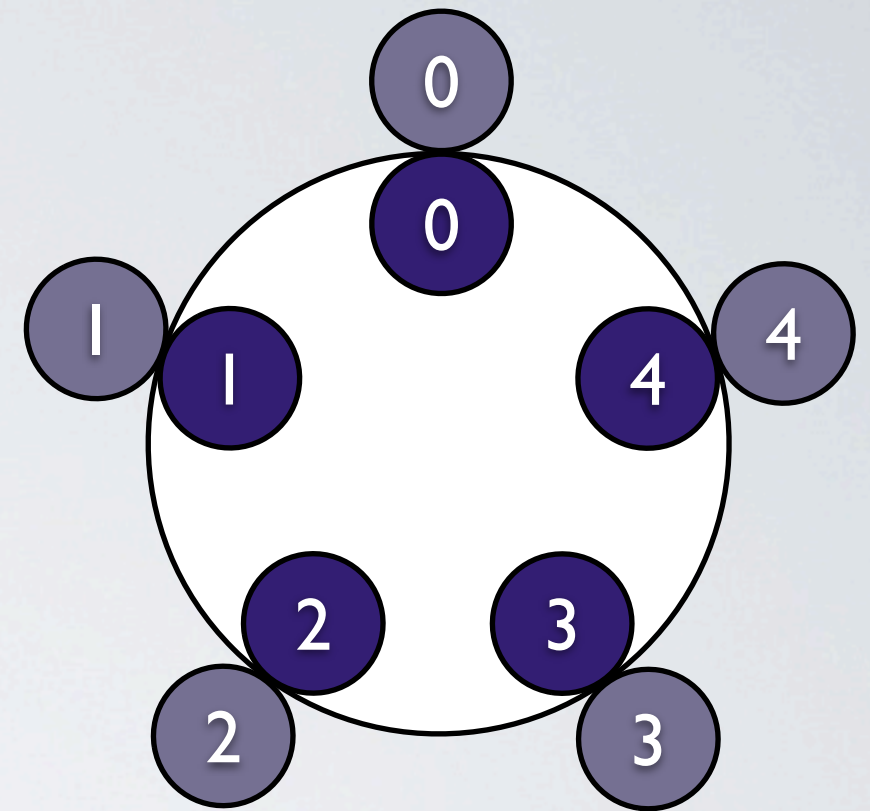


DAY4 中華料理(CHINESE)

山下洋史

おさらい

- こんな感じの回転台があります→
- 外側に委員, 内側に料理
- 回転台を回して料理を目の前に持ってくる食べられる
- K 理事長(委員0)が料理 k を食べた後, 回転台を回して全員が好きな料理を食べられるようにする
- すべての k について回転量の最小値を計算せよ.



2つの部分

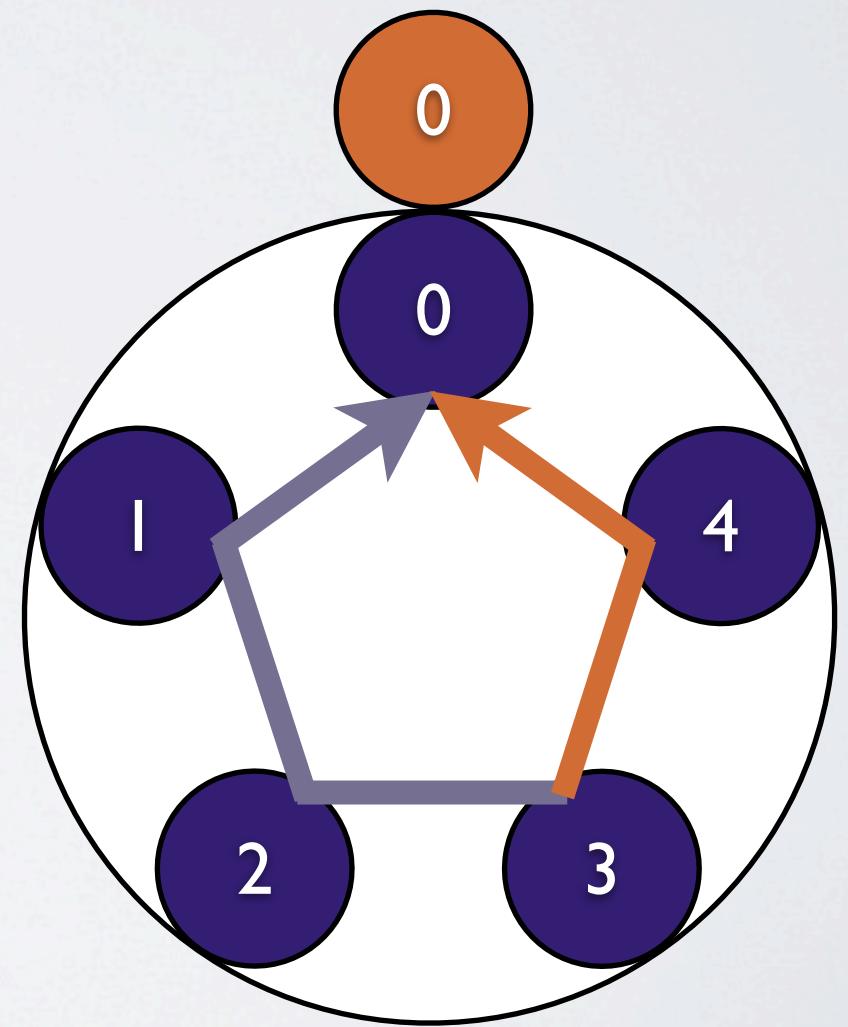
K理事長が食べるまでの回転量

+

K理事長が食べてからの回転量

カンタンな方

- K理事長が料理kを食べるのに必要な回転量
- $\min(k, N-k)$



2つの部分

K理事長が食べるまでの回転量 done

+

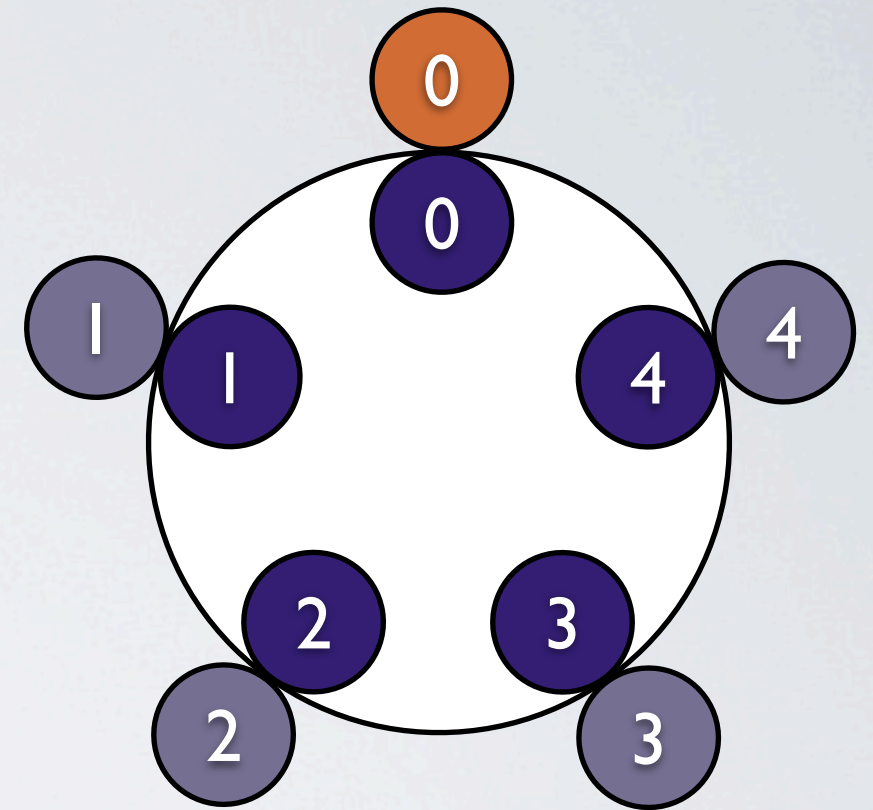
K理事長が食べてからの回転量 ← 本質

ナイーブさん

- 委員の食べる順番を全列挙
- すると最適な回し方はすぐ分かる
- $O(N!)$
- 10%ぐらい($N \leq 10$)

考えにくいので、

- K理事長(委員0)の目の前の料理に注目する
- 委員aが料理bを食べたとき、
K理事長の目の前に来る料理は料理(b-a)
- K理事長が食べた料理から出発して、このような料理たちをすべて1回以上通る



重要な考察

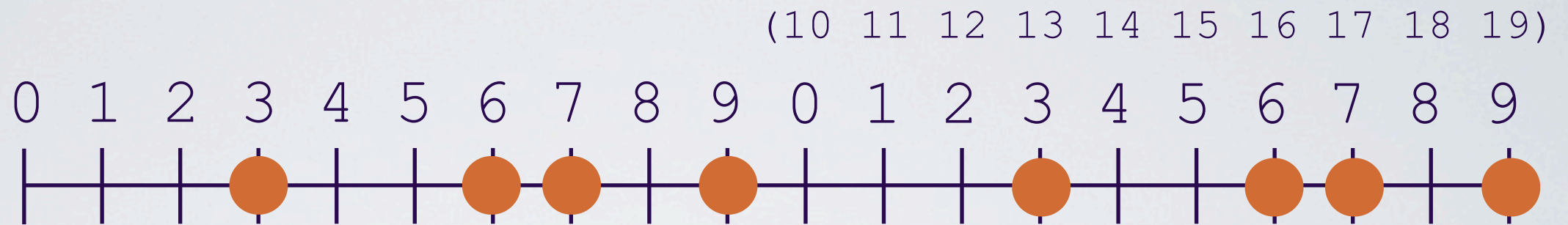


- 折り返しは高々1回だけでよい！
 - 右回り→左回り と 左回り→右回りに場合分けして小さい方をとる
- (折り返さない場合は, 0進んで折り返すと考えるとよい)

部分点解法(40%)

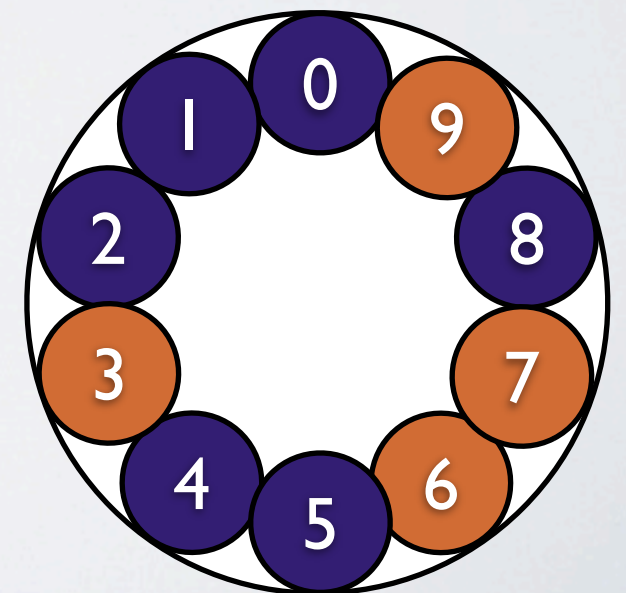
- 開始地点 k が N 個
- 折り返し地点の候補が N 個
- それぞれについて回転量を計算し, 最小値を求める
- $O(N^2)$

K理事長が料理0を食べるとき



折返しまで	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
折返し後	7	8	9	7	8	9	9	8	9	6	7	8	9	7	8	9	9	8	9	6
合計	7	9	11	10	12	14	15	15	17	15	17	19	21	20	22	24	25	25	27	25

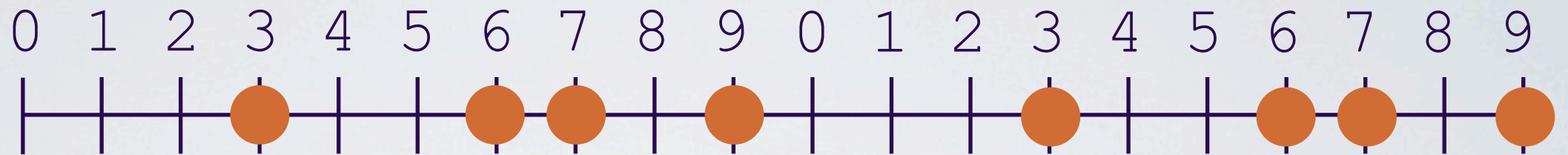
- 「折り返しまで」は1ずつ増えるだけ
- 「折り返し後」は,
チェックポイントの1つ前は(N-1),
そこから前に1ずつ減っていく.
- 「折り返し後」の前半と後半の値は同じ



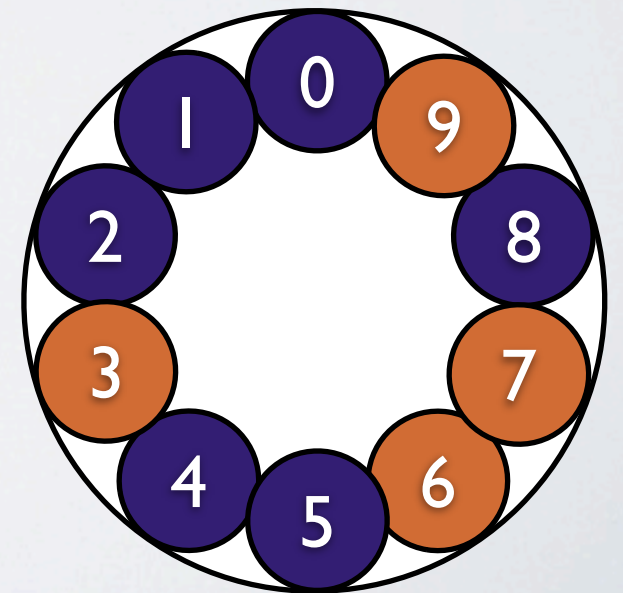
K理事長が料理Kを食べるとき

($k=5$ の場合)

(10 11 12 13 14 15 16 17 18 19)

[illegible]

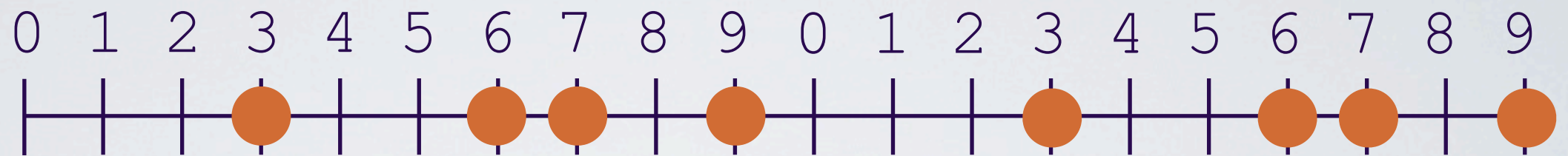
- $k=0$ の場合から k を引いた値になる



満点解法

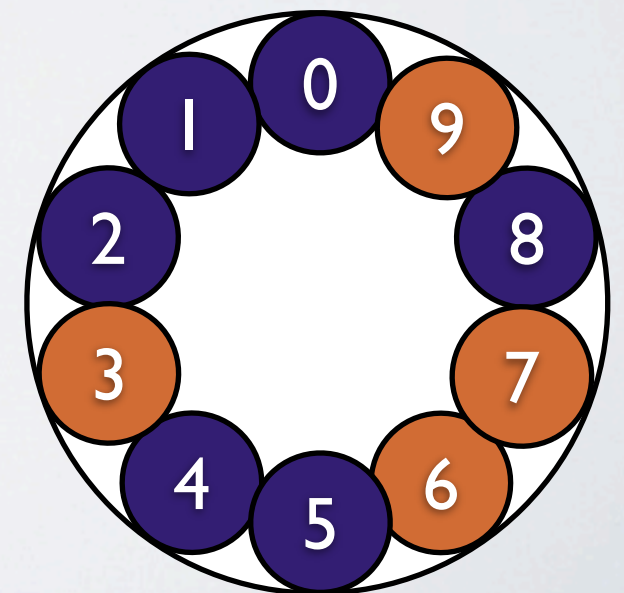
($k=0$ の場合の表)

(10 11 12 13 14 15 16 17 18 19)



折返しまで	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
折返し後	7	8	9	7	8	9	9	8	9	6	7	8	9	7	8	9	9	8	9	6
合計	7	9	11	10	12	14	15	15	17	15	17	19	21	20	22	24	25	25	27	25

- k 番目以降の最小値- k
- 大きい方から順にみていくと $O(N)$
- 他の処理もすべて $O(N)$



注意

↓足し忘れないように↓

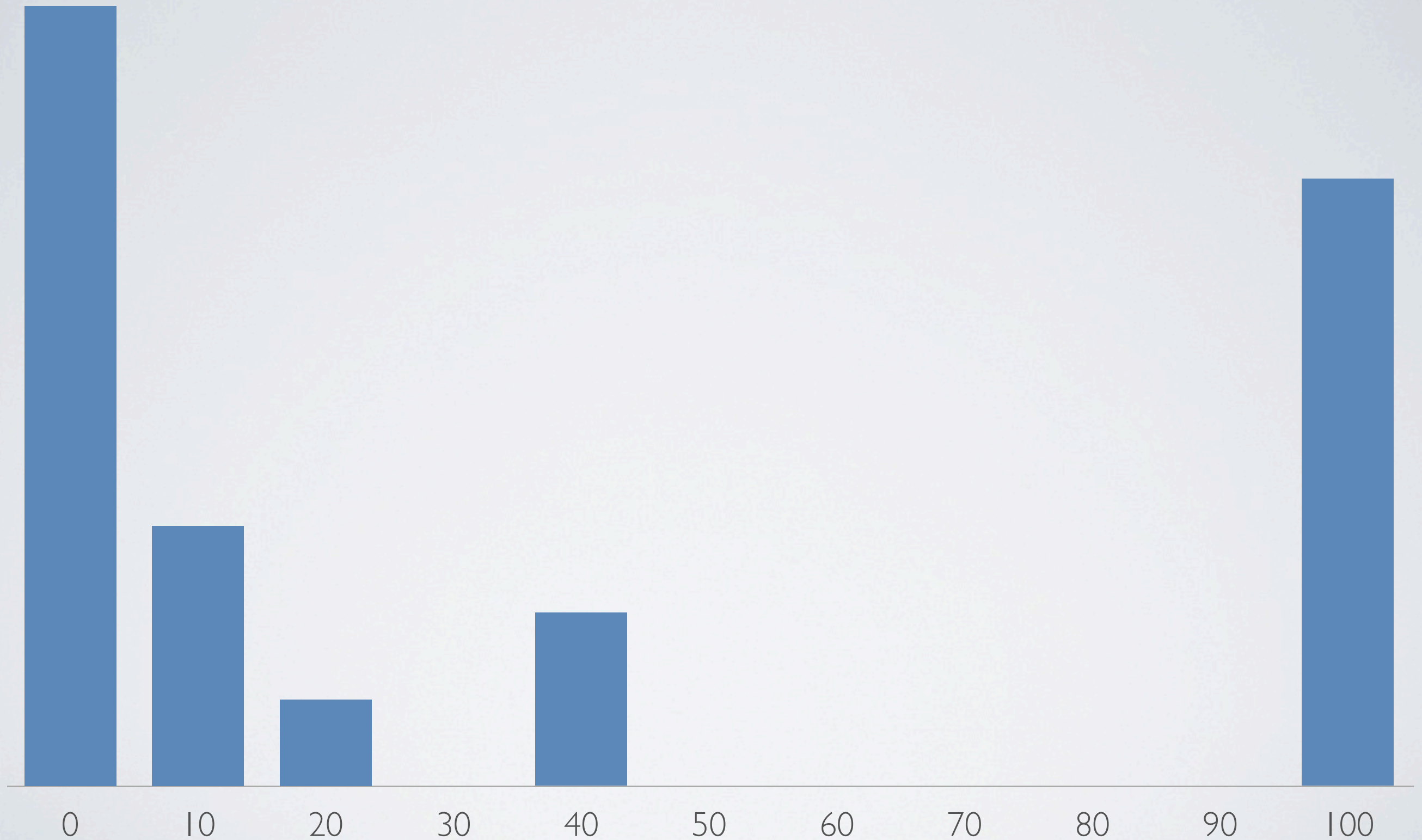
K理事長が食べるまでの回転量

+

K理事長が食べてからの回転量

と、いうわけで

得点分布

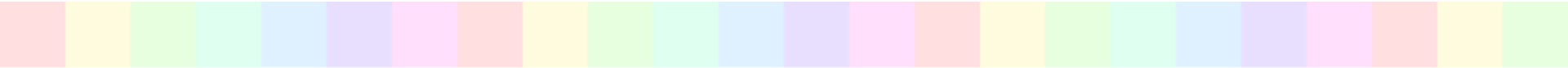


まとめ

- 頑張って考察しましょう
- 問題を単純化できるとうれしい
- 円状に並んでいる問題は、何周か分を一直線にして考えると考えやすくなる(こともある)
- (0 or 1-originは適当に考えやすいように変換する)



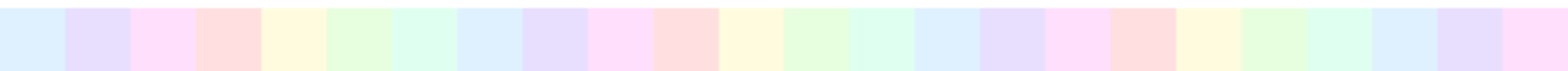
ありがとうございました



コピー & ペースト (Copy and Paste)

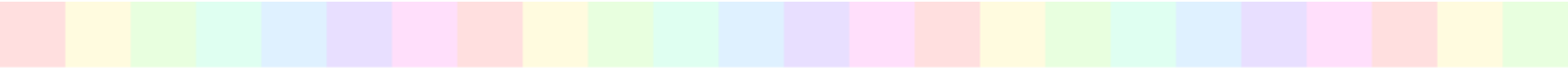
JOI 春合宿 2012 Day 4

解説： 保坂 和宏

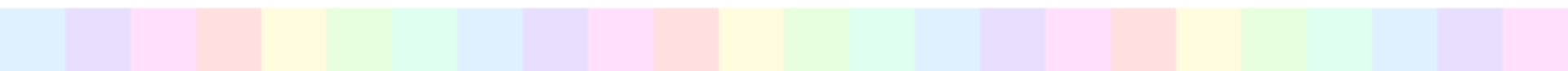


問題概要

- 文字列がある
- 「位置 A_i から位置 B_i までの $B_i - A_i$ 文字をコピーして位置 C_i に貼りつける」というクエリが N 個くる
- 長さが M を超えた部分は右から切られる
- 最終結果を出力せよ



制約

- $M \leq 1,000,000$
 - $N \leq 1,000,000$
 - 時間制限 17 秒
 - メモリ制限 512 MB
- 

単純な解法

for $i = 0$ **to** $N - 1$:

$S := S[0 .. C_i] S[A_i .. B_i] S[C_i .. |S|].$

$S := S[0 .. \min(|S|, M)].$

※ $[]$ は部分文字列

- $O(MN)$
- どう実装するか？

単純な解法

- `std::string` を使う
 - 10 点 ?
- 2 つの配列をとってコピーしあう
- 1 つの配列にとって必要な分だけ移動
 - 0 点 ~ 30 点 ?
 - 必要な分だけ移動 (M 文字を超える分を作らない)
 - `memcpy`, `memmove` 関数を使うと高速

単純な解法

- どんな解法が速いか？
 - メモリの書き換え回数が少ない
 - ループをしない
 - メモリの書き換えは char であるのに int の操作をたくさんするのは実は大変
- いずれにせよ、高い部分点を狙うなら手元でちゃんとテスト

平衡二分木

- 各文字をノードとした平衡二分木を持つことができていたら, split と merge によって部分文字列を切り出したり, 貼り付けたりを効率よく行える
- ではコピーはどうすれば?
 - ノードを全部コピーするとクエリごと $O(M)$ になってしまう

平衡二分木

- コピーして新しいノードを作らずに同じノードを参照すればよい
- 木の複数箇所に同じノードへの参照が生じるので、そのうち 1 つを書き換える、などができなくなる
 - 永続データ構造にすればよい

平衡二分木

- 平衡二分木にはいろいろある
- コピーしたときでも平衡されないと困る
- ダメな例
 - Treap : 同じ優先度のノードが増えすぎ
 - Splay tree : ポテンシャルが増えすぎ

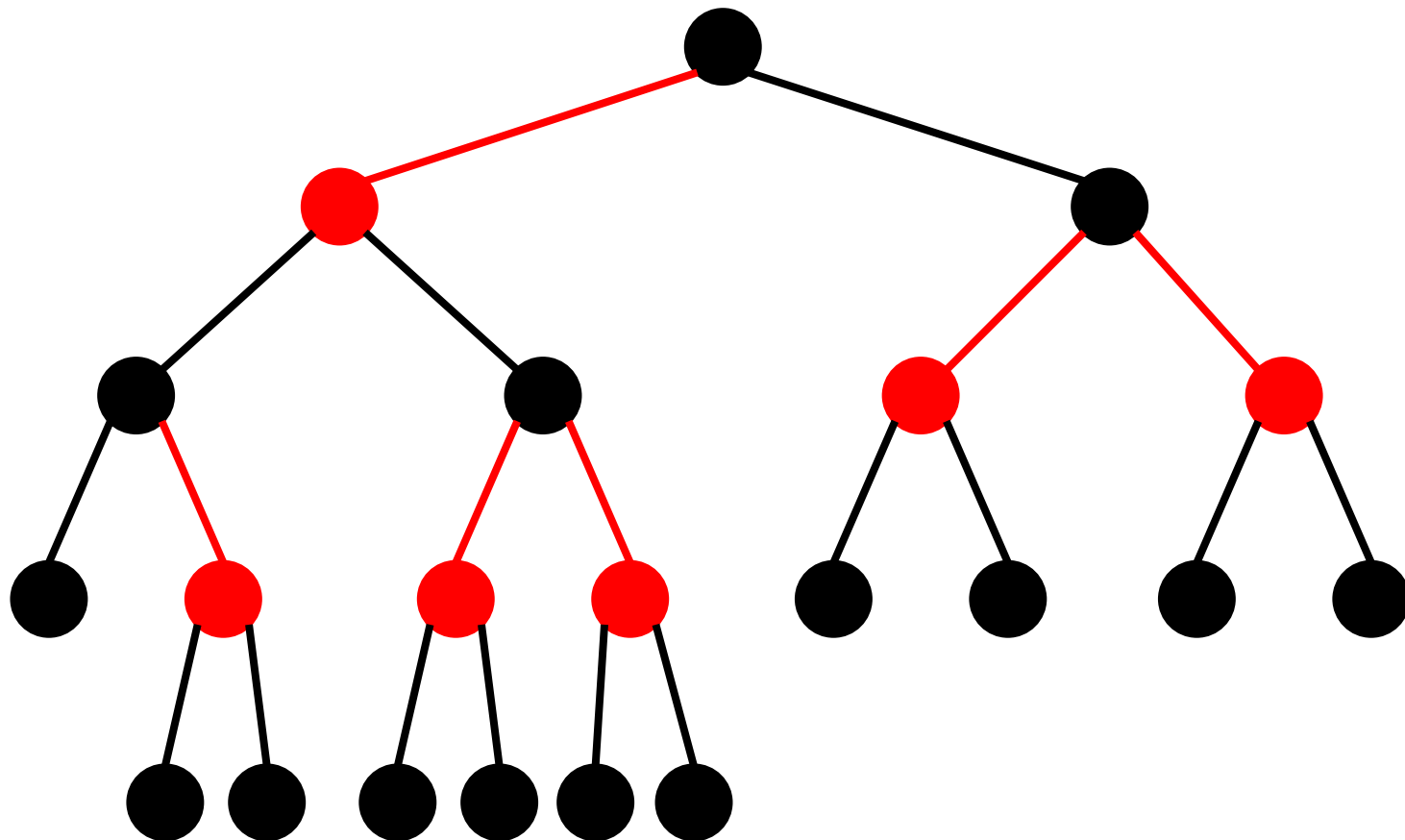
赤黒木

- 赤黒木
 - 操作ごとの最悪計算量が $O(\log (\text{ノード数}))$
 - merge/split ベースにする
 - 葉ノードにのみ値 (文字) を持たせると楽
 - ノード数が 2 倍 ($\rightarrow$ 深さは +1) になるだけ
 - 永続にしやすいようにうまく実装する

赤黒木

- 「**赤**黒木」
- 条件
 - 各ノードには**赤**か**黒**の色がついている
 - 根は**黒**, 葉は**黒**
 - **赤**いノードは隣接しない
 - 根から葉までのパス上の**黒**いノード数は一定

赤黒木

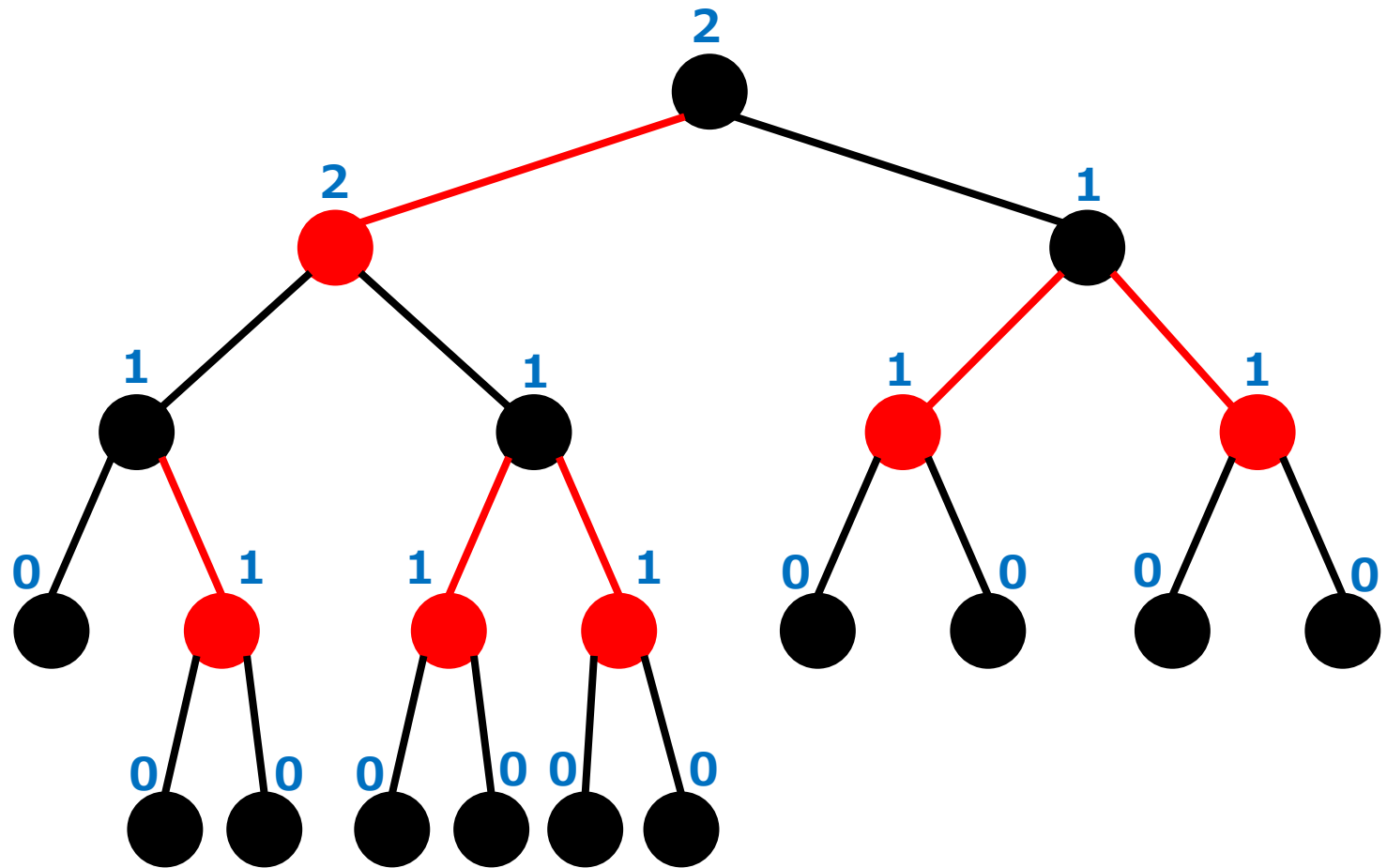


赤黒木

- 平衡する理由
 - 根から葉までのパスの長さは最大のものも最小の 2 倍以下
- ノードに「ランク」を持たせて実現
$$\text{rank}[u] = (\text{u から葉までの黒いノードの数})$$

(ただし u は含まない)

赤黒木



赤黒木：ノード

- ノードが持つ情報
 - l : 左の子へのポインタ
 - r : 右の子へのポインタ
 - $color$: 色
 - $rank$: ランク
 - $size$: 部分木のサイズ
 - val : 値 (葉のとき)

赤黒木：ノード

- ノードが持つ情報は、一度決めたら変更してはいけない
 - 子へのポインタと色を変更したくなる
 - ランクや部分木のサイズは、子を 2 つ決めたときに決まる
 - 変更したいときは新しいノードを作る
 - *node*(左の子, 右の子, 色) のような関数を用意
 - この呼び出し回数分メモリを消費する

赤黒木：merge

- 方針：ランクが等しい部分木を繋げる
- 「根が**黒**」という条件を一旦無視
- ランクが異なったら，部分木と再帰的に merge
- ランクが等しかったら，**赤**いノードを新たに作って繋げたものを返す
- **赤**いノードが隣接したら修正

赤黒木 : merge

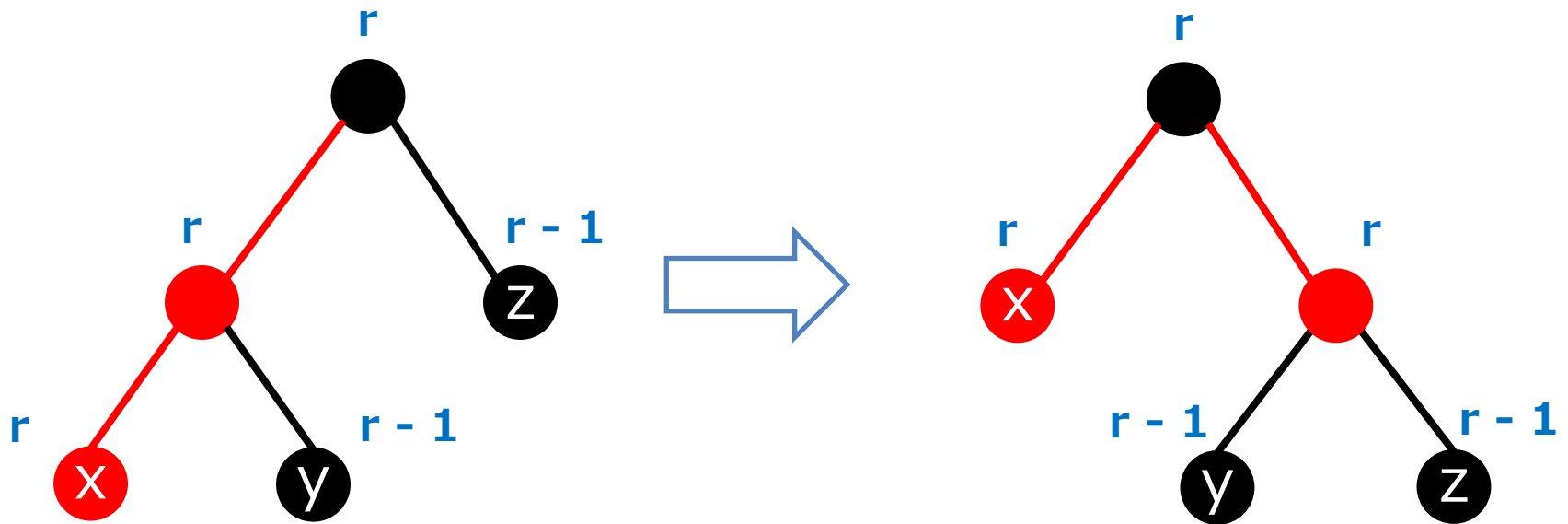
```
function mergeSub(a, b)
  if (a.rank < b.rank):
    c := mergeSub(a, b.l).
    if (b.color = 黒 and c.color = 赤 and c.l.color = 赤):
      if (b.r.color = 黒):
        return node(c.l, node(c.r, b.r, 赤), 黒).
      else:
        return node(node(c.l, c.r, 黒), node(b.r.l, b.r.r, 黒), 赤).
    else:
      return node(c, b.r, b.color).
  else if (a.rank > b.rank):
    (上と同様)
  else:
    return node(a, b, 赤).
```

赤黒木 : merge

```
function merge(a, b)  
  if (a is null):  
    return b.  
  if (b is null):  
    return a.  
  c := mergeSub(a, b).  
  if (c.color = 赤):  
    return node(c.l, c.r, 黒).  
  return c.
```

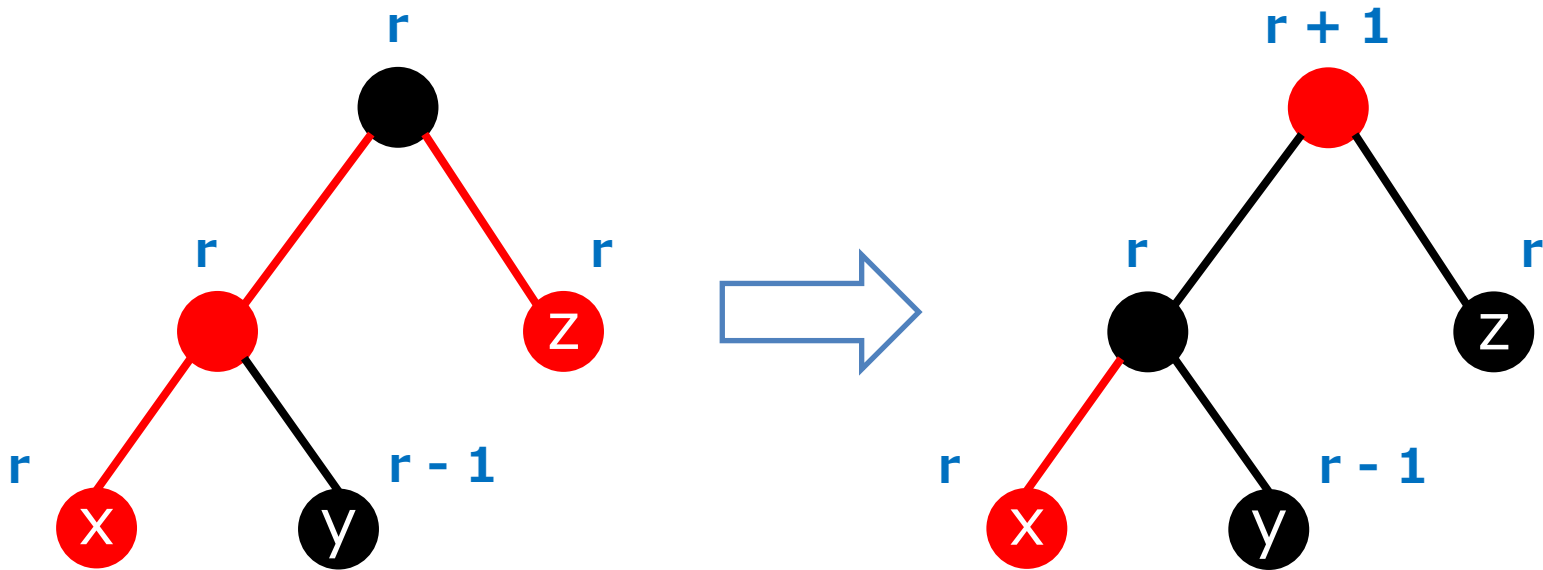

赤黒木 : merge

- 黒 - 赤 - 赤 が生じそうになったら (1)



赤黒木 : merge

- 黒 - 赤 - 赤 が生じそうになったら (2)
 - これらのパターンは上の方に伝搬しうる



赤黒木 : merge

- 計算量 : $O(|a.rank - b.rank|)$

赤黒木：split

- 左側が葉を k 個含むように 2 つに分ける
- 再帰的に split する途中で、木 2 つを繋げる必要がある
 - 普通に繋げると制約が満たされなくなる
 - merge を呼ぶ
 - 何回も呼ぶが合計では $O(\log (\text{ノード数}))$

赤黒木 : split

```
function split(a, k)  
  if (k = 0):  
    return <null, a>.  
  if (k = a.size):  
    return <a, null>.  
  if (k < a.l.size):  
    <L, R> := split(a.l, k).  
    return <L, merge(R, a.r)>.  
  else if (k > a.l.size):  
    <L, R> := split(a.r, k - a.l.size).  
    return <merge(a.l, L), R>.  
  else:  
    return <a.l, a.r>.
```

初期構築

- まず最初の文字列を赤黒木に
- 1 文字ずつ葉ノードを作って merge
 - $O(M \log M)$

初期構築

- まず最初の文字列を赤黒木に
- 同じサイズのを merge
 - サイズ 1 どうしを $M/2$ 回 merge
 - サイズ 2 どうしを $M/4$ 回 merge
 - サイズ 4 どうしを $M/8$ 回 merge
 -
 - $O(M)$

コピー & ペースト

- merge と split があれば簡単
 - $[0 \dots A_i] [A_i \dots B_i] [B_i \dots L_i]$ に split
 - $[0 \dots C_i] [C_i \dots L_i]$ に split
 - $[0 \dots C_i] [A_i \dots B_i] [C_i \dots L_i]$ を merge
 - できた文字列を $[0 \dots M] [M \dots L_{i+1}]$ に split
 - クエリごとに $O(\log M)$

計算量

- $O(M + N \log M)$
- これは *node* を呼ぶ回数でもある
 - 毎回メモリを使うとメモリが足りない

メモリ節約法 (1)

- 要らなくなったノードのメモリを解放
- ノードの情報に, 「それを子にもつノードの個数」も持たせる
 - 子へのリンクが張り替わるたびに更新
 - 0 になったらノードを消す
- メリット: 省メモリ (64 MB でも足りる)
- デメリット: やや遅い (1.5 倍くらい?)

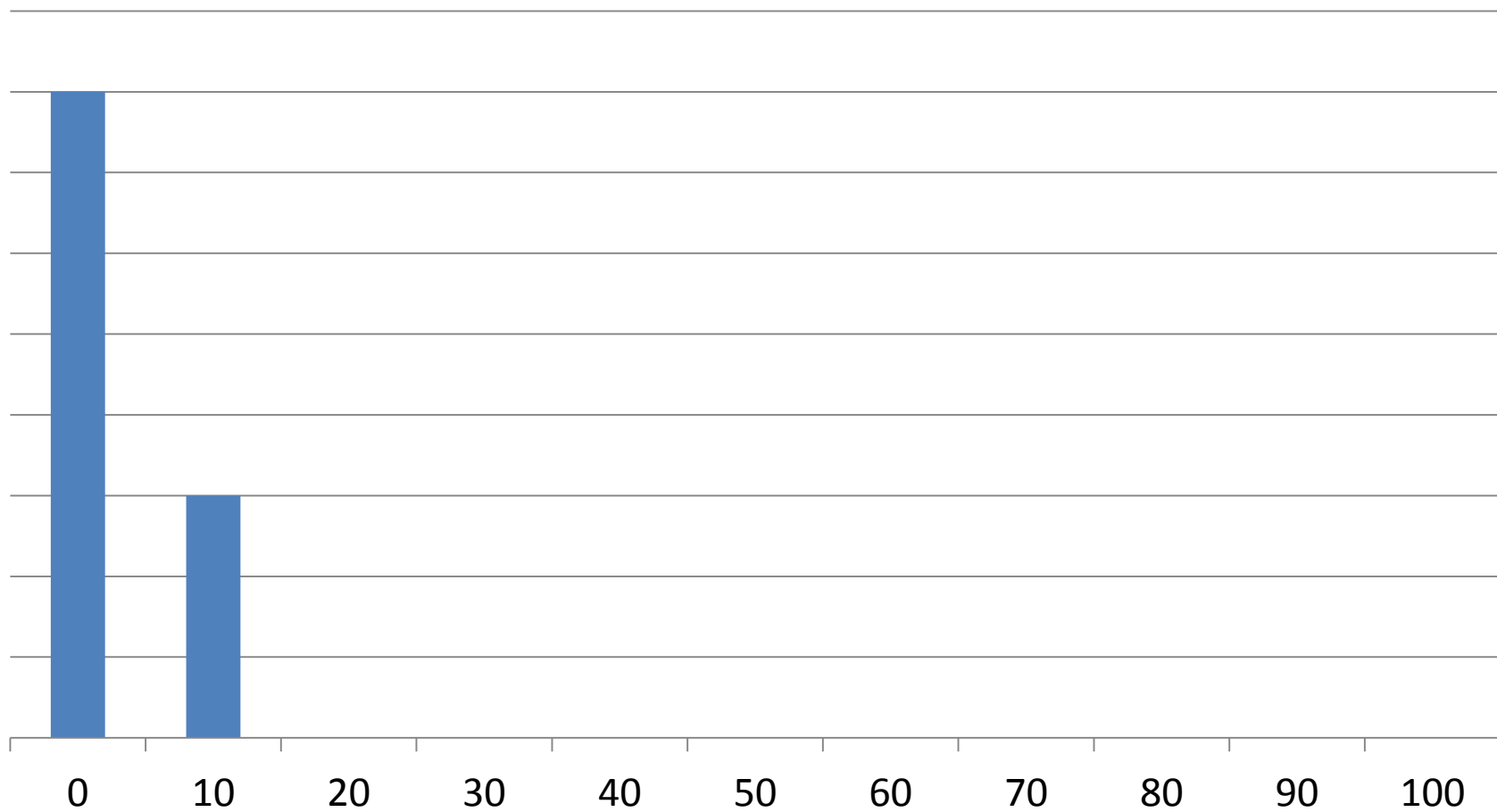
メモリ節約法 (2)

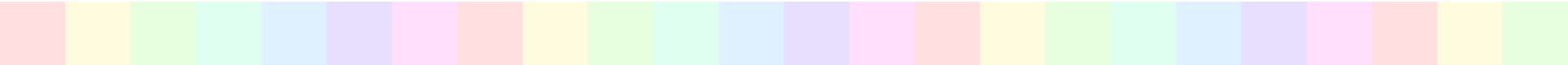
- メモリが限界に近づいたら木を作り直し
 - 文字列を一度出力させ, メモリを再利用して木を再構築
- 再構築は $O((\text{メモリ}) / \log M)$ 回ごと
 - 全体 : $O(M + N \log N + M N \log M / (\text{メモリ}))$
 - 最大データでも再構築は数回

別解？

- Randomized Binary Search Tree
 - 実装はやや楽？
 - 計算量は？
 - コピーにより確率が独立でなくなるので解析が難しい……
 - やってみると結構平衡されていて十分高速

得点分布

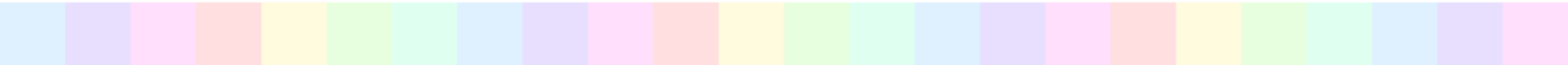




招待 (Invitation)

JOI 春合宿 2012 Day 4

解説： 保坂 和宏



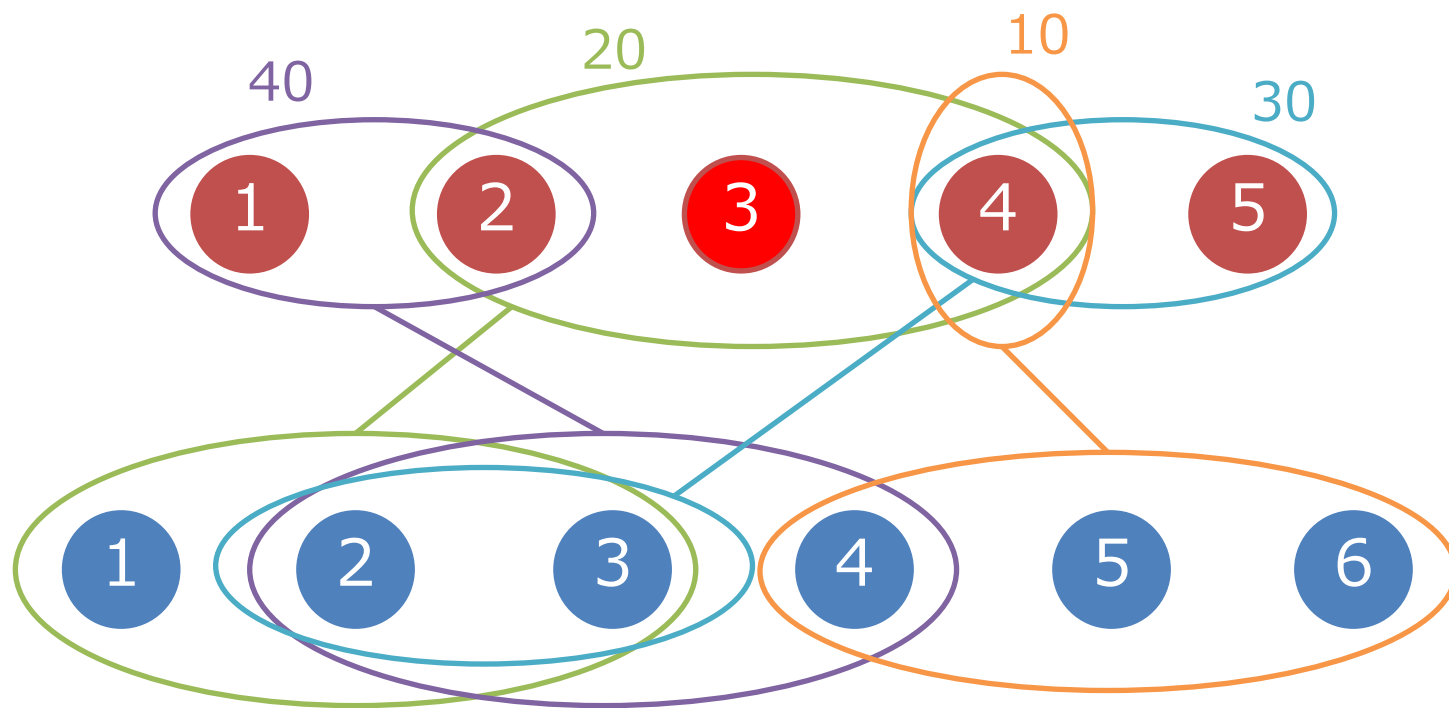
問題概要

- A 匹の犬と B 匹の猫
- N 個の仲良しグループ
 - 犬 P_i から犬 Q_i まで, 猫 R_i から 猫 S_i まで
 - 仲良し度 T_i
- みんなをパーティーに招待する
 - 問題文に招待のアルゴリズムが書かれている

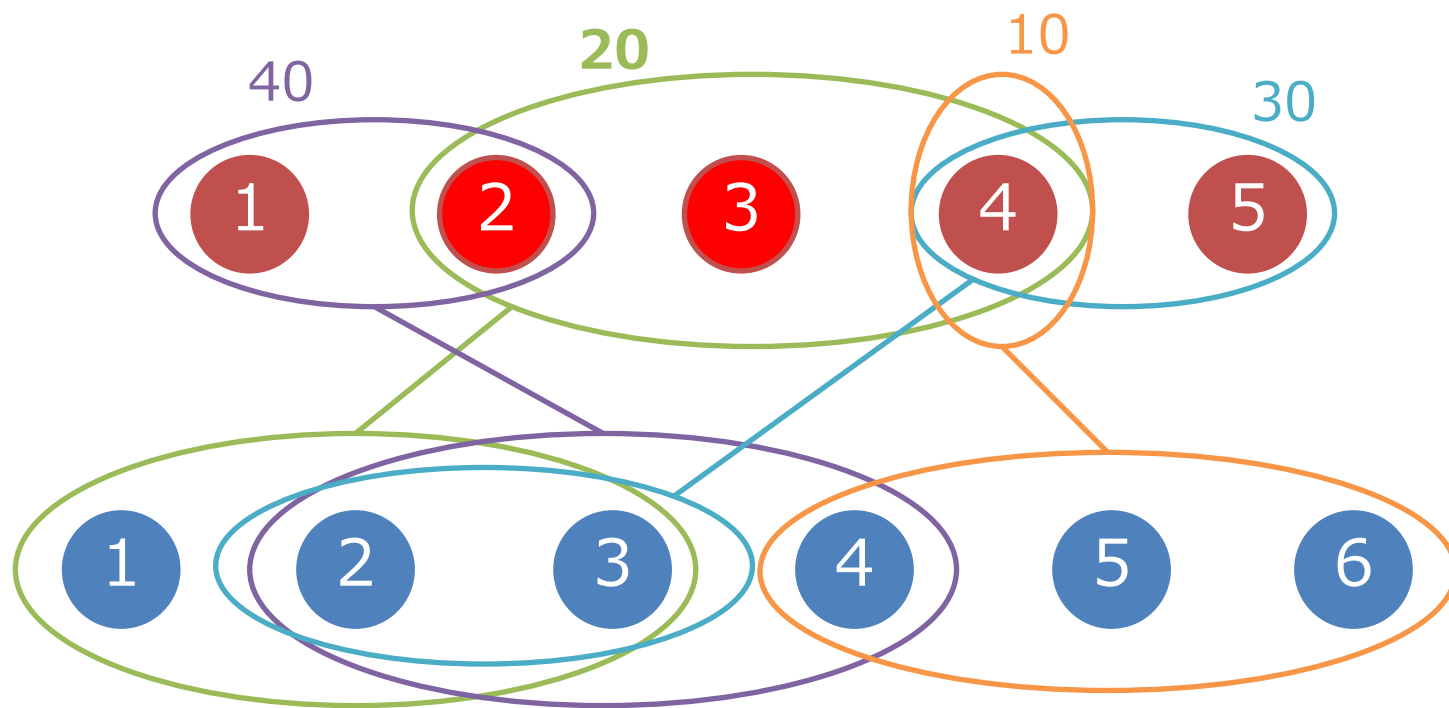
問題概要

1. 犬 C を招待
 2. 幸せ値最大の動物を招待することを繰り返す
 - 幸せ値：既に招待した動物と一緒に所属する仲良しグループのうちの仲良し度の最大値
 - 同じときは犬，番号小さい方を優先
 - 幸せ値 0 の動物しかいないときは失敗
- 幸せ値の和を答える

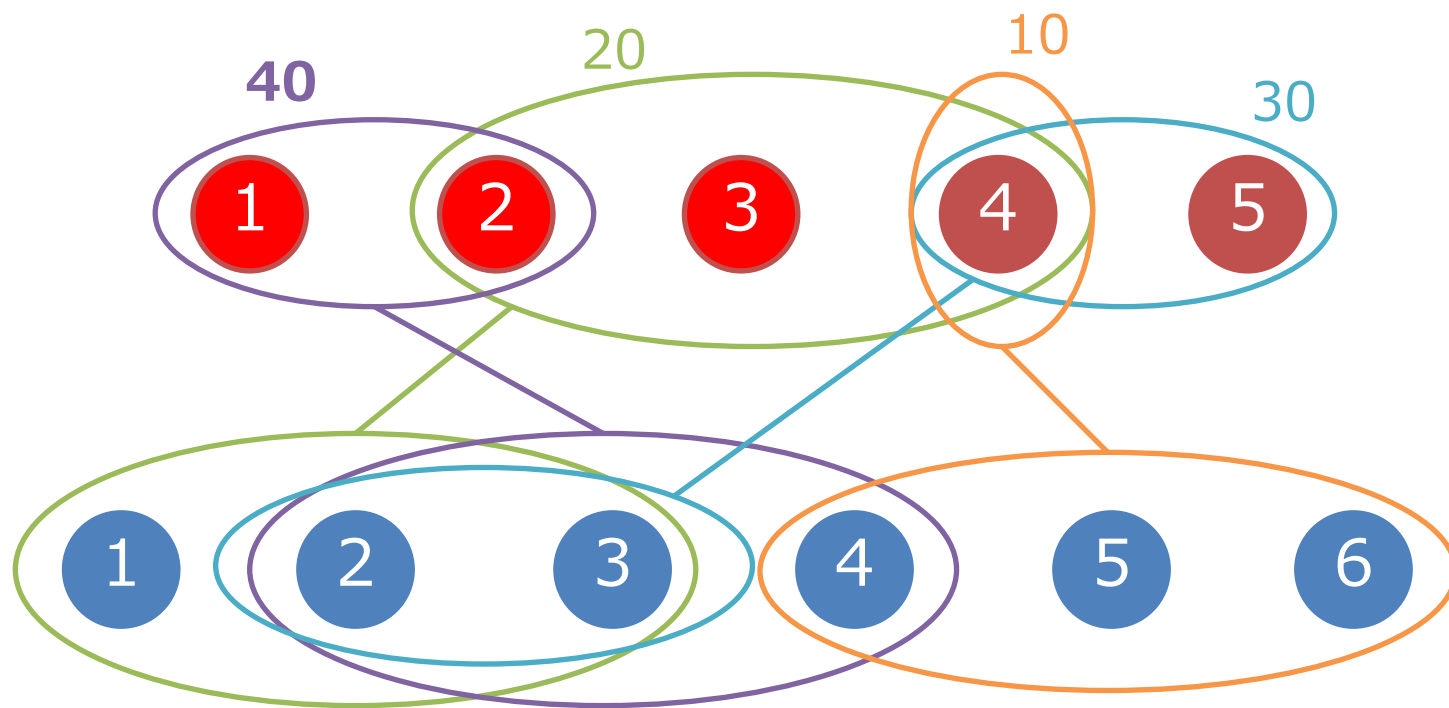
問題概要



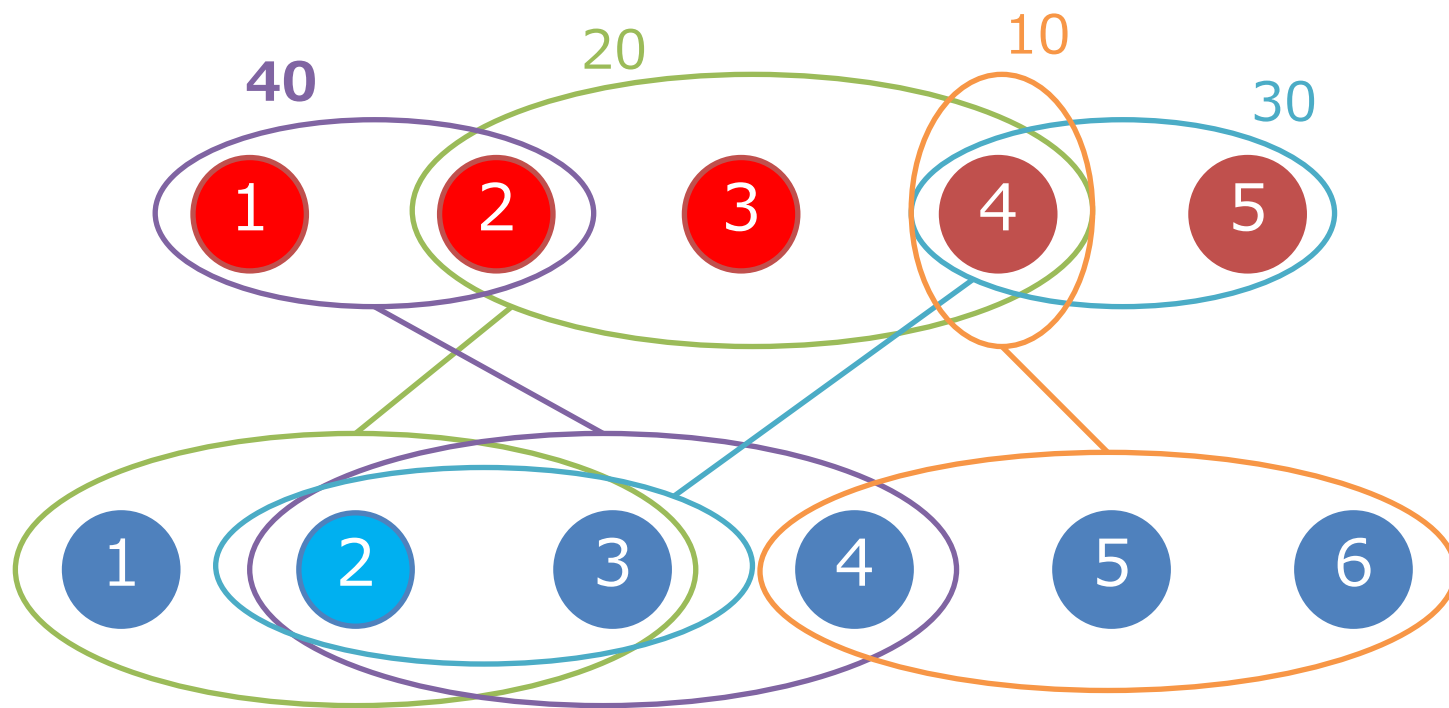
問題概要



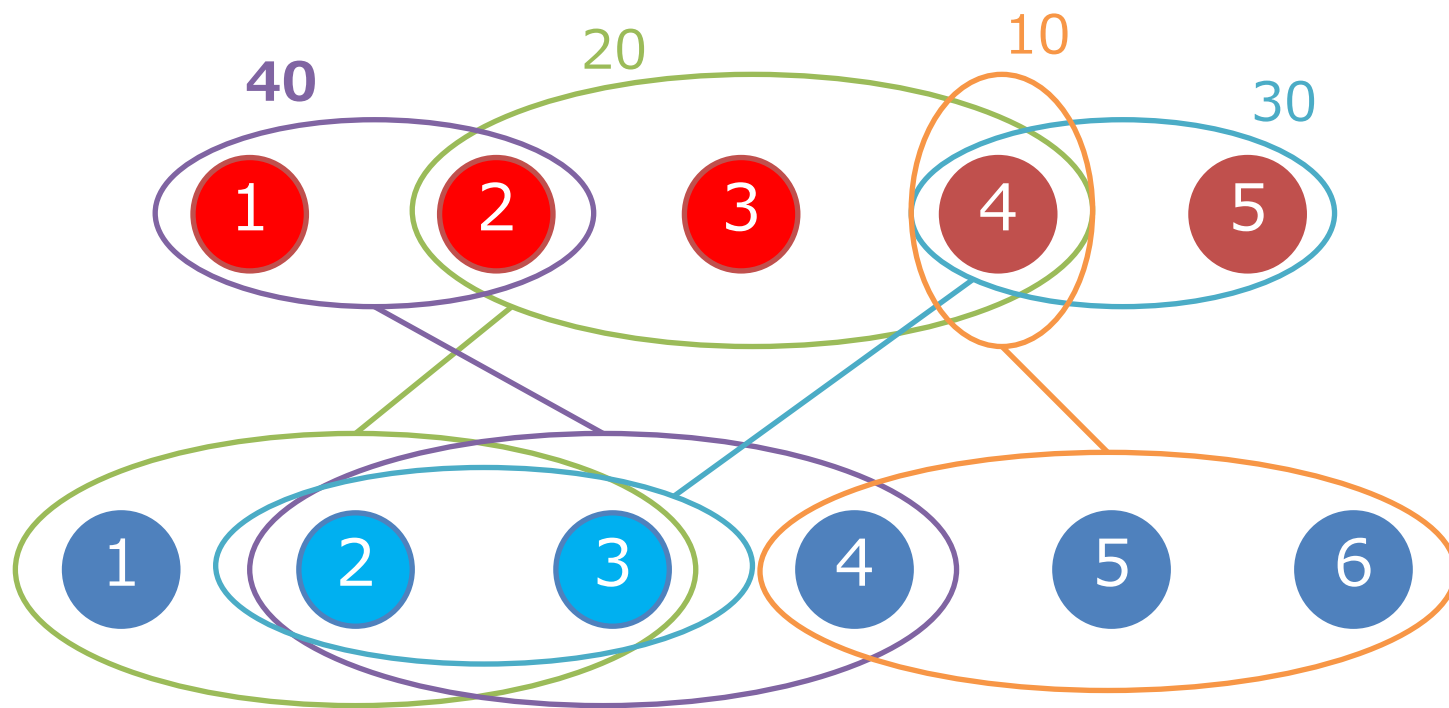
問題概要



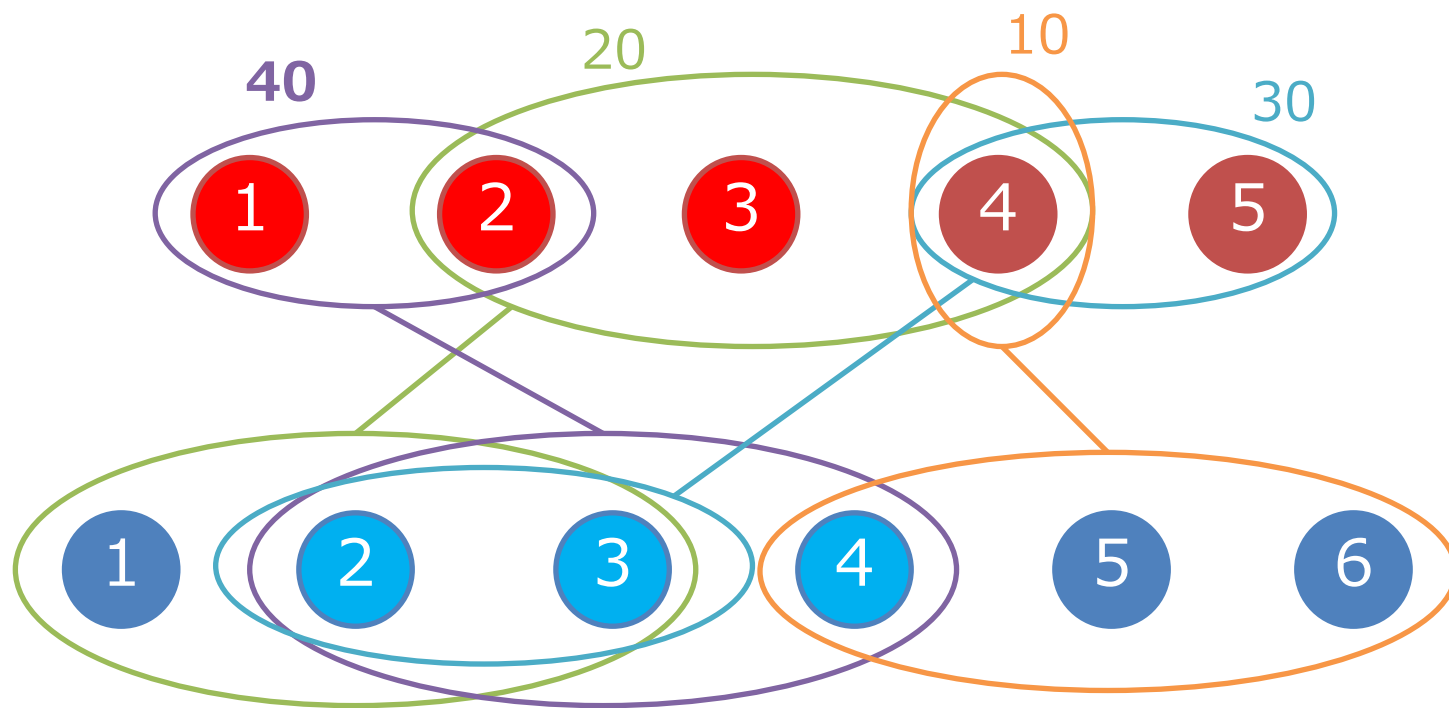
問題概要



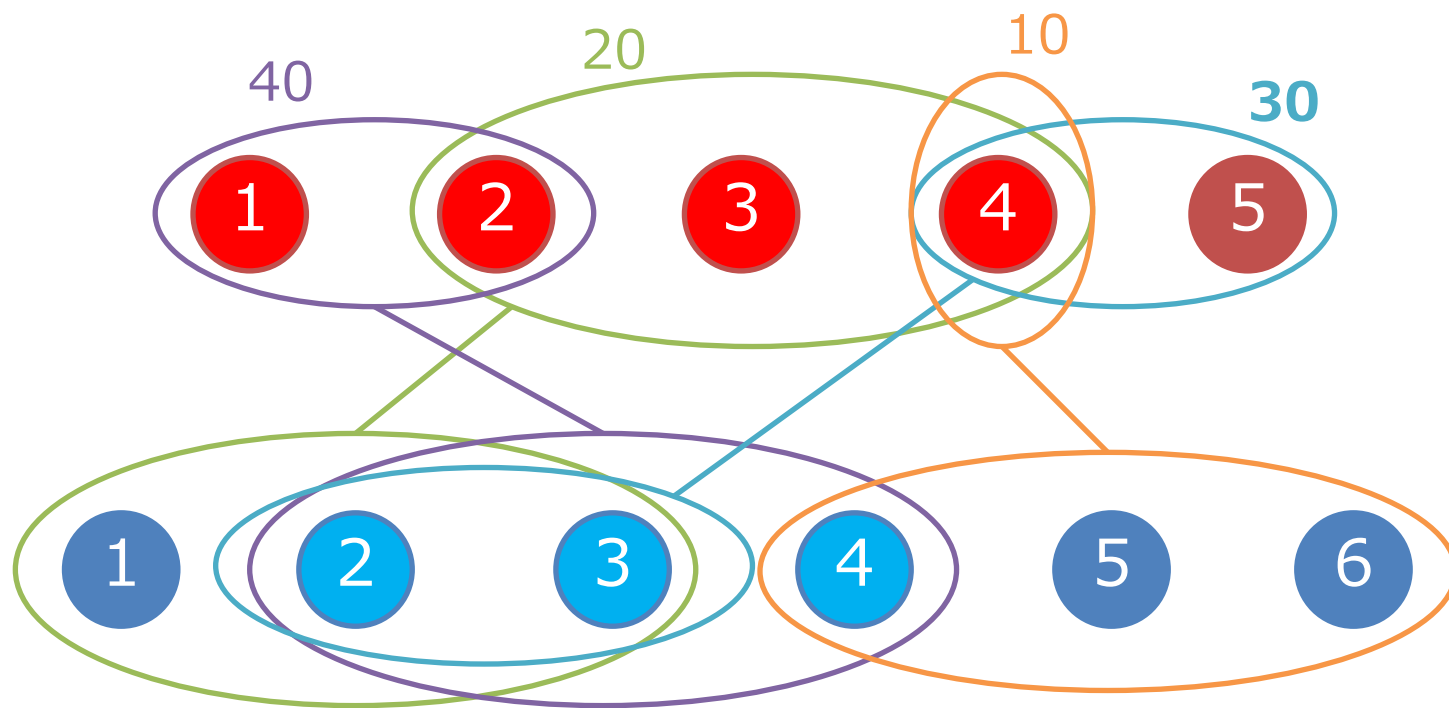
問題概要



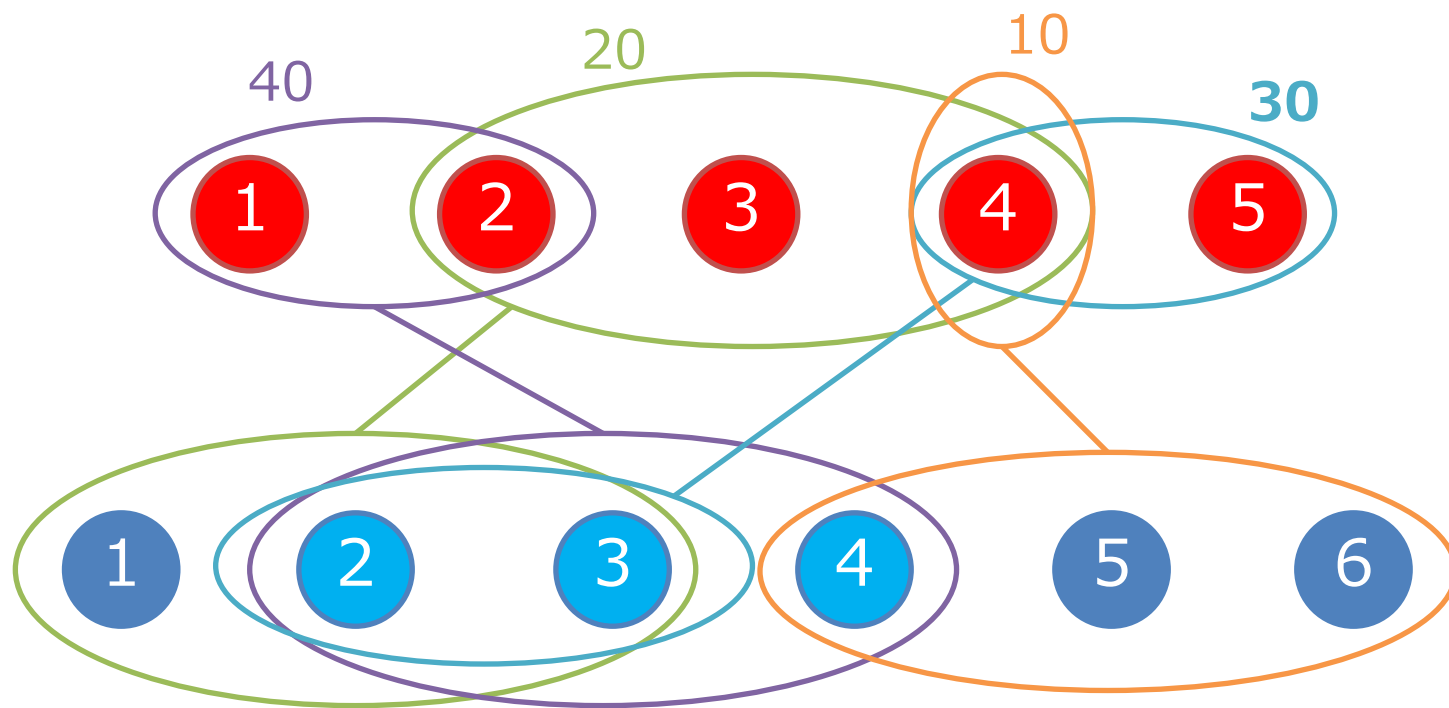
問題概要



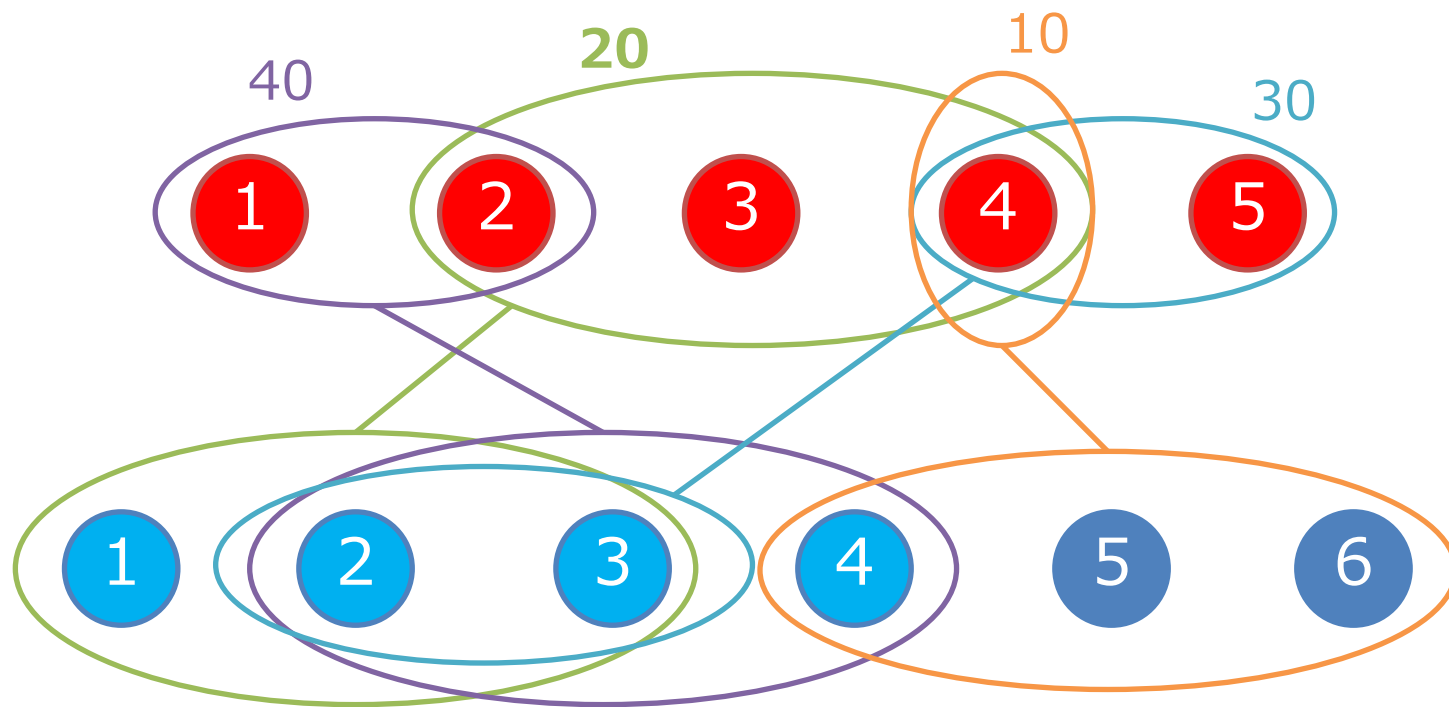
問題概要



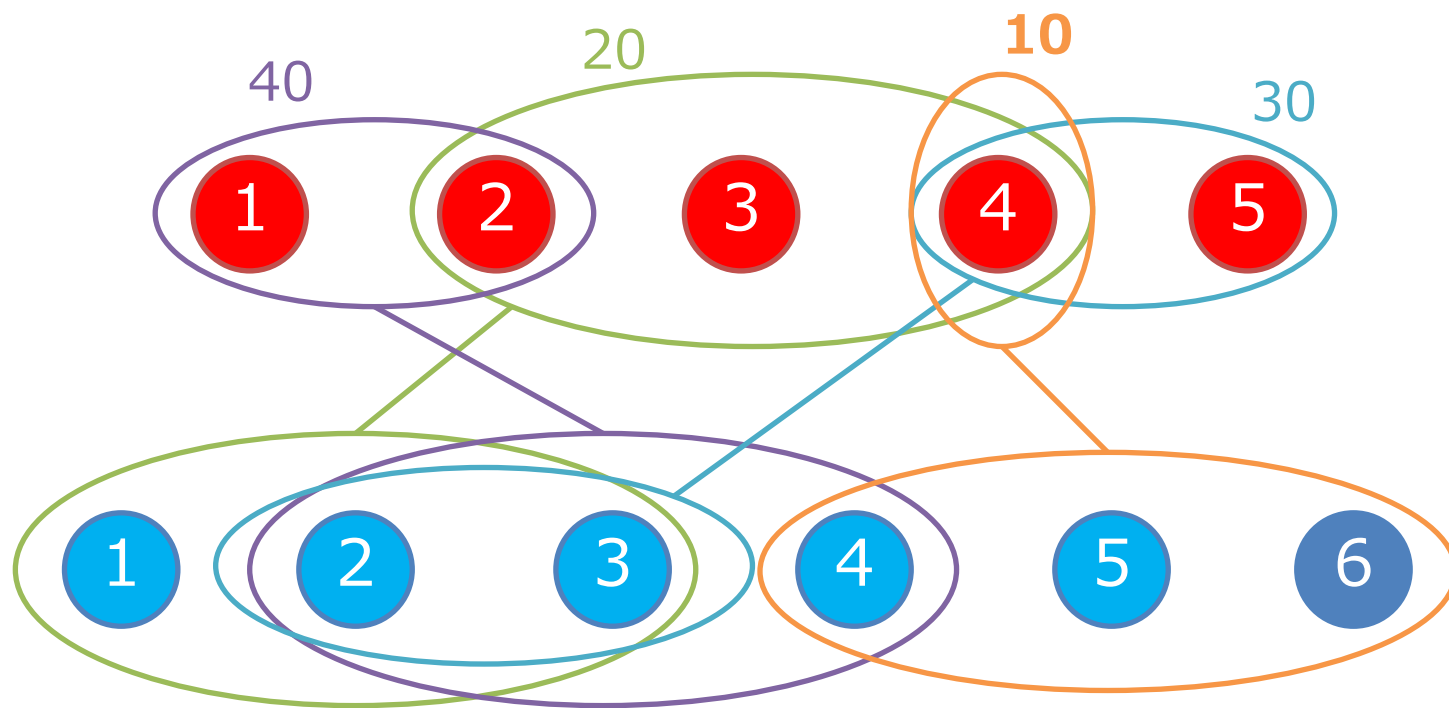
問題概要



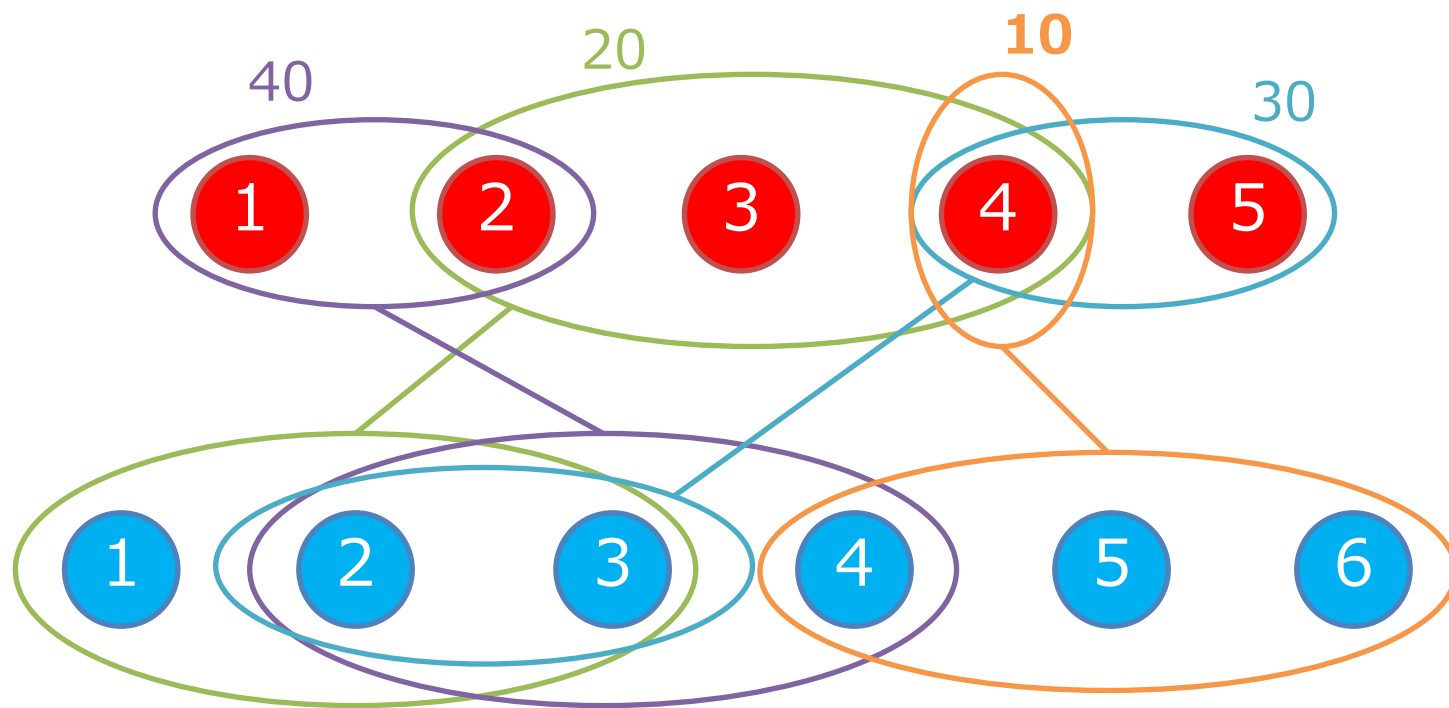
問題概要



問題概要



問題概要



制約

- $N \leq 100,000$
- $A, B \leq 1,000,000,000$
- 50 点 $\cdots N \leq 2,000$
- 30 点 $\cdots N \leq 2,000, A, B \leq 1,000$

シミュレーション解法

- 問題文に書かれた通りにシミュレーションする
 - 実装の指針はいろいろ
 - 繰り返しは $A + B$ 回, $O(A + B)$ 匹のうち最大を求める
- $O(N (A + B)^3)$ or $O(N (A + B)^2)$
 - 0 点 ~ 10 点 ?

シミュレーション解法

- シミュレーションを効率よくしたい
- 誘われていない動物の幸せ値は減らない
- 幸せ値が更新されるのは高々 N 回

シミュレーション解法

- $A + B$ 回の繰り返しに対して,
 - 幸せ値最大を選ぶ ($O(A + B)$)
 - 各仲良しグループに対して ($O(N)$),
 - 新しい仲良しグループなら, 幸せ度を更新する ($O(A + B)$)
- $O(N(A + B) + (A + B)^2)$
 - 30 点

シミュレーション解法

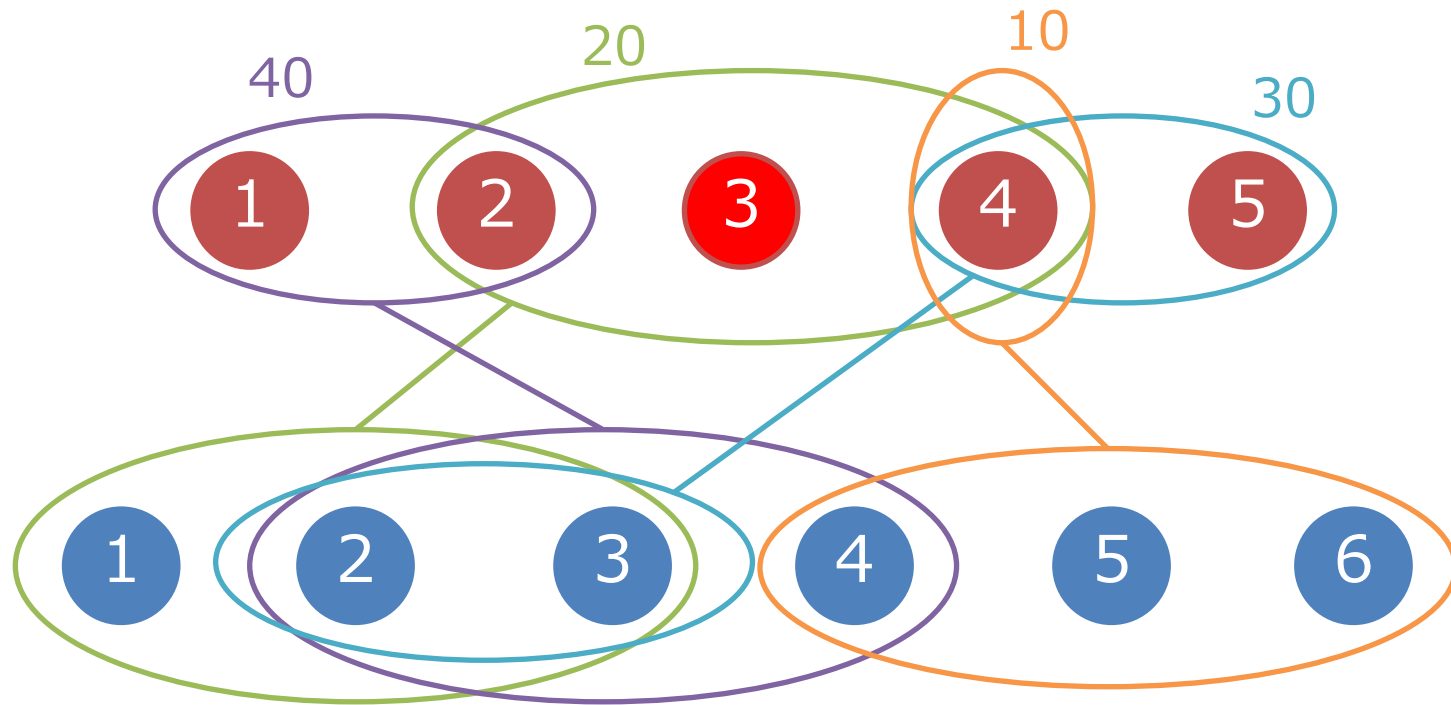
- 制約は A, B の値が大きい
- 仲良しグループへの入り方が同じな犬 or 猫は番号が連続している
- 座標圧縮
 - 結構大変？
- $O(N^2)$
 - 50 点？

シミュレーション解法

- さらに列に対するデータ構造を使ってがんばる
 - Segment tree ?
 - 超大変 ?
- $O(N \log N)$
 - 100 点 ?

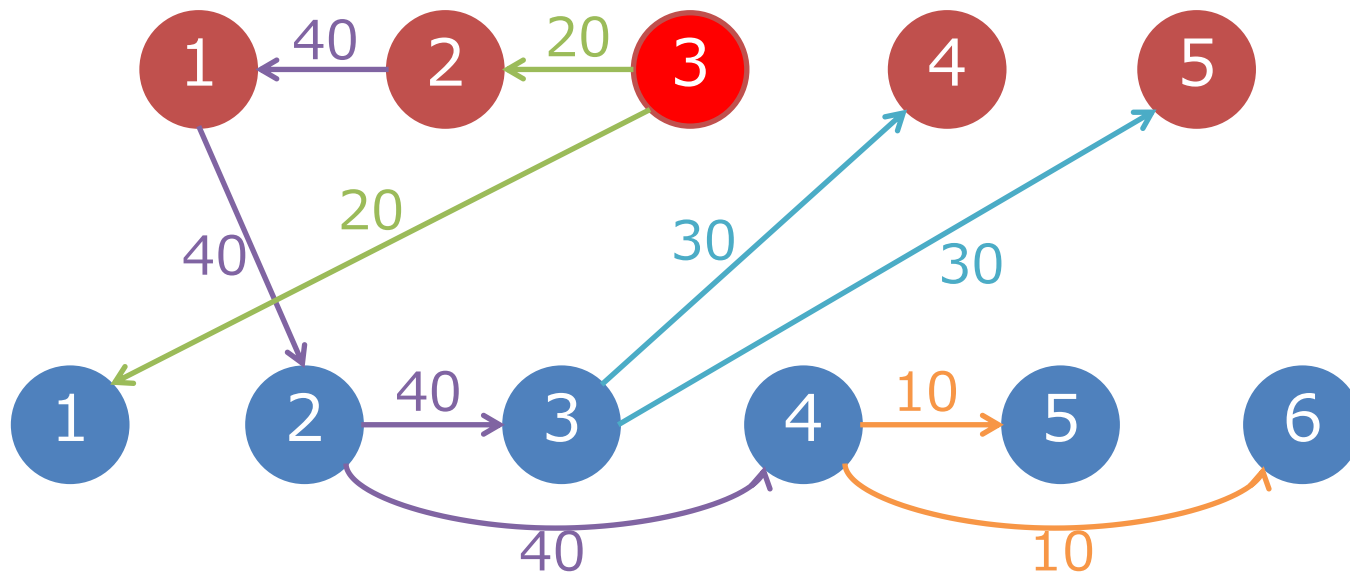
アイデア

- 誰のおかげで招待できたか？



アイデア

- 誰のおかげで招待できたか？



アイデア

- 誰のおかげで招待できたか？
- 全域木ができている
 - コストが対称なので無向グラフ

アイデア

- 問題文のアルゴリズム： 1 頂点から始めて，現在最大コストでとれる頂点を次々と繋げていく
- これは **Prim 法** そのまま
 - **最大全域木** が求まっている
 - C は関係ない
 - 番号の小さい方を優先， も関係ない

最大全域木

- 最大全域木のコストは **Kruskal 法**でも同じ答が求まる
- Kruskal 法
 - 辺をコストの大きい方からソート
 - 辺を順番に見ていき, 異なる連結成分を繋いでいけば使う

最大全域木

- 仲良しグループを T_i の降順にソート
- グループには $O((\text{動物の数})^2)$ 本の辺があるが、それらを結ぶ $O(\text{動物の数})$ 本の辺だけを使えばよい
 - 犬 P_i と猫 R_i
 - 犬 P_i と犬 x ($P_i + 1 \leq x \leq Q_i$)
 - 猫 R_i と猫 y ($R_i + 1 \leq y \leq S_i$)

最大全域木

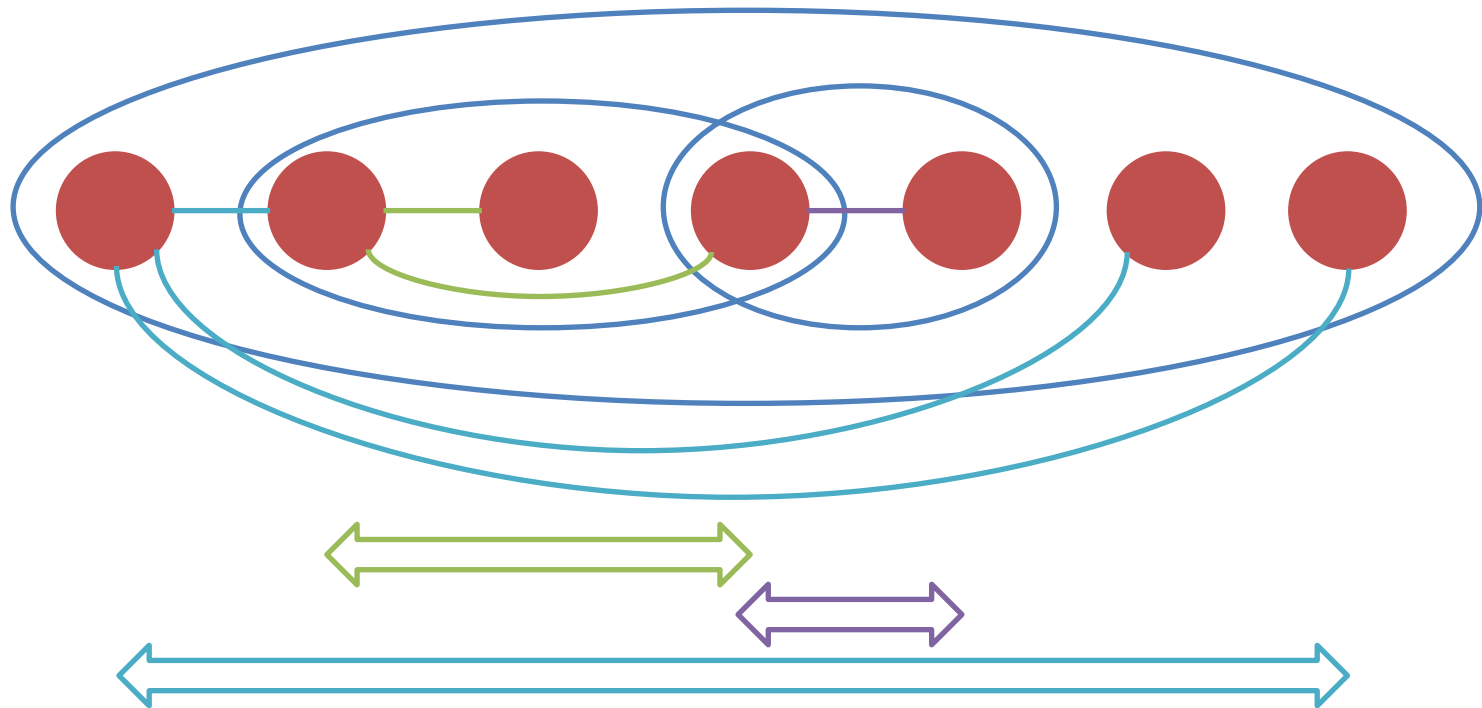
- 辺を繋ぐ処理は Union-Find を使う
- $O(N (A + B) \alpha(A + B))$
 - α : Ackermann 関数の逆
 - 定数と思ってよい
 - 30 点

座標圧縮

- この方針なら座標圧縮は比較的容易
- 犬と猫を $O(N)$ 個の区間にわけ
- 区間たちを Union-Find で繋いでいく
- 各区間は, 最初に触れたときに内部を全部結ぶ
- $O(N^2)$
 - 50 点

高速化

- 各グループの辺を繋ぐのに $O(N)$ かかるのが無駄

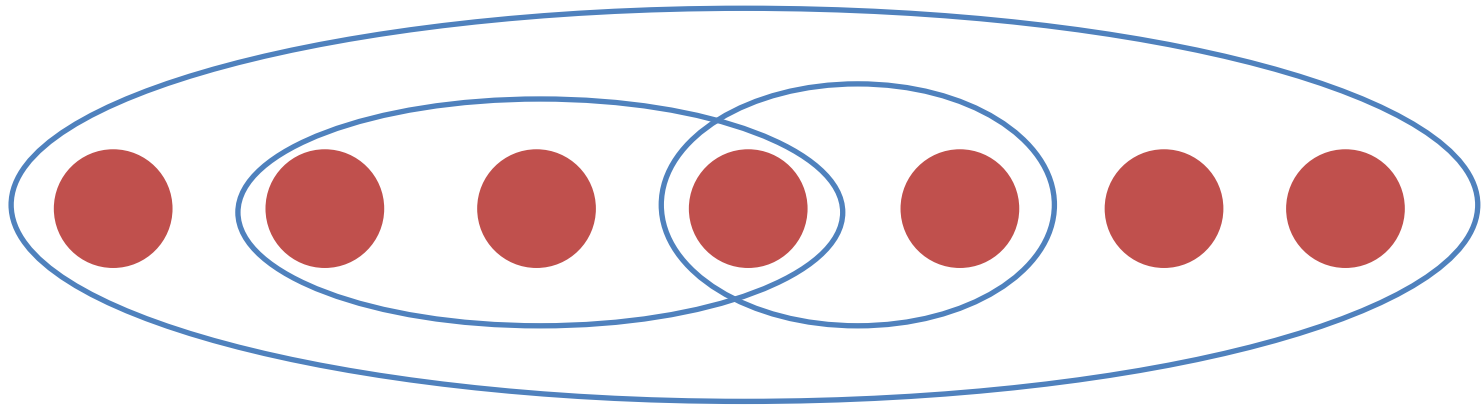


高速化

- 犬と猫の間の辺は 1 本でよいので実際に繋ぐ
- 犬どうし, 猫どうしの辺で実際に繋ぐ必要がある辺は全部の合計は $O(N)$ 本
- 区間を繋げていくとき, すでに繋がっている部分はどんどんスキップしたい

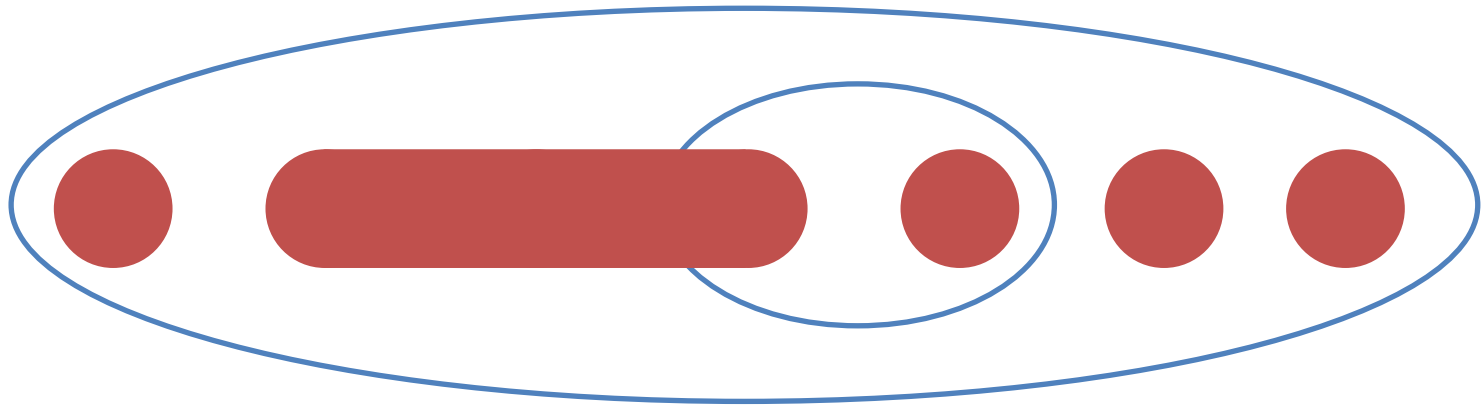
高速化

- 繋ぐたびに 1 つにしていく



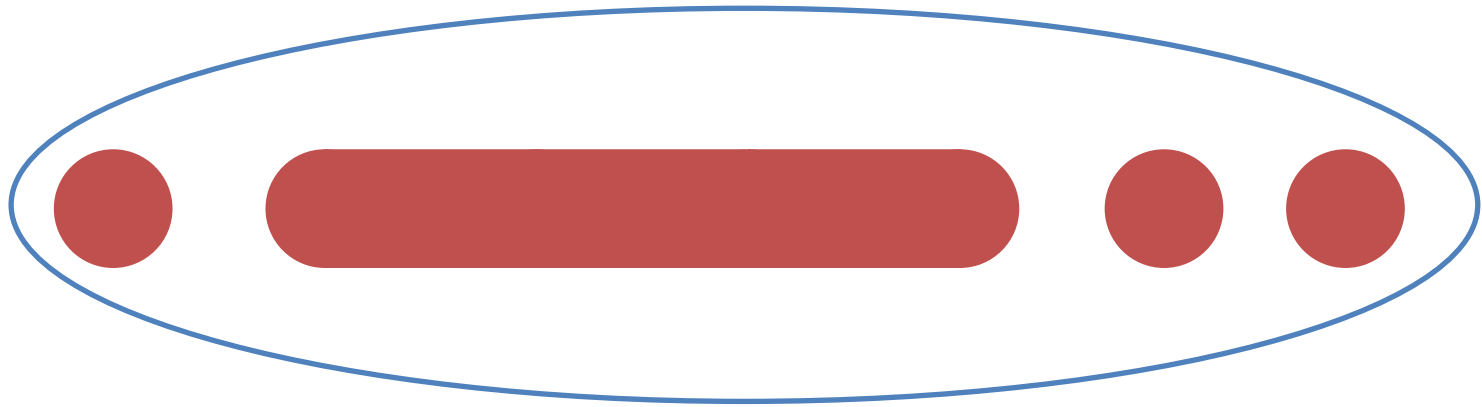
高速化

- 繋ぐたびに 1 つにしていく



高速化

- 繋ぐたびに 1 つにしていく



高速化

- 繋ぐたびに 1 つにしていく



高速化

- 繋ぐたびに 1 つにしていく
- 実現方法
 - 線形リストを使う
 - ちょっと複雑だが高速
 - `std::set` を使う
 - 実装は簡単
 - どうせ $O(N \log N)$ はかかるので

高速化

- 全体で $O(N \log N)$
 - 線形リストを使った場合は最初のソート以外は $O(N)$
 - 100 点

得点分布

